人 间 铸 剑 录

Go 后端与 Harness AI 工程

十二人群像的二十四场工程问剑

全书
图解



图像负责气韵，代码负责证据，版式负责记忆。

人物先立住，对话才有分量

人物不是装饰：有人讲全局，有人追边界，有人验事实，也有人用最小的问题戳破最大的漏洞。


阿良


齐静春


陈平安


宁姚


崔瀺


左右


李宝瓶


阮秀


剑灵


陆沉


顾璨


崔东山

- **阿良**
  全局与比喻；斗笠、养剑葫与黄狗
- **陈平安**
  边界、所有权、复盘
- **崔瀺**
  架构演进、治理、状态代价
- **李宝瓶**
  语法、笔记、跨语言
- **剑灵 · 老剑条**
  确定性护栏、工具执行、门禁
- **顾璨**
  失败路径、技术债、错误选择
- **齐静春**
  规格、契约、长期主义
- **宁姚**
  证据、安全、纠错、验收
- **左右**
  编译、类型、lint、review
- **阮秀**
  持久化、事务、数据完整性
- **陆沉**
  抽象、Context、Agent
- **崔东山**
  Agent 编排、状态机、恢复

全书视觉约束

暖色国漫 3D · 靛青、玉绿与柔粉矿物色 · 图片无对白文字 · 技术图独占大页 · 角色形象连续

修行路线

目录卷一：Go 从一行走到并发

01 第一章 路边茶棚：读懂一段 Go 8

02 第二章 铸出账单剑模：类型、Struct 与 Tag 21

03 第三章 剑匣、窗口与名册：Array、Slice、Map 与指针 35

04 第四章 谁会这一剑：Method、Interface 与组合 45

05 第五章 山门与院落：Module、Package 与依赖方向 56

06 第六章 错误要留因果：Error 与边界测试 67

07 第七章 众剑齐出：Goroutine、GMP 与 Race 82

08 第八章 飞剑须能收：Channel、Select 与 Context 93

修行路线

目录卷二：后端从请求走到上线

09 第九章 山门来客：HTTP、Gin 与契约 103

10 第十章 城中三层：Handler、Service、Repository 132

11 第十一章 真账本：SQL、事务与幂等 141

12 第十二章 两座库房：Redis 与缓存一致性 159

13 第十三章 山洪过窄桥：并发闸门与过载拒绝 166

14 第十四章 流光传书：SSE、断线与跨层取消 177

15 第十五章 夜行留灯：Log、Metric 与 Trace 187

16 第十六章 下山之前：配置、Docker、迁移与回滚 201

修行路线

目录卷三：Harness 管住模型不确定性

17 第十七章 会说不等于会做：LLM、Workflow 与 Tool Calling 224

18 第十八章 大脑、招式、驿道与剑匣 229

19 第十九章 一次执行的一生：Run、Step 与恢复 239

20 第二十章 行囊不能装整座山：Context Engineering 255

21 第二十一章 剑有护手：权限、沙箱与人在回路 260

22 第二十二章 失败要有证词：预算、Trace、重放与 Eval 274

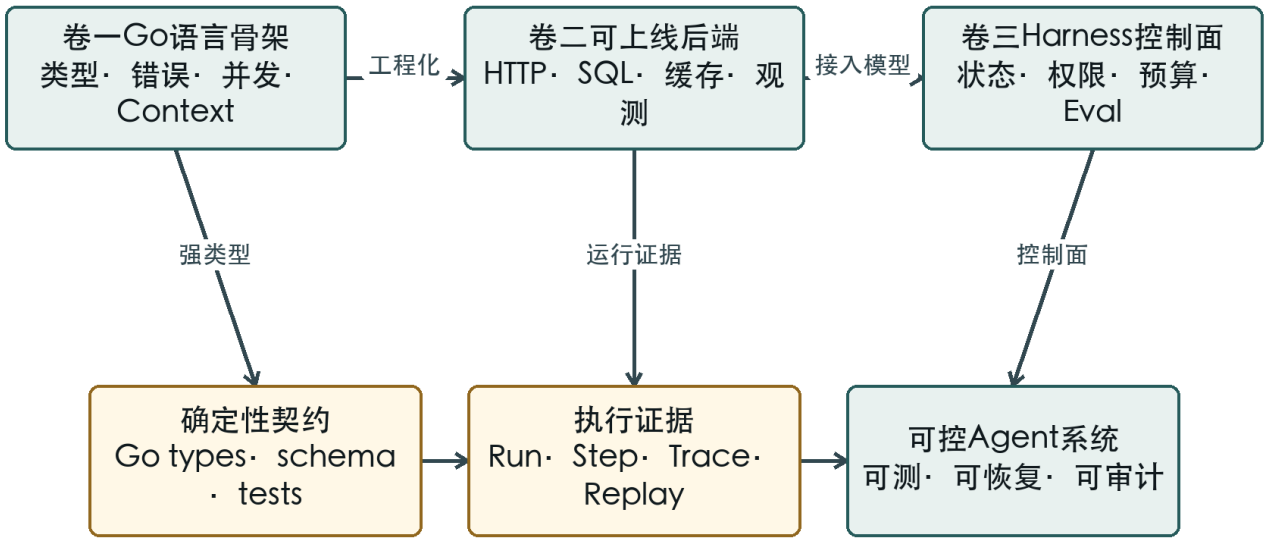
23 第二十三章 AI 写代码也要过剑阵：Coding Harness 292

24 第二十四章 最后一炉：可控 Ledger Agent 301

先见全局

三卷二十四章，最后汇成一条可控执行链

卷一建立 Go 语言骨架，卷二把它变成可上线后端，卷三用 Harness 管住模型的不确定性。



卷一

语法、类型、接口、错误、并发

卷二

HTTP、事务、缓存、容量、观测、部署

卷三

Tool、MCP、Run、权限、评测、Coding Harness

贯穿全书的验收问题

它能运行吗？失败会收口吗？状态能恢复吗？结果能证明吗？



卷一

Go 从一行到并发

从读懂一段函数开始，建立类型、接口、错误与并发的肌肉记忆。

第 01-08 章

第 01 场 · 闯祸与群聊

第一章 路边茶棚：读懂一段 Go

目标：用一次七文结账读懂 package、main、func、输入、返回值、守门条件、error 和表驱动测试。

闯祸现场

山雨刚停，陈平安、宁姚和阿良钻进路边茶棚。一碗面七文，陈平安递出十文，掌柜收钱、端面，转身就走。

阿良没有动筷子，只摊开空手。桌下那条戴斗笠的黄狗也抬起头，等着看那三文找零究竟去了哪里。



陈平安：面都端来了，你还等什么？



阿良：等三文找零。它准备留在掌柜手里修炼？



宁姚：你写的结账函数只收钱，没有把找零交回来。



陈平安：那它接收价格和付款，成功返回找零。



周米粒：要是钱不够呢？总不能端半碗面。



宁姚：返回错误。成功给结果，失败说原因。

收束

阿良拿回三文才动筷子：“面可以坨，账不能糊。”

先有画面，再落到工程

第一章 路边茶棚：读懂一段 Go

先懂一句

函数要说清收什么、成功交回什么、失败怎么讲。

直觉类比

像茶棚结账：先说清掌柜收到了价格和付款，再说成功应找回多少钱；若钱不够，也要明确告诉来客，而不是让他猜。

术语落地

- package：代码住在哪个院子
- error：事情没办成时一起带回来的原因条

读代码抓手

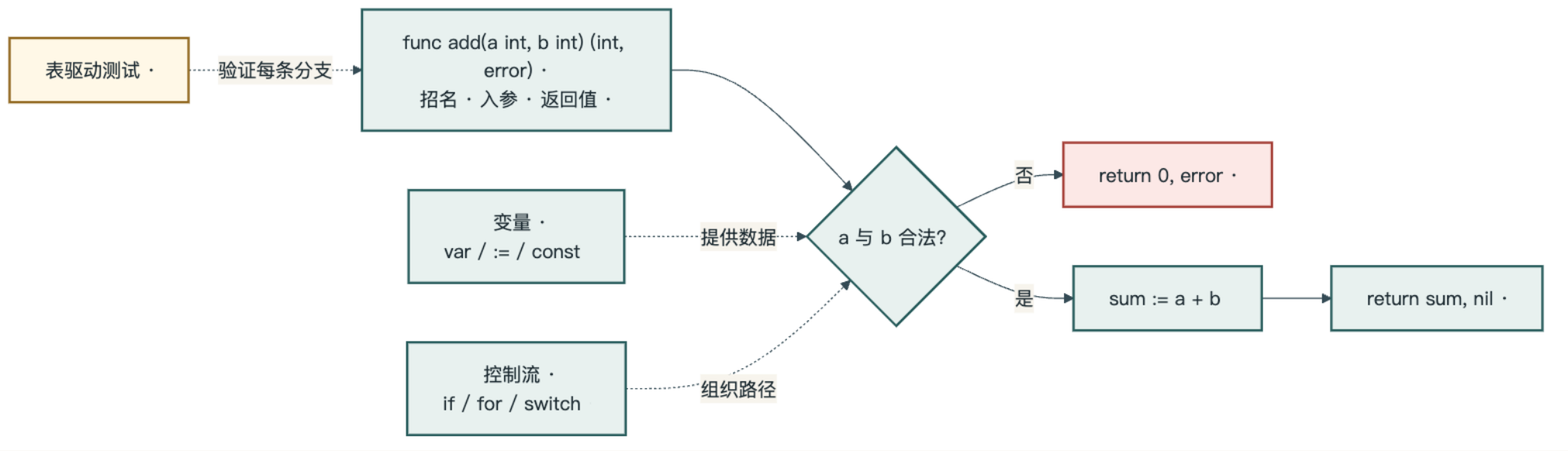
- 先看 settle 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 settle 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 1.1

Go 函数的输入、动作、返回与错误路径

小白先懂：函数要说清收什么、成功交回什么、失败怎么讲。



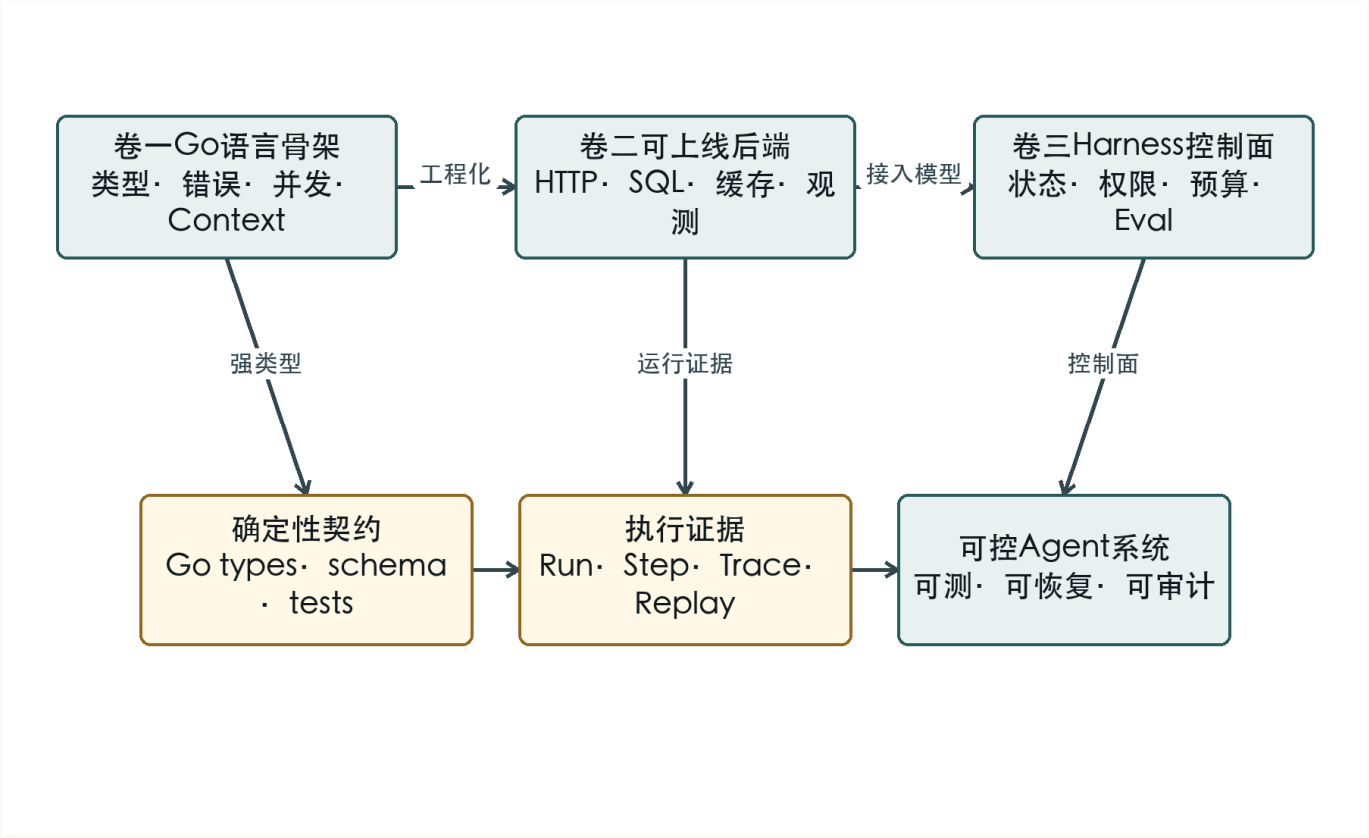
读图要点

- 文件从 package 开始；可执行程序属于 package main，并提供 func main。
- 读函数先看签名：它接收什么参数，成功返回什么，失败如何返回 error。
- 守门条件先处理非法价格和付款不足，让正常路径保持简短。

图 1.2

全书能力地图：从 Go 骨架到 Harness 控制面

小白先懂：函数要说清收什么、成功交回什么、失败怎么讲。



拆去比喻

- 学习顺序从语言边界走向后端可靠性，再进入 Agent 控制面。
- 每一阶段都以失败收口、状态恢复、权限拒绝和证据复核为门槛。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
var (
    ErrInvalidPrice = errors.New("invalid price")
    ErrInsufficientPayment = errors.New("insufficient payment")
)

func settle(price, paid int64) (change int64, err error) {
    if price <= 0 {
        return 0, ErrInvalidPrice
    }
    if paid < price {
        return 0, ErrInsufficientPayment
    }
    return paid - price, nil
}
```

失败路径

- 只看函数内部，不先看输入和返回值
- 把付款不足当成正常找零零元
- 只测刚好付款，不测找零、钱不够和非法价格

练剑任务

为 settle 写表驱动测试，覆盖刚好付款、需要找零、付款不足和非法价格。

验收证据

7 文价格付 10 文稳定返回 3；付款不足返回 ErrInsufficientPayment；非法价格返回 ErrInvalidPrice。

收剑自检

- [ ] main 为什么不会被其他包自动调用？
- [ ] `var x int` 的 x 是什么值？
- [ ] 什么时候不能使用 `:=`？

章内续问 1.1 · 闯祸与群聊

先把分支走明白

闯祸现场　茶棚掌柜把付款与价格的差额写成三张纸：小于零、刚好为零、大于零。李宝瓶用三支不同颜色的笔标出分支，陈平安却把 default 圈得最大。

付款不足被塞进 default 后，错误沿着正常路径继续执行；跨分支的 result 又被 := 重新声明，外层变量仍保持零值。

陈平安



剩下的情况都进 default，不是最省代码吗？



李宝瓶

零是业务状态还是漏网情况，为什么不先说清？

陈平安



我先用 if 拦非法值，再用 switch 列零和正数。



阮秀

跨分支还要读的 result，继续用 := 会发生什么？



宁姚

先用 var 定作用域；Go 的循环统一用 for。

收束　三支笔各守一条路，default 终于不用承包所有意外。

先有画面，再落到工程

先把分支走明白

先懂一句

先让业务状态决定分支，再认识 if、switch、for 和声明的职责。

直觉类比

像过马路：红灯、黄灯、绿灯要先分清，不能把红灯塞进‘其他情况’；if 先拦危险，switch 再把明确状态送到各自路口。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 classify 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 最后核对 classify 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 1.1 · 拆去比喻

先把分支走明白

工程要点

- 用 if 处理守卫条件，先返回错误，缩短正常路径。
- switch 可省略表达式，适合互斥且有命名意义的条件。
- for 同时承担计数、条件循环和无限循环三种形态。
- 短声明只用于局部新变量；跨分支结果用 var 明确作用域。

练手

给 classify 增加一个上限分支，并故意制造一次 := 遮蔽后修正。

验收

测试覆盖负数、零、正常值和上限外值；正常路径无变量遮蔽。

```
type BillID string

func classify(n int64) string {
    switch {
    case n < 0:
        return "invalid"
    case n == 0:
        return "zero"
    default:
        return "positive"
    }
}
```

章内续问 1.2 · 闯祸与群聊

测试要住进 _test.go

闯祸现场

桌上摆着七文面钱、三文找零和一张空白的 settle_test.go，文件名后半截被荧光笔涂亮。李宝瓶把四种付款场景写成四张小票。

陈平安删掉找零后，正文示例仍安静得像没事发生；因为它根本不在 _test.go 里，go test ./... 没发现那三文又失踪了。

陈平安



示例看着能跑，为什么还要单独建测试文件？



李宝瓶

七文付十文，谁保证下次仍找回三文？

陈平安



我先断言找零三文，再整理成表驱动用例。



阮秀

钱不够和价格非法，能只认成同一种错吗？



宁姚

不能。用 errors.Is 分别识别两类错误。

收束

三文找零压住测试文件，这回再想失踪也会先撞上红灯。

先有画面，再落到工程

测试要住进 _test.go

先懂一句

先写一个能失败的普通测试，重复出现后再重构为表驱动。

直觉类比

像给玩具做耐摔测试：先写清怎么摔、摔几次、什么算坏，再真的摔。只说‘我昨天玩过，没坏’不算测试。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 TestSettle 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 TestSettle 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 1.2 · 拆去比喻

测试要住进 _test.go

工程要点

- 生产文件与测试文件同包放置，文件名必须以 _test.go 结尾。
- 表驱动测试把输入、期望和用例名放在同一处。
- t.Run 让失败定位到具体场景，不只停在测试函数名。
- 错误路径要断言错误类别或条件，不能只看返回值。

练手

新建 settle_test.go，用 TestSettle 覆盖刚好付款、三文找零、付款不足和非法价格。

验收

go test ./... 能发现四个场景；成功路径核对找零，失败路径分别识别 ErrInsufficientPayment 与 ErrInvalidPrice。

记忆钉

1. 生产文件与测试文件同包放置，文件名必须以 _test.go 结尾。
2. 表驱动测试把输入、期望和用例名放在同一处。
3. t.Run 让失败定位到具体场景，不只停在测试函数名。
4. 错误路径要断言错误类别或条件，不能只看返回值。

长代码已拆到后续专页

章内续问 1.2 · 代码 1/1

测试要住进 _test.go

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```go
func TestSettle(t *testing.T) {
    tests := []struct {
        name        string
        price, paid int64
        wantChange  int64
        wantErr     error
    }{
        {"exact", 7, 7, 0, nil},
        {"change", 7, 10, 3, nil},
        {"not enough", 7, 5, 0, ErrInsufficientPayment},
        {"invalid price", 0, 10, 0, ErrInvalidPrice},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            got, err := settle(tt.price, tt.paid)
            if !errors.Is(err, tt.wantErr) {
                t.Fatalf("err=%v, want %v", err, tt.wantErr)
            }
            if got != tt.wantChange {
                t.Fatalf("change=%d, want %d", got, tt.wantChange)
            }
        })
    }
}
```

go test ./... 能发现四个场景；成功路径核对找零，失败路径分别识别 ErrInsufficientPayment 与 ErrInvalidPrice。

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 main 为什么不会被其他包自动调用？

陈平安



main 只由 Go 运行时作为可执行程序入口调用，import 不会自动执行它。



宁姚

边界闭合。

第 2 问 `var x int` 的 x 是什么值？

陈平安



var x int 的零值是 0。



宁姚

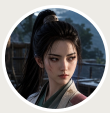
边界闭合。

第 3 问 什么时候不能使用 `:=`？

陈平安



:= 不能用于包级作用域，也不能在当前作用域没有新变量时使用。



宁姚

边界闭合。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 02 场 · 闯祸与群聊

第二章 铸出账单剑模：类型、Struct 与 Tag

目标：理解基本类型、零值、struct、字段可见性、JSON tag，以及 DTO、领域模型和持久化模型的边界。

闯祸现场

午后的会议桌上放着一张 1500 分的奶茶小票，旁边是 CreateBillRequest 的 JSON 和一支红色记号笔。李宝瓶在 msg_content 下画了两道线。

陈平安明明给 Message 填了“周末吃饭”，输出里却没有这个字段；他把反引号里的 Tag 当成了注释，小写 message 也让编码器根本看不见。

陈平安



字段有值，编码器应该自己猜到名字吧？



李宝瓶

Tag 已写着 msg_content，它为什么要猜另一个名字？

陈平安



我按 Tag 查 key，并把 Message 保持为导出字段。



阮秀

请求 DTO 还能让客户端直接填写 user_id 吗？



宁姚

不能。身份来自认证结果，零值也要过领域校验。

收束

红笔最后圈住的不是编码器，而是那枚被当成装饰的小小 Tag。

先有画面，再落到工程

第二章 铸出账单剑模：类型、Struct 与 Tag

先懂一句

Struct 是剑模，Tag 是模具刻字，规则决定能否出炉。

直觉类比

像做冰格。Struct 决定格子长什么样，Tag 决定快递员看到的格子名字；模具刻错一个字，冰块没坏，快递单却会送错门。

术语落地

- struct：把一件东西要填的格子做成固定模具

读代码抓手

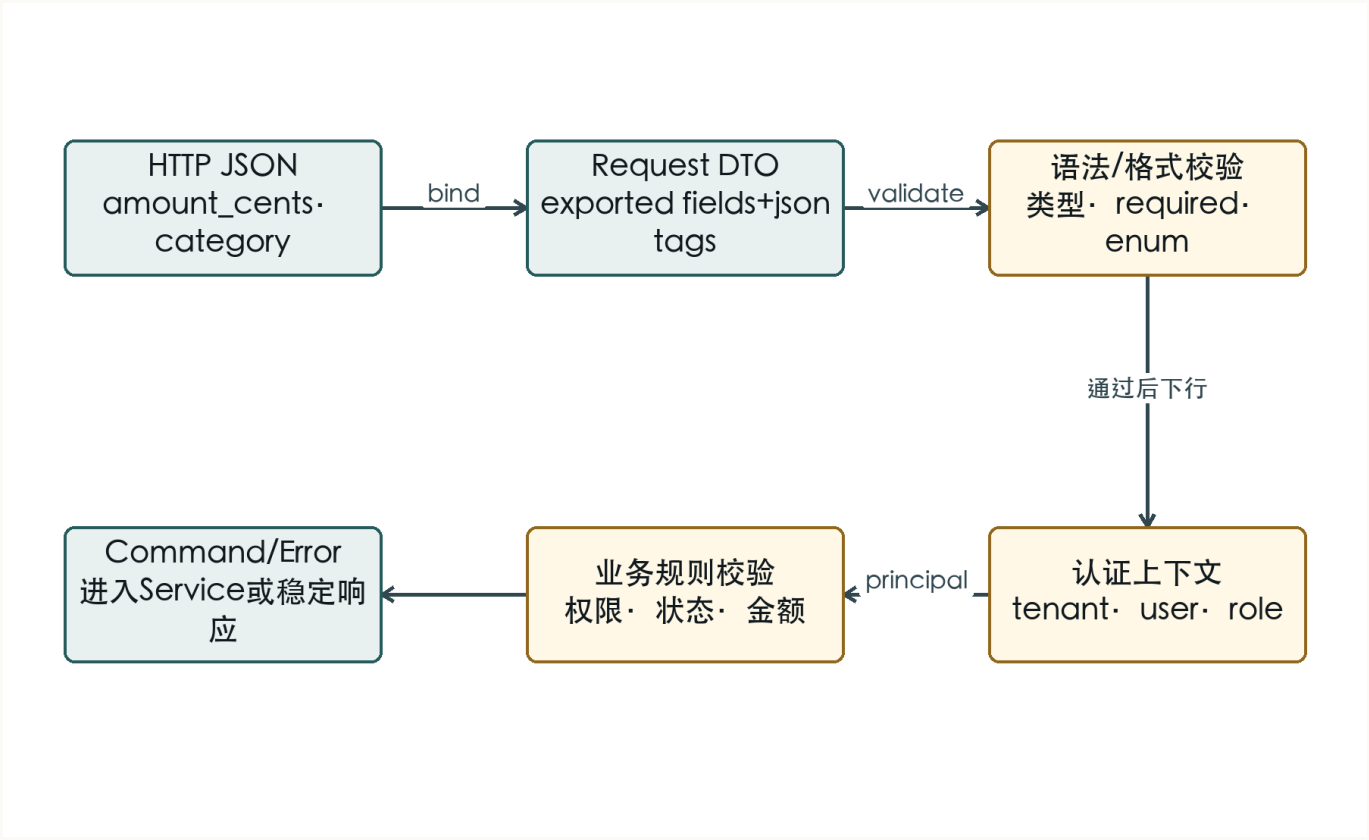
- 先看 Validate 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Validate 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 2.1

Struct、JSON 与请求数据管线

小白先懂：Struct 是剑模，Tag 是模具刻字，规则决定能否出炉。



拆去比喻

- 导出标识符以大写开头；encoding/json 默认只处理可导出字段。
- tag 是元数据，不是运行时魔法；`json`、`binding`、`db` 各服务于不同边界。
- 零值让对象可被创建，但零值未必业务合法，金额、分类和租户仍需领域校验。
- 请求 DTO 不应直接等同于数据库模型，避免客户端控制内部 ID、user_id 和审计字段。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
type CreateBillRequest struct {
    AmountCents *int64 `json:"amount_cents"`
    Category    string `json:"category"`
}

func (r CreateBillRequest) Validate() error {
    if r.AmountCents == nil || *r.AmountCents <= 0 {
        return ErrInvalidAmount
    }
    if r.Category == "" {
        return ErrCategoryRequired
    }
    return nil
}
```

失败路径

- 金额使用 float64
- 把数据库模型直接暴露给客户端
- 以为 binding 通过就代表业务合法

练剑任务

把 CreateBillRequest 映射为 Bill，并测试字段缺失、金额非法和成功三种情况。

验收证据

能说明 JSON 校验与领域校验分别归谁负责。

收剑自检

- [ ] 为什么小写字段不会出现在 JSON？
- [ ] 零值有什么好处和风险？
- [ ] 为什么 user_id 不信任请求体？

章内续问 2.1 · 闯祸与群聊

一个字不一定一个字节

闯祸现场　午休前，白板上写着“周末吃饭”，旁边摆着两格透明尺和一个表情贴纸。陈平安说只要取前两个位置就够了。

note[:2] 切出的却是两段残缺字节，终端立刻显示乱码；他这才发现 string 和 len 数的是字节，不是眼睛看到的两个字。

陈平安



我要前两个字，直接切两个位置为什么不行？



李宝瓶

你说的两个，是字节、码点，还是用户看到的字符？

陈平安

我先验 UTF-8，再转 []rune 取两个码点。





阮秀

遇到组合表情，两个 rune 就等于两个字符吗？



宁姚

不等于。字素簇要另写清产品边界。

收束　那张表情贴纸被留在尺外，提醒大家别把眼睛和字节混为一谈。

先有画面，再落到工程

一个字不一定一个字节

先懂一句

不另开一门字符课，只修复同一张账单的备注失败。

直觉类比

像数积木上的汉字：人眼看见一个字，盒子里可能占了好几个小格。按字节数盒子，和按字符数盒子，不是同一道题。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 inspect 的输入和返回值：它接什么，最后必须交回什么。
- 最后核对 inspect 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 2.1 · 拆去比喻

一个字不一定一个字节

工程要点

- string 是只读字节序列，可以含任意字节，不保证是合法 UTF-8。
- len(s) 返回字节数；utf8.RuneCountInString 返回 rune 数。
- range 遇到非法 UTF-8 时产生 RuneError，并按规则前进。
- 按用户可见字符截断还涉及字素簇，rune 也不等于视觉字符。

练手

比较“Go”“中国”和一个非法 UTF-8 字节串的三项结果。

验收

能解释字节数与 rune 数的差异，并在切片前验证边界或先转 []rune。

```go
func inspect(s string) (int, int, bool) {
    bytes := len(s)
    runes := utf8.RuneCountInString(s)
    valid := utf8.ValidString(s)
    return bytes, runes, valid
}
```

章内续问 2.2 · 闯祸与群聊

JSON 解码要有边界

闯祸现场

评审桌上放着两张 JSON 小票：一张把 amount_cents 拼错，另一张把金额写了两次。宁姚把解码结果贴到小票下方。

Decode 两次都没有按预期报错，未知字段被忽略，重复 key 的后值覆盖前值；尾部还藏着第二个 JSON，第一遍读取后没人继续检查。

陈平安



能 Decode 成结构体，不就代表请求有效吗？



李宝瓶

拼错字段也沉默通过，账单金额到底来自哪一格？

陈平安



我拒绝未知字段，再读一次确认 io.EOF。



阮秀

缺失、null、零值和重复 key 还会混在一起吗？



宁姚

正金额单独校验；三态显式标记，重复 key 拒绝。

收束

两张小票被订在一起，解码器这回不能只看第一张就下班。

先有画面，再落到工程

JSON 解码要有边界

先懂一句

字段能反序列化只是起点，对外请求还要形成严格边界。

直觉类比

像收快递单：字写得像不像只是第一关。多写陌生字段、同一栏写两遍、箱子后面又偷偷绑一个箱子，都应该在门口说清楚。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 decodeBill 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 decodeBill 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 2.2 · 拆去比喻

JSON 解码要有边界

工程要点

- 对外输入拒绝未知字段、尾随 JSON，并在 token 层拒绝重复或仅大小写不同的 key。
- *int64 只能把缺失与 null 合并为 nil，再与零和负数一起按正金额契约拒绝。
- 若必须区分 missing、null、value 三态，使用带 Present/Null 标记并实现 UnmarshalJSON 的类型。
- 重复键策略须在网关、服务和签名规范化层一致，不能依赖 encoding/json 的后值覆盖。

练手

测试缺失、null、零、负数、正数、未知字段、尾随 JSON、重复 amount_cents、大小写变体和嵌套重复 key。

验收

仅正金额请求通过；缺失与 null 同样拒绝；未知、尾随及任意层重复或大小写碰撞 key 均失败。

记忆钉

1. 对外输入拒绝未知字段、尾随 JSON，并在 token 层拒绝重复或仅大小写不同的 key。
2. *int64 只能把缺失与 null 合并为 nil，再与零和负数一起按正金额契约拒绝。
3. 若必须区分 missing、null、value 三态，使用带 Present/Null 标记并实现 UnmarshalJSON 的类型。
4. 重复键策略须在网关、服务和签名规范化层一致，不能依赖 encoding/json 的后值覆盖。

长代码已拆到后续专页

章内续问 2.2 · 代码 1/3

JSON 解码要有边界

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```go
type CreateBillRequest struct {
    AmountCents *int64 `json:"amount_cents"`
}

func decodeBill(r io.Reader) (CreateBillRequest, error) {
    data, err := io.ReadAll(io.LimitReader(r, 1<<20+1))
    if err != nil {
        return CreateBillRequest{}, fmt.Errorf("read body: %w", err)
    }
    if len(data) > 1<<20 {
        return CreateBillRequest{}, ErrBodyTooLarge
    }
    if err := rejectDuplicateKeys(data); err != nil {
        return CreateBillRequest{}, err
    }
    dec := json.NewDecoder(bytes.NewReader(data))
    dec.DisallowUnknownFields()
    var v CreateBillRequest
    if err := dec.Decode(&v); err != nil { return v, err }
    if err := dec.Decode(&struct{}{}); err != io.EOF {
        return v, ErrTrailingJSON
    }
    if v.AmountCents == nil || *v.AmountCents <= 0 {
        return v, ErrInvalidAmount
```

下一页继续同一段代码，不切换事故对象。

章内续问 2.2 · 代码 2/3

JSON 解码要有边界

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
25      }
26      return v, nil
27  }
28
29  func rejectDuplicateKeys(data []byte) error {
30      dec := json.NewDecoder(bytes.NewReader(data))
31      if err := scanJSONValue(dec); err != nil { return err }
32      if _, err := dec.Token(); err != io.EOF { return ErrTrailingJSON }
33      return nil
34  }
35
36  func scanJSONValue(dec *json.Decoder) error {
37      tok, err := dec.Token()
38      if err != nil { return err }
39      d, ok := tok.(json.Delim)
40      if !ok { return nil }
41      switch d {
42      case '{':
43          seen := map[string]struct{}{}
44          for dec.More() {
45              keyToken, err := dec.Token()
46              if err != nil { return err }
47              key, ok := keyToken.(string)
48              if !ok { return ErrInvalidJSON }
```

下一页继续同一段代码，不切换事故对象。

章内续问 2.2 · 代码 3/3

JSON 解码要有边界

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
            folded := strings.ToLower(key)
            if _, exists := seen[folded]; exists {
                return ErrDuplicateKey
            }
            seen[folded] = struct{}{}
            if err := scanJSONValue(dec); err != nil { return err }
        }
    case '[':
        for dec.More() {
            if err := scanJSONValue(dec); err != nil { return err }
        }
    default:
        return ErrInvalidJSON
    }
    _, err = dec.Token()
    return err
}
```

仅正金额请求通过；缺失与 null 同样拒绝；未知、尾随及任意层重复或大小写碰撞 key 均失败。

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 为什么小写字段不会出现在 JSON？

陈平安



小写字段未导出，encoding/json 在包外看不见它。



宁姚

边界闭合。

第 2 问 零值有什么好处和风险？

陈平安



零值便于直接创建对象，但 0 金额等零值可能不满足业务规则。



宁姚

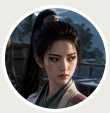
边界闭合。

第 3 问 为什么 user_id 不信任请求体？

陈平安



user_id 必须来自服务端认证结果，不能信任请求体自报。



宁姚

边界闭合。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 03 场 · 闯祸与群聊

第三章 剑匣、窗口与名册：Array、Slice、Map 与指针

目标：分清 array、slice、map、指针、复制与共享，并能用 race detector 发现并发读写问题。

闯祸现场

窗边晾着一卷十米长的标签纸，顾璨只撕下一小段，线头却还连着整卷。桌上的账单列表旁又摆着一把写有 owner 的钥匙。

陈平安把内部 slice 直接交给调用方，对方 append 一笔后，账本原数据也变了；另一段代码还同时写普通 map，race 检查立刻亮红。

陈平安



slice 传出去不是已经复制了吗？



阮秀

复制的是指针、长度和容量，底层数组归谁？

陈平安



我复制元素再返回，map 输出前固定排序 key。



李宝瓶

nil map 若要写入，谁负责先初始化？



宁姚

所有权先写清；并发访问再用锁或单一 owner 收口。

收束

顾璨剪断那根线头时，十米标签纸终于不用陪一小段数据加班。

先有画面，再落到工程

第三章 剑匣、窗口与名册：Array、Slice、Map 与指针

先懂一句

Array 是固定柜，Slice 是窗口，Map 是名册，指针是门牌地址。

直觉类比

Array 像固定座位表，Slice 像你手里框住的一排座位，Map 像按名字查座位的名册，指针则是写着‘去三楼第二排’的小纸条。

术语落地

- goroutine：很轻的小跑堂，可以同时派出很多个
- race：两个人同时改一本账，谁先落笔说不清

读代码抓手

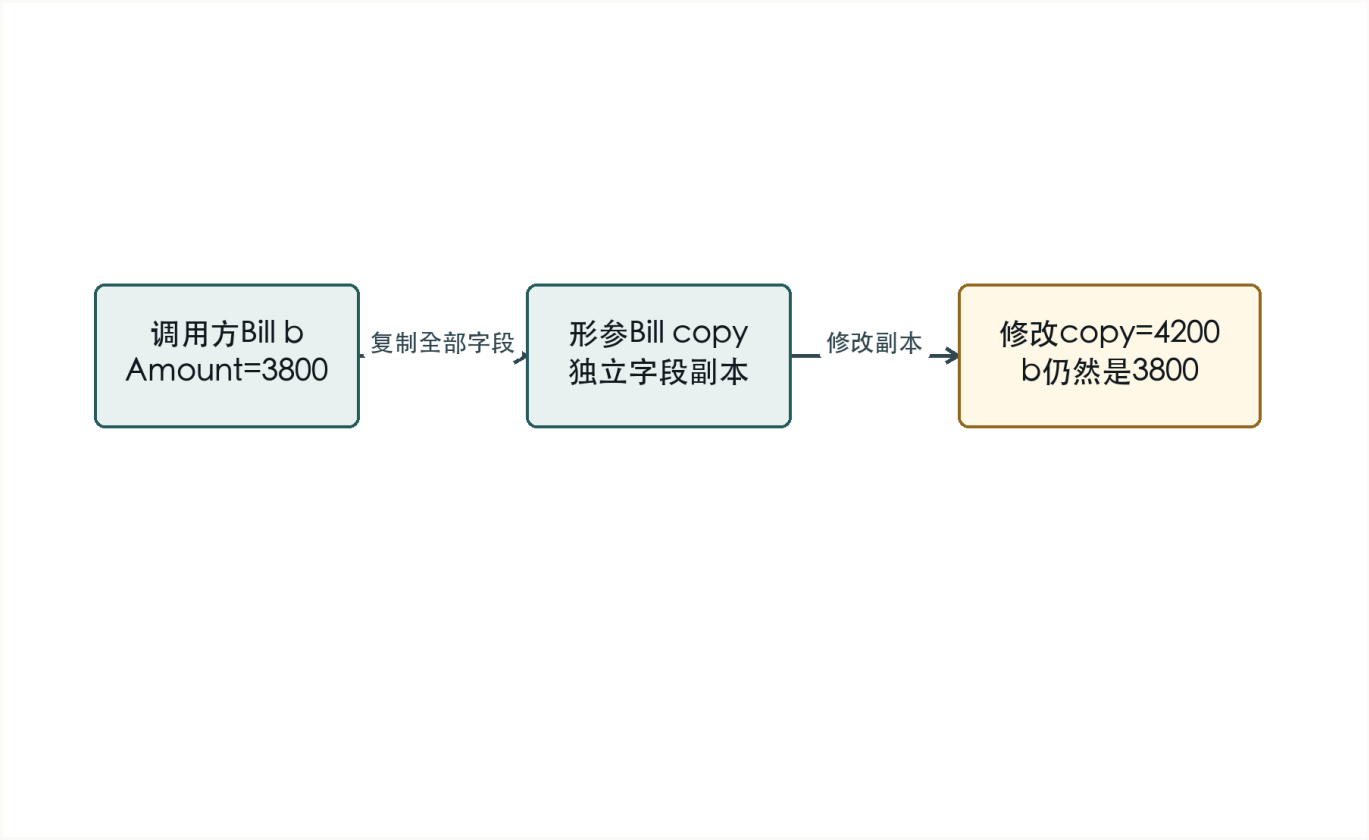
- 先看 NewSafeIndex 的输入和返回值：它接什么，最后必须交回什么。
- 最后核对 NewSafeIndex 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 3.1

Struct 按值复制：副本与原值各自独立

小白先懂：Array 是固定柜，Slice 是窗口，Map 是名册，指针是门牌地址。



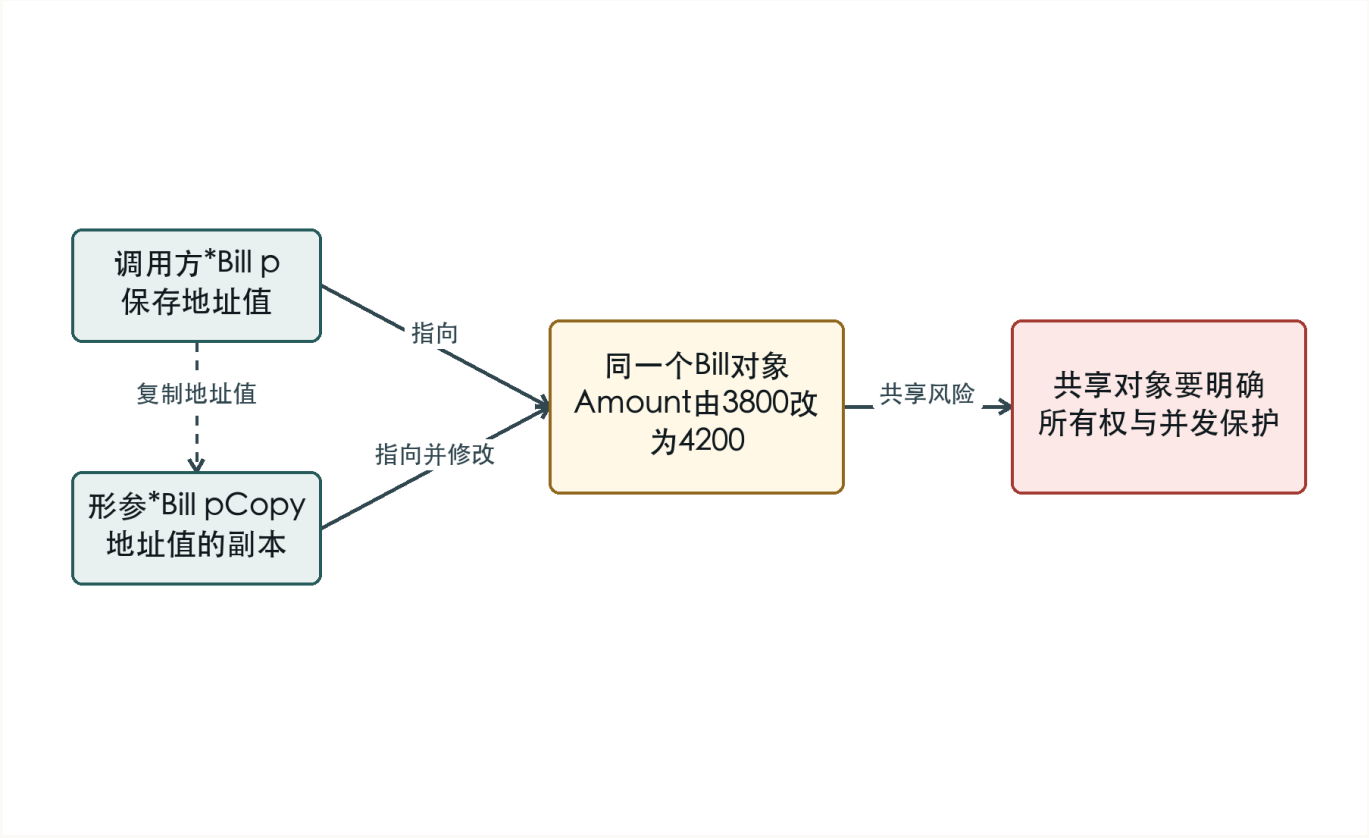
拆去比喻

- array 长度属于类型；slice 是包含指针、长度和容量的描述符，复制描述符不等于复制底层数组。
- append 未超容量时可能修改共享底层数组；扩容后返回的新 slice 才可能指向新数组。
- map 必须初始化后写入；并发读写需要锁、所有权隔离或并发安全结构。
- 含 mutex 的值首次使用后不得复制，通常通过构造函数返回指针并使用指针 receiver。

图 3.2

指针也按值复制：地址副本指向同一对象

小白先懂：Array 是固定柜，Slice 是窗口，Map 是名册，指针是门牌地址。



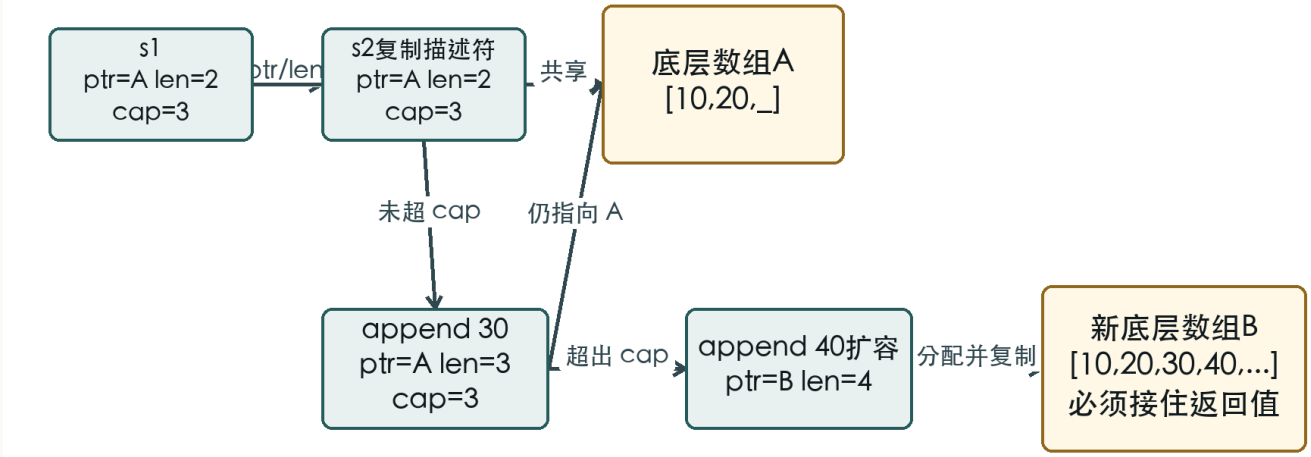
拆去比喻

- 复制指针只复制地址值；两个指针仍可能修改同一对象。
- 共享对象进入并发路径前，必须明确 owner、锁或消息传递边界。

图 3.3

Slice 描述符、底层数组与 append 扩容

小白先懂：Array 是固定柜，Slice 是窗口，Map 是名册，指针是门牌地址。



拆去比喻

- Slice 复制的是 ptr/len/cap 描述符；扩容前可能共享底层数组。
- append 可能返回指向新数组的 Slice，调用方必须接住返回值。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
type SafeIndex struct {
    mu sync.RWMutex
    m  map[string]int
}

func NewSafeIndex() *SafeIndex {
    return &SafeIndex{m: make(map[string]int)}
}
func (s *SafeIndex) Set(k string, v int) {
    s.mu.Lock(); defer s.mu.Unlock(); s.m[k] = v
}
func (s *SafeIndex) Get(k string) (int, bool) {
    s.mu.RLock(); defer s.mu.RUnlock()
    v, ok := s.m[k]; return v, ok
}
```

失败路径

- 把 slice 叫作不复制的引用类型
- 保留小切片导致大数组无法释放
- 多个 goroutine 直接写普通 map

练剑任务

实现 NewSafeIndex、Get、Set，并用多个 goroutine 并发读写。

验收证据

go test -race 通过；空构造不会发生 nil map panic；测试说明含锁对象不被复制。

收剑自检

- [ ] append 何时更换底层数组？
- [ ] 为什么小切片可能留住大内存？
- [ ] map 并发读写如何收口？

章内续问 3.1 · 闯祸与群聊

容器值也有契约

闯祸现场　资料架上有一个标着 null 的空盒和一个标着 [] 的空盒，旁边是一份按 key 排序的账单目录。阮秀把内部列表的复印件压在最上面。

陈平安却把原 slice 直接交出去，调用方 append 后内部账本也多了一笔；普通 map 的输出顺序每次又不同，快照测试连续失败。

陈平安



返回列表就行，nil、空集合和顺序有必要约定吗？



阮秀

客户端看到 null 和 []，会当成同一份契约吗？

陈平安



我返回空集合副本，并在输出 map 前排序 key。



李宝瓶

调用方修改副本，还能碰到内部底层数组吗？



宁姚

不能。所有权与 nil 语义都要用测试写明。

收束　两个空盒终于各贴各的标签，不再轮流扮演同一个返回值。

先有画面，再落到工程

容器值也有契约

先懂一句

nil、顺序和共享底层存储，会共同改变容器 API 的真实含义。

直觉类比

像共享一盒彩笔：你以为只是把颜色表借给别人，对方添一支笔，原盒子也可能跟着变；借出前要说清是否共享、顺序是否重要。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 sortedKeys 的输入和返回值：它接什么，最后必须交回什么。
- 最后核对 sortedKeys 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 3.1 · 拆去比喻

容器值也有契约

工程要点

- nil slice 可读、可 append；nil map 可读但不能写。
- map 遍历顺序未定义，测试和输出都不能依赖它。
- slice 复制只复制描述符，元素仍可能共享底层数组。
- API 应明确返回 nil 还是空集合，以及是否交付所有权。

练手

写一个返回副本的 Names 方法，并比较 nil 与空 slice 的 JSON 输出。

验收

调用方修改返回值不会污染内部状态；排序输出可重复；nil 语义有测试。

```go
func sortedKeys(m map[string]int) []string {
    keys := make([]string, 0, len(m))
    for k := range m {
        keys = append(keys, k)
    }
    sort.Strings(keys)
    return keys
}
```

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 append 何时更换底层数组？

陈平安

append 超过容量时通常分配新数组；未超过时仍可能写入共享底层数组。



宁姚

边界闭合。

第 2 问 为什么小切片可能留住大内存？

陈平安

小切片仍引用原 backing array，因此可能阻止整块大内存回收。



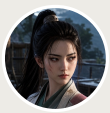
宁姚

边界闭合。

第 3 问 map 并发读写如何收口？

陈平安

并发读写 map 要用锁、单一 owner 或并发安全结构收口。



宁姚

边界闭合。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 04 场 · 闯祸与群聊

第四章 谁会这一剑：Method、Interface 与组合

目标：掌握 receiver、隐式接口满足、依赖倒置、组合和泛型的适用边界。

闯祸现场

白板上贴着 MemoryBillRepo、SQLRepository 和 FakeRepository 三张卡片，旁边只留了 Find 与 Save 两个空位。阿良把一摞写满方法名的纸推到角落。

陈平安找不到 implements 声明，以为 MemoryBillRepo 不能交给 Service，于是准备给所有实现补一套庞大接口，测试也跟着被真实数据库绑住。

陈平安



没有 implements，我是不是得让实现方先登记？



李宝瓶

Service 真正会调用的，究竟是全部方法还是两个动作？

陈平安



我在使用方定义只含 Find、Save 的小接口。



阮秀

值 receiver 和指针 receiver 的方法集一样吗？



宁姚

不一样。先让编译器和 fake 测试证明接口满足。

收束

那摞巨型接口被压在桌脚下，倒是第一次发挥了稳定作用。

先有画面，再落到工程

第四章 谁会这一剑：Method、Interface 与组合

先懂一句

接口按能力立约，不按身份封爵。

直觉类比

班里不问你是谁家的孩子，只问你会不会扫地。会扫地就能加入 Cleaner 接口；这就是按能力组队，不按血统封官。

术语落地

- struct：把一件东西要填的格子做成固定模具
- Service：真正执行业务规矩的账房
- Repository：专门和数据库打交道的库管

读代码抓手

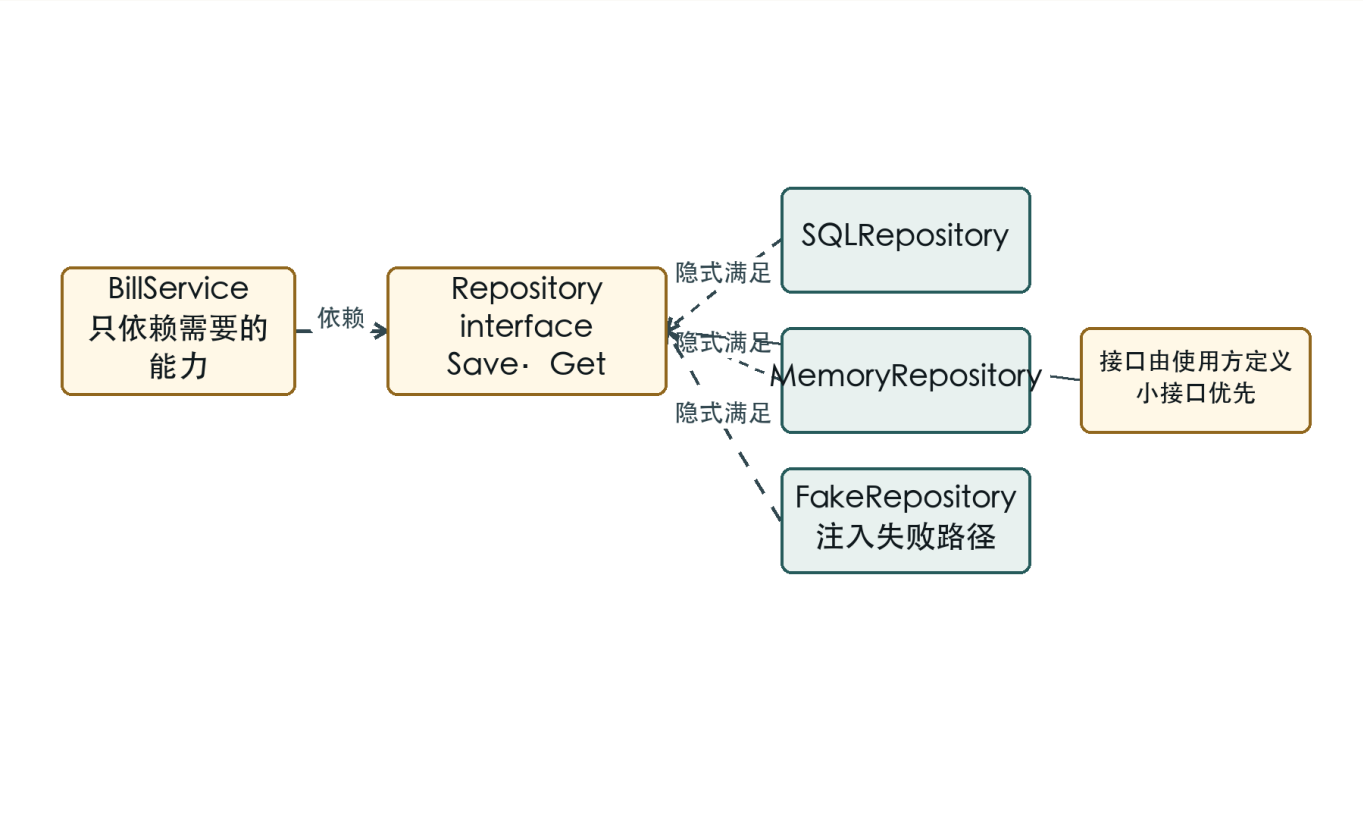
- 先看 NewService 的输入和返回值：它接什么，最后必须交回什么。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 NewService 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 4.1

接口、能力契约与依赖注入

小白先懂：接口按能力立约，不按身份封爵。



拆去比喻

- 接口由使用方按最小能力定义，具体类型通过方法集隐式满足。
- 值 receiver 的方法属于 T 与 *T 的方法集；指针 receiver 的方法只属于 *T。
- 组合复用行为而非模拟继承；嵌入会提升方法，但不自动建立 is-a 关系。
- 泛型适合数据结构和算法复用，不应用于隐藏领域差异或制造抽象层。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
type Repository interface {
    Save(context.Context, Bill) (Bill, error)
    Get(context.Context, string) (Bill, error)
}

type Service struct { repo Repository }
func NewService(repo Repository) *Service { return &Service{repo: repo} }
```

失败路径

- 接口由实现方定义成巨型清单
- 所有 struct 都用指针 receiver
- 为了一处重复就引入泛型

练剑任务

实现 SQLRepository 与 FakeRepository，让同一个 Service 测试无需真实数据库。

验收证据

测试能注入 fake，并验证 repository 错误如何影响业务结果。

收剑自检

- [ ] 接口应该由谁定义？
- [ ] 值与指针 receiver 的方法集有何差异？
- [ ] 组合为何不等于继承？

章内续问 4.1 · 闯祸与群聊

接口里的 nil 有两层

闯祸现场　显示器旁放着一个透明文件袋，袋内写着 *Problem，里面却什么都没有。李宝瓶把它和真正的空袋并排摆好。

陈平安把 nil 的 *Problem 装进 error 接口返回，成功路径仍进入失败分支；接口保留了动态类型，所以它并不等于 nil。

陈平安



里面的指针是 nil，error 为什么还不为空？



李宝瓶

袋子还标着 *Problem，类型这一层消失了吗？

陈平安



我先检查具体指针，成功时直接 return nil。



阮秀

类型不确定时还直接断言，会不会再次 panic？



宁姚

用 v, ok；只有不变量保证类型时才直接断言。

收束　透明文件袋被收进抽屉，空指针总算不再冒充空接口。

先有画面，再落到工程

接口里的 nil 有两层

先懂一句

接口值同时携带动态类型和动态值，不能只看里面的指针是否为空。

直觉类比

像一个贴着‘蛋糕’标签的盒子，里面却是空的。接口先看见标签，就会说‘我不是 nil’；打开盒子才发现里面的指针是 nil。

术语落地

- error：事情没办成时一起带回来的原因条

读代码抓手

- 先看 Error 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Error 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 4.1 · 拆去比喻

接口里的 nil 有两层

工程要点

- 接口值由动态类型和动态值组成；typed nil 仍保留动态类型。
- 返回 error 时不要把 nil 具体指针装进接口后返回。
- 外部数据或可变实现使用 v, ok := x.(T)，不要赌断言成功。
- type switch 适合有限分支；开放扩展优先依靠接口方法。

练手

写一个失败版本返回 p，再用测试证明 err != nil，最后按示例修复。

验收

成功路径的 error 接口确实为 nil；所有不确定断言都处理 ok。

```go
type Problem struct{ Msg string }

func (*Problem) Error() string { return "problem" }

func validate(ok bool) error {
    var p *Problem
    if !ok {
        p = &Problem{Msg: "bad input"}
    }
    if p == nil { return nil }
    return p
}
```

章内续问 4.2 · 闯祸与群聊

泛型只抽共同算法

闯祸现场　白板上画着创建账单和撤销账单两条三步流程，旁边是一张只做 map keys 的小卡片。阮秀把两条流程的副作用用红线分开。

陈平安为了消除几行重复，把创建与撤销硬塞进一个泛型函数；代码短了一点，错误处理和补偿语义却搅在一起。

陈平安



外形都是三步，抽成一个泛型流程不更整齐吗？



阮秀

创建与撤销失败后果不同，共同的稳定算法在哪里？

陈平安



业务保留具名函数，只把取 map keys 写成泛型。



李宝瓶

若算法只比较相等，约束还需要更多能力吗？



宁姚

不需要。只保留 comparable 这类最小约束。

收束　两条业务流程各回各的泳道，泛型只带走那张真正通用的小卡片。

先有画面，再落到工程

泛型只抽共同算法

先懂一句

先确认重复的是稳定算法，再决定是否引入类型参数。

直觉类比

泛型像饼干模具：当好多饼干只差面团口味、切法完全一样时，做一个模具很划算；如果每块都要雕花，硬套模具只会把饼干压烂。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 keys 的输入和返回值：它接什么，最后必须交回什么。
- 最后核对 keys 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 4.2 · 拆去比喻

泛型只抽共同算法

工程要点

- 泛型适合容器、集合变换和与具体业务无关的稳定算法。
- 类型约束保持最小；只比较相等就使用 comparable。
- 业务规则差异优先用具名函数或接口，不强凑类型参数。
- 调用处比抽象前更难读时，保留少量重复通常更便宜。

练手

找两段真实重复代码，判断共同的是算法还是仅仅长得相似。

验收

能说明类型参数消除的具体重复；约束未暴露无用操作。

```go
func keys[K comparable, V any](m map[K]V) []K {
    out := make([]K, 0, len(m))
    for k := range m {
        out = append(out, k)
    }
    return out
}
```

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 接口应该由谁定义？

陈平安



接口通常由使用方按所需的最小能力定义。



宁姚

边界闭合。

第 2 问 值与指针 receiver 的方法集有何差异？

陈平安



值 receiver 属于 T 和 *T 的方法集；指针 receiver 只属于 *T。



宁姚

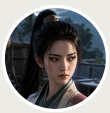
边界闭合。

第 3 问 组合为何不等于继承？

陈平安



组合复用行为，但不自动建立 is-a 继承关系。



宁姚

边界闭合。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 05 场 · 闯祸与群聊

第五章 山门与院落：Module、Package 与依赖方向

目标：理解 module、package、internal、目录边界和 composition root。

闯祸现场

清晨的工位上排着数据库、Repo、Service、Handler 四块磁贴，最右边是一张写着 main 的启动卡。go.mod 打印稿被咖啡杯压在显示器下。

陈平安删掉 Service 的装配后启动程序，Handler 一接请求就摸到 nil；他还把 module、package 和 internal 当成同一道围墙，本地能跑的 import 到另一棵目录树便失败。

陈平安



这些包不能自动找到彼此吗？



阮秀

数据库都没连好，Handler 凭什么先开门？

陈平安



我在 main 里按数据库、存取层、业务层、入口层顺序组装。



李宝瓶

go.mod 有依赖，是否就能绕过 internal 的目录限制？



宁姚

不能。module 管依赖图，internal 仍按父目录树限制。

收束

四块磁贴排好后，咖啡杯终于只需要负责压住 go.mod。

先有画面，再落到工程

第五章 山门与院落：Module、Package 与依赖方向

先懂一句

Module 是山界，Package 是院落，main 只负责组装。

直觉类比

Module 是整所学校，Package 是各个教室，internal 是只许本楼学生进的小仓库，main 像早晨开门时负责把老师、钥匙和课表凑齐的值日生。

术语落地

- package：代码住在哪个院子
- module：一整座项目山门和它的依赖清单
- internal：只准自家院子进入的内门

读代码抓手

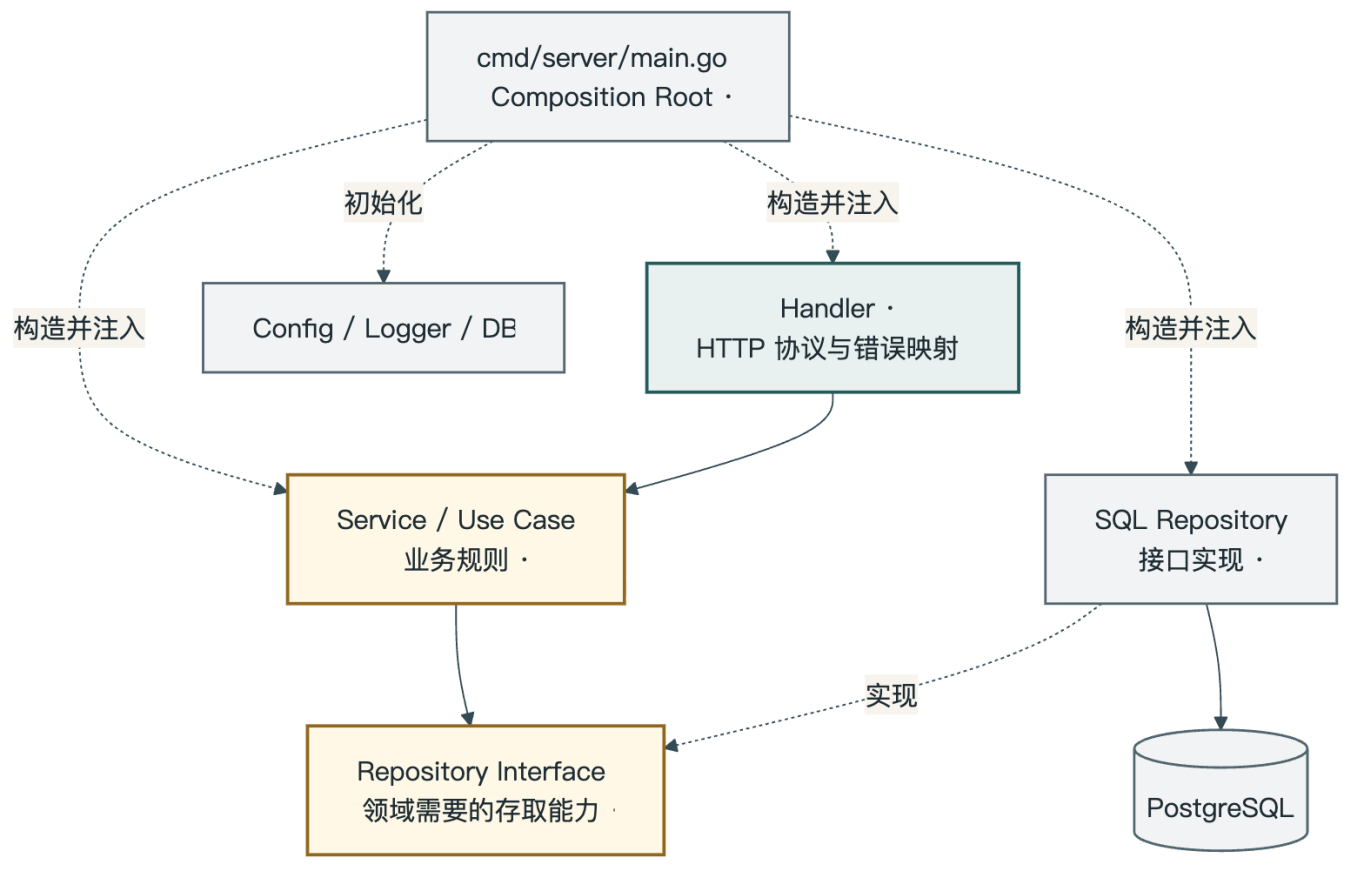
- 先看这段代码在读写什么状态，再找它对外承诺的结果。
- 最后核对状态出口：成功改变什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 5.1

Go 后端工程的目录与依赖方向

小白先懂：Module 是山界，Package 是院落，main 只负责组装。



拆去比喻

- module 由 go.mod 定义主模块路径和依赖图；import 可来自主模块、标准库或依赖图中的外部模块。
- package 是编译与封装边界；internal 的可见性由目录位置决定，不等同于 module 边界。
- internal/x 只允许 internal 上一级目录树内的代码导入；main 仍只负责选择实现、注入依赖和启动。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```
ledger-agent/
  go.mod
  cmd/server/main.go
  internal/bill/handler.go
  internal/bill/service.go
  internal/bill/repository.go
  internal/platform/config/config.go
  internal/platform/database/database.go
```

失败路径

- 把 go.mod 误解成只准导入本模块
- 忽略 internal 的父目录树限制
- 业务层反向依赖 Gin 或 SQL 行

练剑任务

建立示例目录，再分别从允许与不允许的父目录树导入 internal/bill；同时引入一个 go.mod 声明的外部依赖并运行 go test ./...。

验收证据

能区分 module 依赖图、package 封装与 internal 父目录树限制，并说明 main 为何是 composition root。

收剑自检

- [ ] module 与 package 的边界有何不同？
- [ ] 业务为什么不应依赖 Gin？
- [ ] main 为什么是 composition root？

章内续问 5.1 · 闯祸与群聊

依赖问题先问模块

闯祸现场　CI 屏幕显示依赖版本 v1.4.2，本地终端却指向另一条引入链。桌上摊着 go.mod、go.sum 和一张模块关系图。

陈平安看到本地可以 import 就准备重跑 CI，却没有解释间接依赖从哪里来；一次 tidy 还删了两行，他差点把删除当成依赖无用的证明。

陈平安



本地能 import，CI 拿错版本是不是缓存坏了？



李宝瓶

先不猜缓存，哪个模块通过哪条链把它引进来？

陈平安



我用 go mod why -m 和 go mod graph 查路径。



阮秀

tidy 删除和 verify 通过，能证明依赖没漏洞吗？



宁姚

不能。审查 diff；verify 只校验缓存与 go.sum。

收束　模块关系图被钉到屏幕旁，重跑按钮终于失去了甩锅资格。

先有画面，再落到工程

依赖问题先问模块

先懂一句

先问模块边界和引入路径，再判断依赖是否健康。

直觉类比

像查地铁换乘：别一看到堵车就拆轨道。先看这条依赖从哪站上车、经过哪些线、为什么会来到这里，再决定 tidy 该拿走什么。

术语落地

- 缓存：为了快而做的副本，不是最终事实

读代码抓手

- 设计练习 5.1：提交状态表、测试矩阵、命令输出或验收证据。
- 本节不提供可运行实现，不用漂亮伪代码冒充实验结果。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 5.1 · 拆去比喻

依赖问题先问模块

工程要点

- go.mod 记录模块路径、Go 版本和直接或间接依赖。
- go mod tidy 会考虑主模块相关包、测试及各 OS/架构/build tag 组合，执行后必须审查 diff。
- go mod why -m 解释引入路径；go mod graph 展开模块关系。
- go mod verify 校验缓存内容与 go.sum，不代表依赖没有漏洞。

练手

任选一个间接依赖，写出 why 路径、版本来源和是否仍需保留。

验收

能从导入链解释依赖；tidy 后只有预期 go.mod/go.sum 变化。

记忆钉

1. go.mod 记录模块路径、Go 版本和直接或间接依赖。
2. go mod tidy 会考虑主模块相关包、测试及各 OS/架构/build tag 组合，执行后必须审查 diff。
3. go mod why -m 解释引入路径；go mod graph 展开模块关系。
4. go mod verify 校验缓存内容与 go.sum，不代表依赖没有漏洞。

章内续问 5.2 · 闯祸与群聊

工作区不是发布配置

闯祸现场　两台电脑之间连着一根写有 go.work 的蓝线，左边是账本模块，右边是共享模块。阮秀把 GOWORK=off 写在便签上。

陈平安拔掉蓝线后单独测试，账本模块立刻找不到旁边依赖；原来本地 workspace 遮住了缺失的正式版本和临时 replace。

陈平安



go.work 能让联调通过，直接当发布配置不行吗？



阮秀

离开这台电脑，正式版本从哪里解析？

陈平安



我查 go env GOWORK，再关闭工作区逐模块测试。



李宝瓶

若 GOWORK=off 失败，只把 go.work 提交上去够吗？



宁姚

不够。补正式依赖，发布的 go.mod 不混本地 replace。

收束　蓝线可以继续方便联调，但不再假装自己是发布时的电源线。

先有画面，再落到工程

工作区不是发布配置

先懂一句

go.work 是本地联调视图，不是发布依赖的替代品。

直觉类比

go.work 像你桌上临时拼起来的联调沙盘，方便同时摆几个项目；发布时真正装箱的仍是每个 module 自己的 go.mod，不能把桌子一起寄走。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 设计练习 5.2：提交状态表、测试矩阵、命令输出或验收证据。
- 本节不提供可运行实现，不用漂亮伪代码冒充实验结果。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 5.2 · 拆去比喻

工作区不是发布配置

工程要点

- go work use 聚合多个本地模块，不改各模块的发布依赖。
- go env GOWORK 显示当前是否被工作区接管。
- 用 GOWORK=off go test ./... 验证每个模块能独立成立。
- replace 若仅用于本地开发，应避免混入准备发布的 go.mod。

练手

建立含两个模块的临时工作区，再关闭 GOWORK 验证各自测试。

验收

工作区内联调通过；每个模块在 GOWORK=off 下仍可解析并测试。

记忆钉

1. go work use 聚合多个本地模块，不改各模块的发布依赖。
2. go env GOWORK 显示当前是否被工作区接管。
3. 用 GOWORK=off go test ./... 验证每个模块能独立成立。
4. replace 若仅用于本地开发，应避免混入准备发布的 go.mod。

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 module 与 package 的边界有何不同？

陈平安

module 定义版本、主模块路径与依赖图；package 定义编译和封装；internal 另受父目录树限制。



宁姚

边界闭合。

第 2 问 业务为什么不应依赖 Gin？

陈平安

业务层不依赖 Gin，才能独立测试并替换传输协议。



宁姚

边界闭合。

第 3 问 main 为什么是 composition root？

陈平安

main 负责选择实现、注入依赖并启动进程，所以是 composition root。



宁姚

边界闭合。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 06 场 · 闯祸与群聊

第六章 错误要留因果：Error 与边界测试

目标：掌握错误包装、分类、取消与超时的边界翻译，以及表驱动测试。

闯祸现场

雨点敲着玻璃，屏幕上同时弹出 ErrMissing、context.Canceled 和 upstream timeout 三条记录。桌边放着三只颜色不同的文件夹，等着接各自的错误证据。

陈平安在 Repo、Service、Handler 每层都打印一次，还处处 recover；同一个错误刷了三行日志，最后却都被翻成 500，连接断了还继续写响应。

陈平安



都是 error，统一打日志再回 500 不是更省事吗？



李宝瓶

客户端取消和上游超时，用户看到的结果能一样吗？

陈平安



我用 %w 留因果，Handler 用 errors.Is 分类。



阮秀

panic 也让每个小函数自己 recover 吗？



宁姚

不。recover 只守边界；连接断开就不再写响应。

收束

三只文件夹各自合上，同一个错误总算不再复印三份。

先有画面，再落到工程

第六章 错误要留因果：Error 与边界测试

先懂一句

内部留因果，边界翻契约。

直觉类比

错误不是一句‘完蛋了’，而是一串快递追踪：哪一站坏了、上一站交了什么、门口最后该回哪种状态码，都要留得下来。

术语落地

- error：事情没办成时一起带回来的原因条
- Trace：一次请求一路经过哪些地方的脚印串

读代码抓手

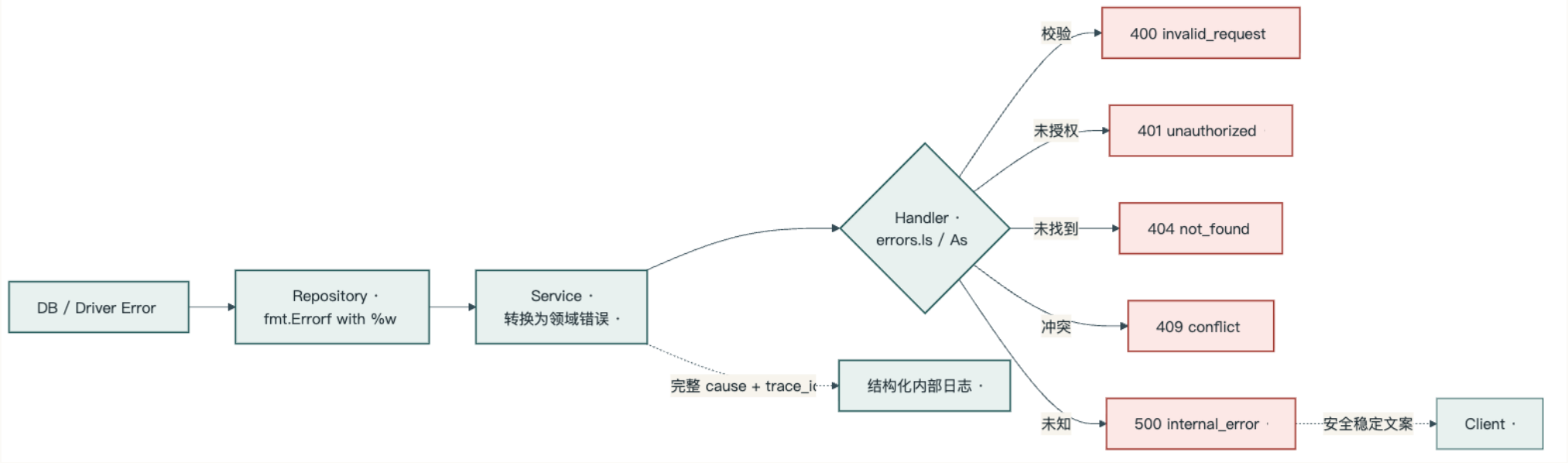
- 先看 mapError 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 mapError 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 6.1

错误因果链与边界契约翻译

小白先懂：内部留因果，边界翻契约。



读图要点

- 使用 %w 保留错误因果，用 errors.Is/As 分类，不比较 err.Error()。
- 客户端取消、服务自身 deadline 和上游 timeout 是不同事件，不能全部翻成 500 或 504。
- 连接已断时不要继续写响应；日志记录 cancelled，并让下游协作式收口。

代码完整展开，不牺牲阅读字号

小白先懂：内部留因果，边界翻契约。

```go
type HTTPDecision struct {
    Status int
    Code   string
    Write  bool
}

func mapError(err error) HTTPDecision {
    switch {
    case err == nil:
        return HTTPDecision{200, "ok", true}
    case errors.Is(err, context.Canceled):
        return HTTPDecision{0, "request_cancelled", false}
    case errors.Is(err, ErrUpstreamTimeout):
        return HTTPDecision{504, "upstream_timeout", true}
    case errors.Is(err, context.DeadlineExceeded):
        return HTTPDecision{504, "deadline_exceeded", true}
    case errors.Is(err, ErrInvalidBill):
        return HTTPDecision{422, "invalid_bill", true}
    default:
        return HTTPDecision{500, "internal_error", true}
    }
}
```

下一页核对失败路径、练剑任务、验收证据与自检问题。

失败要能复现，完成要有证据

失败路径

- 比较 `err.Error()`
- 既记录错误又层层重复记录
- 把 panic 当普通错误处理

练剑任务

为 mapError 写表驱动测试，覆盖包装错误、客户端取消、服务 deadline、上游超时和未知错误。

验收证据

公开 code 稳定；取消不写响应且不计 500；deadline 不落入 500；内部 cause 不进入 body。

收剑自检

- [ ] 为什么不用 err.Error() 比较？
- [ ] panic 适合普通业务错误吗？
- [ ] 500 响应为什么不能带内部 cause？

章内续问 6.1 · 闯祸与群聊

错误要能被识别

闯祸现场　工单上先写着 ErrMissing，经过 Repo 和 Service 后，又盖上了 load bill 的操作章。李宝瓶把原始错误与新文案用一根线连起来。

陈平安却只留下“加载失败”四个字，Handler 只能比较字符串；文案稍微改动，账单不存在就被误判成未知错误。

陈平安



Handler 比较错误文本，改起来不是最快吗？



李宝瓶

文案换一个词，ErrMissing 的类别还找得到吗？

陈平安



我用 fmt.Errorf 和 %w 包装，再用 errors.Is。



阮秀

若还要读取结构字段，继续堆字符串吗？



宁姚

用 errors.As；错误只在增加语义的边界处理一次。

收束　那根线穿过两枚印章，ErrMissing 总算没有在路上改名失踪。

先有画面，再落到工程

错误要能被识别

先懂一句

错误文本给人看，保留下来的错误链给程序判断。

直觉类比

像医院转诊单：每一科都可以补一句‘我这里看见了什么’，但不能把上一个诊断撕掉。程序最后才能沿着单子认出最初的病因。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 load 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 load 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 6.1 · 拆去比喻

错误要能被识别

工程要点

- 使用 fmt.Errorf 加上下文时，以 %w 保留可遍历的原因链。
- 稳定类别用哨兵错误配合 errors.Is，结构信息用自定义类型。
- errors.As 抽取链上的目标类型，不依赖具体包裹层数。
- 错误应在能增加业务语义的边界处理一次，避免重复记录。

练手

为 load 补一层业务上下文，并用 errors.Is 验证底层 ErrMissing。

验收

错误文本包含操作上下文；调用方不解析字符串即可识别错误类别。

```go
var ErrMissing = errors.New("missing")

func load(id string) error {
    err := read(id)
    if err != nil {
        return fmt.Errorf("load %s: %w", id, err)
    }
    return nil
}
```

章内续问 6.2 · 闯祸与群聊

recover 只守边界

闯祸现场　测试台上放着一张非法金额小票、三张清理卡和一份堆栈记录。宁姚把“业务错误”和“panic 边界”分成两列。

陈平安让解析函数遇到坏金额就 panic，再在每个小函数里 recover 成零；账单看起来成功，真正的坏状态和来路却全被吞掉。

陈平安



recover 后返回零，至少服务不会崩吧？



李宝瓶

零金额像成功结果，调用方怎么知道输入失败？

陈平安



我改为返回 error，只在请求边界 recover。



阮秀

新 goroutine 的 panic，外层边界能替它 recover 吗？



宁姚

不能。各自顶层收口，记堆栈并执行 defer 清理。

收束　三张清理卡按后进先出翻完，零金额终于不用假装平安无事。

先有画面，再落到工程

recover 只守边界

先懂一句

可预期失败返回 error；recover 只在能够制定隔离策略的边界工作。

直觉类比

defer 像离开教室前自动关灯，error 像正常举手报告，panic 像消防警铃。不能每次铅笔断了都拉警铃，也不能警铃响后假装什么都没发生。

术语落地

- error：事情没办成时一起带回来的原因条

读代码抓手

- 先看 boundary 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 boundary 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 6.2 · 拆去比喻

recover 只守边界

工程要点

- defer 在函数返回前按后进先出执行，适合就地登记清理。
- defer 的实参在登记时求值；闭包读取变量时要留意后续变化。
- 可预期的 I/O、校验和业务失败都应返回 error。
- 边界 recover 后要记录堆栈、转换结果，并让资源清理继续执行。

练手

让 next 依次正常返回、返回业务错误、触发 panic，观察三条路径。

验收

业务错误原样传播；panic 仅在边界转稳定错误；堆栈只进内部日志且 defer 清理执行。

```go
func boundary(next func() error) (err error) {
    defer func() {
        if v := recover(); v != nil {
            logPanic(v, debug.Stack())
            err = ErrInternal
        }
    }()
    return next()
}
```

章内续问 6.3 · 闯祸与群聊

测试从例子走向性质

闯祸现场　夜班桌上有一盒彩色输入卡，最上面两张写着空串和中文，旁边放着临时目录钥匙。阮秀把“往返相等”写成唯一判定。

手写表格用例全部通过，一段怪字节却让解码器撞坏；并行子测试还共用同一个状态，失败后临时文件也留在磁盘。

陈平安

再多补几十个例子，能不能覆盖未知输入？


阮秀
先说稳定性质，什么结果对任意支持的 string 都成立？

陈平安

我加空串和中文种子，断言编解码往返相等。


李宝瓶
并行用例若共享状态，失败还能归因到输入吗？


宁姚
不能。隔离状态，用 t.Cleanup 收资源，反例留回归。

收束　临时目录钥匙被准时归还，怪字节则被正式收编进回归名单。

先有画面，再落到工程

测试从例子走向性质

先懂一句

单元测试锁定已知场景，fuzz 用稳定性质寻找未知反例。

直觉类比

普通测试像拿已知钥匙开锁，fuzz 像让机器递来一大篮奇怪钥匙；不管哪把卡住，测试结束都要把临时房间收拾干净。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 FuzzRoundTrip 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 FuzzRoundTrip 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 6.3 · 拆去比喻

测试从例子走向性质

工程要点

- t.Helper 让失败定位回调用处，t.Cleanup 统一登记资源回收。
- 子测试按场景命名，并只在互不共享状态时使用 t.Parallel。
- f.Add 放入有意义的种子；Fuzz 函数断言稳定性质。
- 发现的失败输入进入语料库，修复后成为持续回归资产。

练手

为声明支持任意 string 的编解码器加入空串和中文种子，断言往返必成功且相等。

验收

常规测试可重复；fuzz 不靠吞 panic 通过；临时资源由 Cleanup 回收。

```go
func FuzzRoundTrip(f *testing.F) {
    f.Add("hello")
    f.Fuzz(func(t *testing.T, in string) {
        out, err := roundTrip(in)
        if err != nil { t.Fatal(err) }
        if out != in {
            t.Fatalf("got %q want %q", out, in)
        }
    })
}
```

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 为什么不用 err.Error() 比较？

陈平安



错误文本会变化；分类应使用 errors.Is 或 errors.As。



宁姚

边界闭合。

第 2 问 panic 适合普通业务错误吗？

陈平安



普通业务失败应返回 error，panic 留给当前调用无法继续的异常状态。



宁姚

边界闭合。

第 3 问 500 响应为什么不能带内部 cause？

陈平安



500 响应只暴露稳定错误码，内部 cause 留在受控日志和 trace。



宁姚

边界闭合。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 07 场 · 闯祸与群聊

第七章 众剑齐出：Goroutine、GMP 与 Race

目标：理解 goroutine 调度、WaitGroup、mutex、共享状态与 race detector。

闯祸现场

午饭后的白板写着 total=0，旁边画了四个 goroutine 和一条汇总 channel。李宝瓶把秒表放在键盘前，提醒大家别靠等待时间猜执行顺序。

两个 goroutine 同时修改 total，结果一会儿是 3000，一会儿少一笔；陈平安加了 50ms 的 sleep，本地偶尔正确，go test -race 仍旧报警。

陈平安



把 sleep 再加长一点，顺序会不会稳定？



李宝瓶

同一位置并发读写，延时能建立哪条同步关系？

陈平安



我让各任务发送小计，由一个 owner 汇总并等待。



阮秀

若要共同保护一组字段，还只靠 channel 吗？



宁姚

用 Mutex 守不变量；测试必须跑到真实竞争路径。

收束

秒表被收回抽屉，终于不用兼职当同步原语。

先有画面，再落到工程

第七章 众剑齐出：Goroutine、GMP 与 Race

先懂一句

G 等事，M 干活，P 给资格；并发先定主人。

直觉类比

像饭馆突然来了很多桌客人。goroutine 是小跑堂，M 是真正在跑的腿，P 是允许上岗的工牌；若两人同时改同一张菜单，race 就来了。

术语落地

- goroutine：很轻的小跑堂，可以同时派出很多个
- channel：小跑堂之间传东西的窗口
- select：几个窗口都可能来消息时，等能动的那个
- Context：沿调用链传下去的取消铃和截止钟
- race：两个人同时改一本账，谁先落笔说不清

读代码抓手

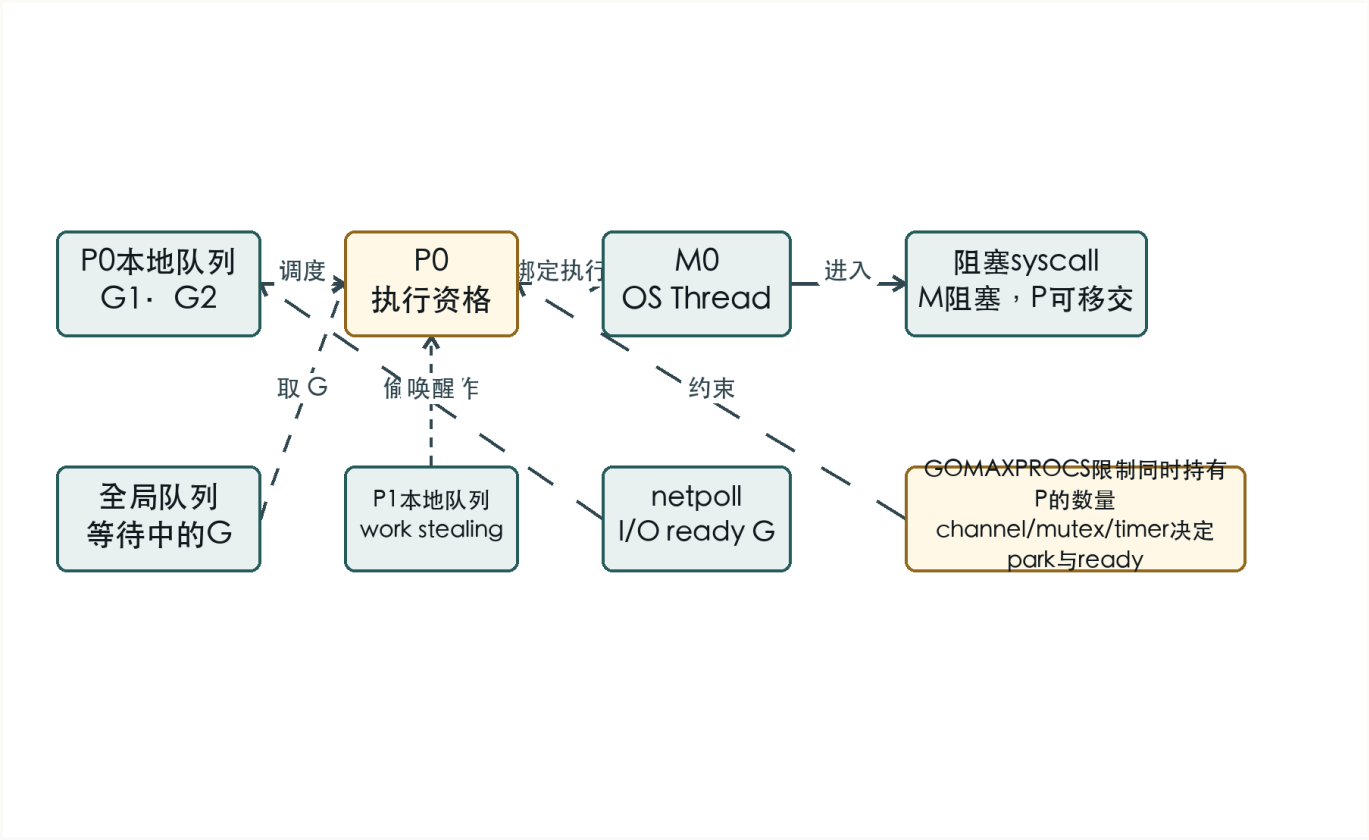
- 先看 fetchAll 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 fetchAll 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 7.1

Goroutine、Channel 与 GMP 运行时关系

小白先懂：G 等事，M 干活，P 给资格；并发先定主人。



拆去比喻

- goroutine 是并发执行单元；GMP 是运行时调度模型。P 数量由 GOMAXPROCS 控制，不等同于物理核心定律。
- channel 提供通信与同步，但 channel 外共享变量仍可能 data race。互斥锁保护共享状态，channel 适合传递所有权或事件。
- `select` 在多个 ready case 中选择，但不承诺业务公平或无饥饿；带 default 的循环可能忙轮询。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
func fetchAll(ctx context.Context, urls []string, limit int) error {
    if limit <= 0 {
        return fmt.Errorf("limit must be positive: %d", limit)
    }
    g, ctx := errgroup.WithContext(ctx)
    g.SetLimit(limit)
    for _, url := range urls {
        url := url
        g.Go(func() error { return fetch(ctx, url) })
    }
    return g.Wait()
}
```

失败路径

- fire-and-forget goroutine
- 关闭由接收方控制的 channel
- 用 Sleep 协调并发

练剑任务

实现 fetchAll，并测试 limit=0、limit<0、子任务失败和父 context 取消。

验收证据

非法 limit 立即返回；并发峰值不超过上限；失败与取消都能让所有 goroutine 收口，race 检测通过。

收剑自检

- [ ] P 等于物理 CPU 核吗？
- [ ] Channel 外共享变量会竞态吗？
- [ ] race detector 能证明无竞态吗？

章内续问 7.1 · 闯祸与群聊

没有同步，就没有顺序

闯祸现场　白板上画着两个同时指向 total 的箭头，旁边是一张写有 happens-before 的透明片。李宝瓶把 50ms 秒表压在透明片下。

陈平安用 sleep 调整顺序后，结果偶尔正确，race 检查仍报告同一位置的读写冲突；日志先后看起来整齐，却没有任何同步边。

陈平安



日志每次都是先写后读，为什么还算竞争？



李宝瓶

日志顺序能让两次内存访问建立 happens-before 吗？

陈平安



启动前写配置，其余共享访问用 channel 或锁同步。



阮秀

race 没报警，就能证明所有并发路径都安全吗？



宁姚

不能。它只检查本次真正执行到的路径。

收束　透明片盖住两支箭头，秒表则被礼貌请离并发设计会。

先有画面，再落到工程

没有同步，就没有顺序

先懂一句

并发正确性依赖 happens-before，不依赖日志顺序、延时或调度运气。

直觉类比

像两个小朋友同时改黑板分数。你在门外听见谁先喊，不代表黑板上谁先写；没有锁、消息或原子操作，就没有可靠的先后顺序。

术语落地

- goroutine：很轻的小跑堂，可以同时派出很多个
- channel：小跑堂之间传东西的窗口
- race：两个人同时改一本账，谁先落笔说不清

读代码抓手

- 先看 publish 的输入和返回值：它接什么，最后必须交回什么。
- 最后核对 publish 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 7.1 · 拆去比喻

没有同步，就没有顺序

工程要点

- 同一内存位置并发访问且至少一次为写，缺少同步就是数据竞争。
- goroutine 启动、channel 收发、锁和原子操作可建立特定同步关系。
- sleep、日志顺序和本机多次成功都不是同步证据。
- go test -race ./... 是动态检查，仍需覆盖真实并发路径。

练手

把一个共享 bool 轮询示例改成 channel 通知，并分别跑普通测试和 race。

验收

能指出同步边；race 检查无报告；实现不依赖 sleep 或调度运气。

```go
func publish() <-chan int {
    ch := make(chan int, 1)
    go func() {
        value := 42
        ch <- value
        close(ch)
    }()
    return ch
}
```

章内续问 7.2 · 闯祸与群聊

先定所有权，再选同步器

闯祸现场　工具板上挂着 Mutex、RWMutex、Once、atomic 和 channel 五张卡，桌面分成初始化、计数、传结果三个格子。阮秀先给每格写上 owner。

陈平安把三种问题全改成 channel，初始化绕远路、计数代码变长、退出协议也更难读；含锁结构还被复制了一份。

陈平安



统一用 channel，团队不是更容易记吗？



阮秀

传所有权和守一组共享不变量，问题形状相同吗？

陈平安

初始化用 Once，独立计数用 atomic，结果归 owner。





李宝瓶

读多就直接换 RWMutex，收益从哪里证明？



宁姚

先跑基准。含锁对象首次使用后不得复制。

收束　五张工具卡各回各位，工具板终于不再像只卖一种螺丝刀。

先有画面，再落到工程

先定所有权，再选同步器

先懂一句

先定状态的所有者和不变量，再选择匹配问题形状的同步工具。

直觉类比

先决定粉笔归谁，再选工具：一个人写就交给单一所有者；偶尔轮流写可上锁；只改一个计数器可用 atomic；需要交接任务再用 channel。

术语落地

- race：两个人同时改一本账，谁先落笔说不清
- 缓存：为了快而做的副本，不是最终事实

读代码抓手

- 先看 Get 的输入和返回值：它接什么，最后必须交回什么。
- 最后核对 Get 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 7.2 · 拆去比喻

先定所有权，再选同步器

工程要点

- Mutex 保护一组必须共同保持不变量的字段，锁范围要可说明。
- RWMutex 只有在读多、竞争真实且基准证明收益时再采用。
- Once 表达恰好一次初始化；atomic 只适合简单独立状态。
- 禁止复制已使用的锁；包含锁的结构通常以指针传递。

练手

为一个计数器和一个配置缓存分别选择同步工具，并写出选择理由。

验收

每个共享状态有明确所有者；工具与不变量匹配；race 检查通过。

```
type ConfigCache struct {
    once sync.Once
    cfg  Config
}

func (c *ConfigCache) Get() Config {
    c.once.Do(func() { c.cfg = loadConfig() })
    return c.cfg
}
```

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 P 等于物理 CPU 核吗？

陈平安



P 是调度资源，不等同于某一颗物理 CPU 核。



宁姚

边界闭合。

第 2 问 Channel 外共享变量会竞态吗？

陈平安



会；channel 不会自动保护它之外的共享变量。



宁姚

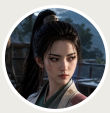
边界闭合。

第 3 问 race detector 能证明无竞态吗？

陈平安



不能；race detector 只检查本次执行实际覆盖到的并发路径。



宁姚

边界闭合。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 08 场 · 闯祸与群聊

第八章 飞剑须能收：Channel、Select 与 Context

目标：掌握 channel 关闭、select 语义、context 取消、超时和 goroutine 泄漏收口。

闯祸现场

傍晚的桌上有四张账单卡片和一个只能装三张的收件盒，旁边贴着 cancel、close、wait 三枚标签。空调风把第四张卡片吹到盒边。

消费者取走三笔就返回，生产者仍堵在第四次发送；请求已经取消，测试却一直等到超时，陈平安还以为 select 会按书写顺序自动选中取消。

陈平安



我给 select 加 default，生产者就不会堵了吧？



李宝瓶

default 若丢结果或忙等，退出证据在哪里？

陈平安



发送时监听 ctx.Done，消费者 cancel 后等待退出。



阮秀

channel 由接收方关，还是由确认不再发送的人关？



宁姚

发送方关闭。Context 只通知，不能强杀任务。

收束

第四张账单被放回文件夹，三枚标签终于按顺序贴稳了。

先有画面，再落到工程

第八章 飞剑须能收：Channel、Select 与 Context

先懂一句

谁等待、谁取消、错归谁、上限多少。

直觉类比

派小跑堂出去时，不能只喊‘快去’，还要告诉他什么时候回来、谁能叫停、回来把结果交给谁。否则饭馆关门了，他还在后巷端着盘子发呆。

术语落地

- struct：把一件东西要填的格子做成固定模具
- goroutine：很轻的小跑堂，可以同时派出很多个
- channel：小跑堂之间传东西的窗口
- select：几个窗口都可能来消息时，等能动的那个
- Context：沿调用链传下去的取消铃和截止钟
- race：两个人同时改一本账，谁先落笔说不清
- owner：这份资源真正归属的那个人或主体

读代码抓手

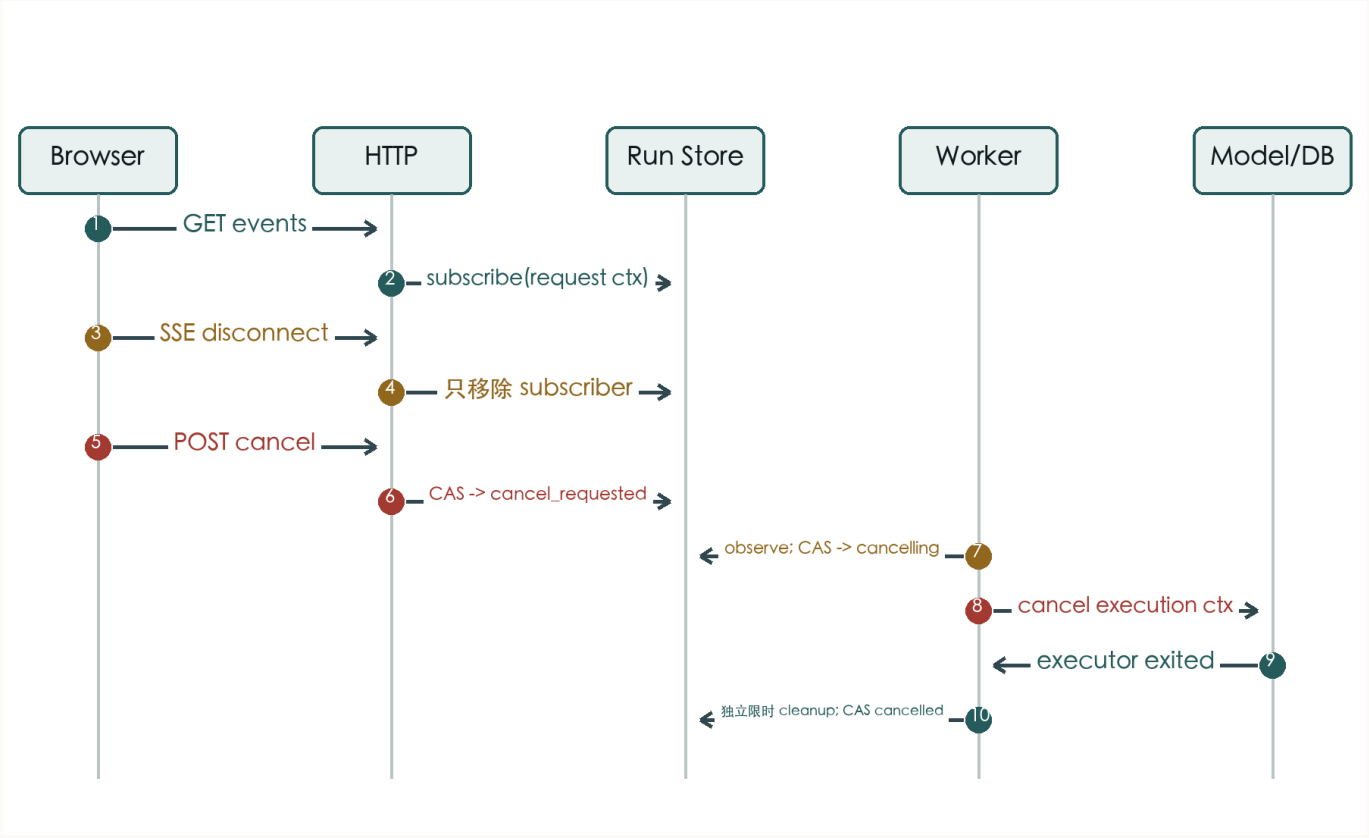
- 先看 stream 的输入和返回值：它接什么，最后必须交回什么。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 最后核对 stream 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 8.1

Context 取消信号跨层传递时序

小白先懂：谁等待、谁取消、错归谁、上限多少。



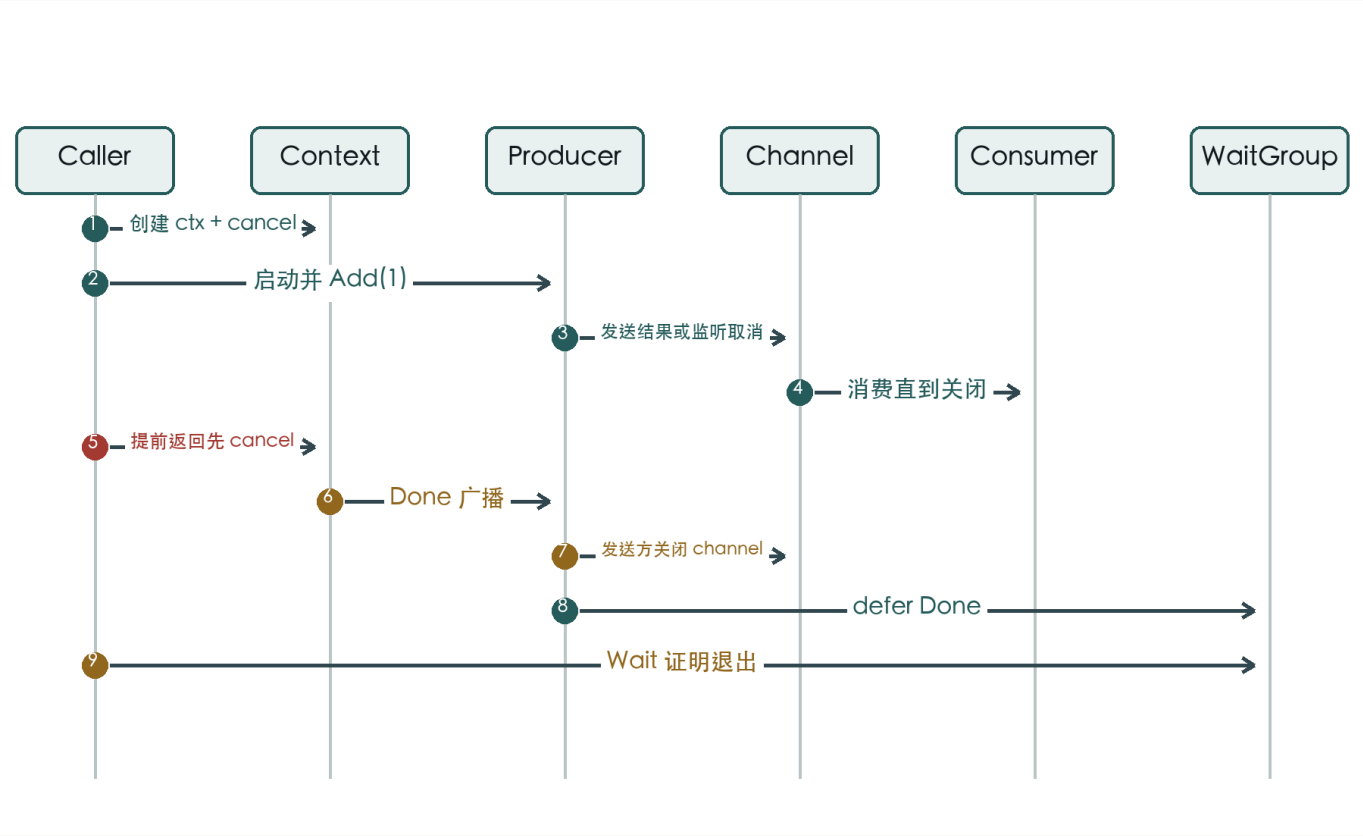
拆去比喻

- 能保证不再发送的唯一 owner/协调者负责关闭 channel；接收方使用 range 直到关闭。
- select 在多个 case 同时就绪时伪随机选择，不承诺绝对公平。
- context 只广播取消意图，不能强杀 goroutine、解锁 mutex 或保证驱动立即停止。
- 每个 goroutine 都要定义等待对象、取消来源、错误归属与退出证据。

图 8.2

Goroutine 的取消、关闭与等待闭环

小白先懂：谁等待、谁取消、错归谁、上限多少。



拆去比喻

- 启动 goroutine 时同时写下正常完成、取消、关闭和等待路径。
- 发送方关闭 channel；调用方 cancel 后仍要 Wait 证明退出。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
func stream(ctx context.Context, values []int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, value := range values {
            select {
            case <-ctx.Done():
                return
            case out <- value:
            }
        }
    }()
    return out
}

func consumeStream(ctx context.Context, values []int) {
    for value := range stream(ctx, values) {
        consume(value)
    }
}
```

失败路径

- 把 context 放进 struct 长期保存
- 把业务参数塞进 context
- 忽略 ctx.Err

练剑任务

实现可取消 producer/consumer，记录 goroutine 启动和退出，并覆盖消费者提前停止。

验收证据

发送方唯一关闭 channel；取消后所有 goroutine 在期限内退出；go test -race 与泄漏检查通过。

收剑自检

- [ ] 谁应该关闭 channel？
- [ ] select 是否保证绝对公平？
- [ ] context 能强杀 goroutine 吗？

章内续问 8.1 · 闯祸与群聊

启动 goroutine 时就写退出路

闯祸现场　桌上排着四张账单卡，消费者的托盘只标着“取三张”，旁边还有 cancel、close、wait 三个按钮。李宝瓶把第四张卡放在发送口。

消费者拿满三张便返回，生产者永久堵在第四次发送；陈平安想加 default 绕过阻塞，却可能忙等或把结果直接丢掉。

陈平安



加 default 跳过第四笔，goroutine 不就退出了吗？



李宝瓶

跳过的数据归谁负责，退出又由谁证明？

陈平安



生产者发送时监听取消，并由它关闭 out。



阮秀

消费者提前结束后，只 cancel 但不等待够吗？



宁姚

不够。cancel 后还要 Wait，select 也不保证公平。

收束　第四张卡回到队列，三个按钮终于组成了一条完整退出路。

先有画面，再落到工程

启动 goroutine 时就写退出路

先懂一句

启动 goroutine 时就同时写下正常完成、取消和等待路径。

直觉类比

派小朋友去楼下买盐时，要同时说清三件事：买到后去哪、妈妈喊停时怎么回来、家里怎么知道他已经回来了。goroutine 也要出生就带退出路线。

术语落地

- goroutine：很轻的小跑堂，可以同时派出很多个
- channel：小跑堂之间传东西的窗口
- Context：沿调用链传下去的取消铃和截止钟

读代码抓手

- 先看 produce 的输入和返回值：它接什么，最后必须交回什么。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 最后核对 produce 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 8.1 · 拆去比喻

启动 goroutine 时就写退出路

工程要点

- 接收 context 的函数不保存它，取消信号沿调用链向下传。
- 创建 channel 的一方通常负责关闭；接收方不要替发送方 close。
- 每个 goroutine 都要有正常完成、取消或错误退出路径。
- 使用 WaitGroup 或 errgroup 等待收口，不能只让进程替它回收。

练手

让消费者只取三个值就取消，测试生产者能在超时前退出。

验收

测试能证明取消后 goroutine 收口；无重复关闭、阻塞发送或忙等。

```go
func produce(ctx context.Context) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for i := 0; ; i++ {
            select {
            case out <- i:
            case <-ctx.Done():
                return
            }
        }
    }()
    return out
}
```

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 谁应该关闭 channel？

陈平安



能保证不再发送的唯一发送方或协调者负责关闭 channel。



宁姚

边界闭合。

第 2 问 select 是否保证绝对公平？

陈平安



不保证；多个 case 同时就绪时伪随机选择，不是业务公平策略。



宁姚

边界闭合。

第 3 问 context 能强杀 goroutine 吗？

陈平安



不能；context 只广播取消意图，goroutine 必须主动监听并返回。



宁姚

边界闭合。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。



卷二

Go 后端从请求到上线

把请求、数据、容量、观测与发布连成一座能守住失败的城。

第 09-16 章

第 09 场 · 闯祸与群聊

第九章 山门来客：HTTP、Gin 与契约

目标：掌握请求生命周期、路由、中间件、DTO、校验、认证上下文与状态码。

闯祸现场

午后的雨点敲着窗，陈平安把一杯奶茶和一张写着 bill-0827 的便签摆在显示器旁，准备接住 tenant-7 的 HTTP 创建请求。

他先写好了成功回信，却没读请求体；客户端塞进来的 user_id 和陌生字段就这样越过了门口。

陈平安



我已经回了 JSON，这笔账应该算收到了。



李宝瓶

你回的是信，谁检查来信大小和字段？

陈平安



我先限 body，再严格解码并做字段校验。



宁姚

tenant 和 user 又从哪里来？

陈平安



只取认证生成的 Principal，再交 Service 授权。



宁姚

创建回 201；请求形状错回 400，字段不合规回 422。

收束

便签终于贴在了请求入口，奶茶还没凉。

先有画面，再落到工程

第九章 山门来客：HTTP、Gin 与契约

先懂一句

能读来信，不等于应承来客。

直觉类比

HTTP 像城门收信。信封能拆开，不代表来客可信，也不代表他有权改账；门房要查格式、身份、权限和回信规格。

术语落地

- Context：沿调用链传下去的取消铃和截止钟
- HTTP：客户端和服务端互寄有格式的信
- Handler：门房，负责收信和回信
- Service：真正执行业务规矩的账房
- Trace：一次请求一路经过哪些地方的脚印串

读代码抓手

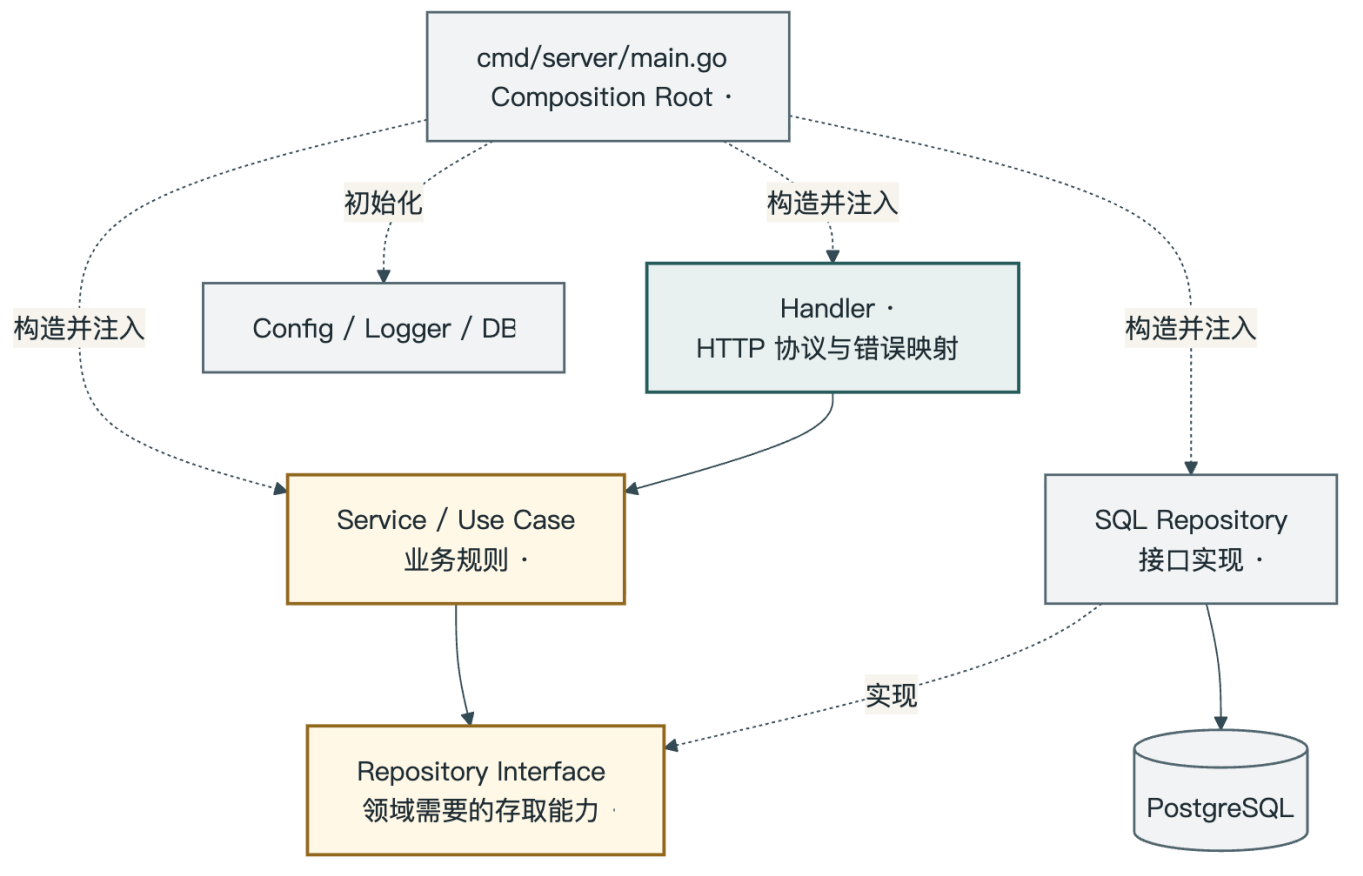
- 先看 Create 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Create 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 9.1

回顾 · Go 后端工程的目录与依赖方向

小白先懂：能读来信，不等于应承来客。



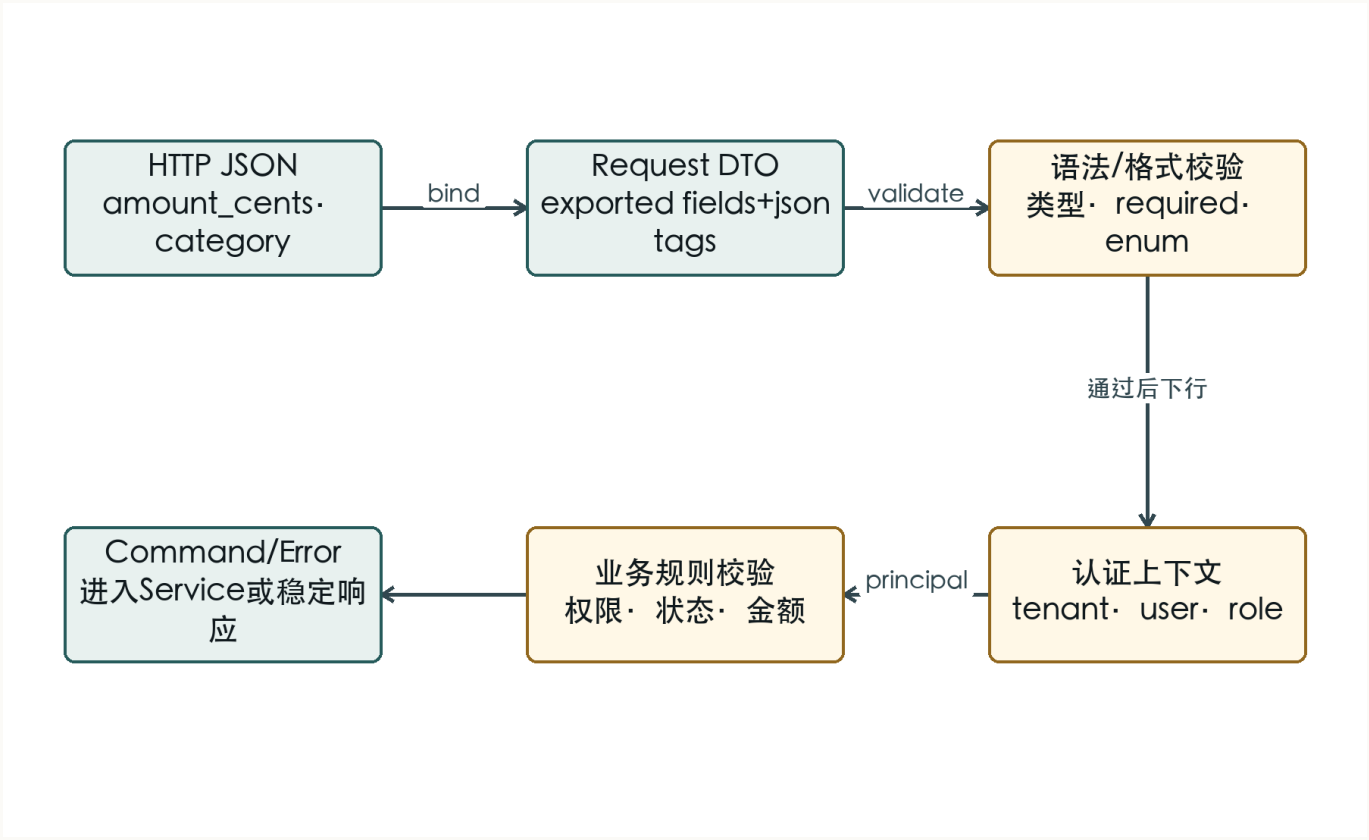
拆去比喻

- 认证中间件生成 typed Principal；Handler 显式把 TenantID/UserID 传给 Service 做授权。
- 请求 body 有大小上限，JSON 拒绝未知字段；解码后显式执行字段校验，再进入领域校验。
- context 传递 deadline、取消和 trace，不隐藏必须参与授权与落库的业务身份。
- 稳定错误 code 属于 HTTP 契约，内部 cause 不对客户端泄露。

图 9.2

回顾 · Struct、JSON 与请求数据管线

小白先懂：能读来信，不等于应承来客。



拆去比喻

- 解码、字段校验、认证与领域校验是四道不同边界。
- DTO 只承接协议字段；Principal 由服务端认证上下文生成。

代码完整展开，不牺牲阅读字号

小白先懂：能读来信，不等于应承来客。

```
type Principal struct { TenantID, UserID string }

func (h *Handler) Create(c *gin.Context) {
    principal, ok := PrincipalFrom(c)
    if !ok {
        writeError(c, 401, "unauthorized"); return
    }
    var req CreateBillRequest
    if err := decodeStrictJSON(c, &req, 1<<20); err != nil {
        writeError(c, 400, "invalid_request"); return
    }
    if err := h.requestValidator.Struct(req); err != nil {
        writeError(c, 422, "invalid_request_fields"); return
    }
    cmd := CreateBill{
        AmountCents: req.AmountCents, Category: req.Category,
    }
    bill, err := h.service.Create(
        c.Request.Context(), principal, cmd)
    writeResult(c, bill, err)
}
```

下一页核对失败路径、练剑任务、验收证据与自检问题。

失败要能复现，完成要有证据

失败路径

- 直接 bind 到数据库模型
- 状态码永远 200
- 信任客户端传来的 user_id

练剑任务

实现创建账单 Handler，覆盖缺少 Principal、未知 JSON 字段、超大 body 和越权租户。

验收证据

TenantID/UserID 从服务端认证上下文生成并显式入参；Service 完成授权；响应状态与错误 code 稳定。

收剑自检

- [ ] binding 与业务校验如何分工？
- [ ] user_id 应从哪里来？
- [ ] 为何不把所有响应都写成 200？

章内续问 9.1 · 闯祸与群聊

先学数据怎么流

闯祸现场　门口的测试机递来一卷看不到尽头的请求体，进度条像被胶带粘住一样不动。陈平安直接 io.ReadAll，内存报警先响，JSON 解码器还没等到文件尾。

陈平安



字段很少，整段读进来应该没事。



李宝瓶

字段少，不代表来客送来的流有尽头。



阮秀

一次 Read 同时返回数据和 error，你都处理吗？

陈平安



我先限长，多读一字节判断超限。

陈平安



取得资源的边界负责 Close，复制交给 io.Copy。



宁姚

Reader、Writer 和错误都必须有明确上限。

收束　那卷纸终于在桌沿前被裁成了规定长度。

先有画面，再落到工程

先学数据怎么流

先懂一句

io.Reader 和 io.Writer 把文件、网络与内存统一成流式边界。

直觉类比

像接水管：水会一段段来，桶有大小，接水人可能走开；必须规定谁关龙头、桶满怎么办、最后一次没水时有没有夹着错误纸条。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 copyLimited 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 copyLimited 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 9.1 · 拆去比喻

先学数据怎么流

工程要点

- Reader 允许分块读取；一次 Read 返回的数据和错误都要一起处理。
- Writer 可能短写，优先使用 io.Copy 等已处理循环的标准工具。
- LimitedReader 在进入解码器前限制输入体积，避免无界内存占用。
- Closer 的关闭责任要在获得资源的边界明确登记。

练手

用 strings.Reader 模拟小输入和超限输入，测试复制结果与错误。

验收

实现不一次性读入全部数据；超限可识别；写入错误能返回调用方。

```
01 func copyLimited(dst io.Writer, src io.Reader) error {
02     lr := &io.LimitedReader{R: src, N: 1<<20 + 1}
03     n, err := io.Copy(dst, lr)
04     if err != nil { return err }
05     if n > 1<<20 { return ErrTooLarge }
06     return nil
07 }
```

章内续问 9.2 · 闯祸与群聊

HTTP 默认值不是生产配置

闯祸现场　测试服务回了一封 HTTP 500，响应体还在慢慢滴字节；连接池面板少了一格可用连接。陈平安只看 Do 没返回 error，便把 req-0827 记成成功，连 Body 也忘了关。

陈平安

没传输错误，不就代表成功吗？


阿良
邮差没摔跤，不代表信里写的是喜讯。


李宝瓶
500 和超大响应体都读到底，连接会怎样？

陈平安
复用有超时的 Client，并用请求 Context 取消。


陈平安
拿到响应先 Close；状态和正文都按上限处理。



宁姚
小错误体可有界 drain，超限就明确放弃复用。

收束　那封 500 回信被夹进故障样本册，不再冒充喜讯。

先有画面，再落到工程

HTTP 默认值不是生产配置

先懂一句

请求生命周期要有超时、状态检查、Body 关闭和大小限制。

直觉类比

像寄一封限时快递：要写最晚送达时间，收到回信要看状态，拆完箱子要关好 Body；快递员没迷路，不等于收件人回的是好消息。

术语落地

- goroutine：很轻的小跑堂，可以同时派出很多个
- HTTP：客户端和服务端互寄有格式的信

读代码抓手

- 先看 fetch 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 fetch 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 9.2 · 拆去比喻

HTTP 默认值不是生产配置

工程要点

- 复用带超时配置的 http.Client；Do 只报告传输错误，非 2xx 由调用方判断。
- 拿到 Response 后立即 defer Body.Close，成功与失败响应都限制读取体积。
- 小型非 2xx 响应在上限内 drain 到 EOF 后关闭，以便 Transport 复用连接。
- 错误体超过 drain 上限时立即停止读取并关闭，明确接受该连接不再复用。

练手

用 httptest.Server 连续返回小型 500、超大 500 和超限 2xx，观察连接复用与读取上限。

验收

小型非 2xx 可复用连接；超大错误体有界关闭且明确放弃复用；所有路径关闭 Body 并无 goroutine 泄漏。

记忆钉

1. 复用带超时配置的 http.Client；Do 只报告传输错误，非 2xx 由调用方判断。
2. 拿到 Response 后立即 defer Body.Close，成功与失败响应都限制读取体积。
3. 小型非 2xx 响应在上限内 drain 到 EOF 后关闭，以便 Transport 复用连接。
4. 错误体超过 drain 上限时立即停止读取并关闭，明确接受该连接不再复用。

长代码已拆到后续专页

章内续问 9.2 · 代码 1/2

HTTP 默认值不是生产配置

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```go
var ErrHTTPStatus = errors.New("unexpected HTTP status")

const (
    maxBody       = int64(1 << 20)
    maxErrorDrain = int64(64 << 10)
)

func fetch(ctx context.Context, c *http.Client, url string) error {
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
    if err != nil { return err }
    res, err := c.Do(req)
    if err != nil { return err }
    defer res.Body.Close()

    if res.StatusCode < 200 || res.StatusCode >= 300 {
        statusErr := fmt.Errorf("%w: %d", ErrHTTPStatus, res.StatusCode)
        n, drainErr := io.CopyN(io.Discard, res.Body, maxErrorDrain+1)
        if drainErr != nil && !errors.Is(drainErr, io.EOF) {
            return errors.Join(statusErr, drainErr)
        }
        if n > maxErrorDrain {
            return fmt.Errorf("%w: drain limit; no reuse", statusErr)
        }
        return statusErr
```

下一页继续同一段代码，不切换事故对象。

章内续问 9.2 · 代码 2/2

HTTP 默认值不是生产配置

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
25      }
26
27      n, err := io.Copy(io.Discard, io.LimitReader(res.Body, maxBody+1))
28      if err != nil { return err }
29      if n > maxBody { return ErrTooLarge }
30      return nil
31  }
```

小型非 2xx 可复用连接；超大错误体有界关闭且明确放弃复用；所有路径关闭 Body 并无 goroutine 泄漏。

章内续问 9.3 · 闯祸与群聊

在进程内测完整 HTTP

闯祸现场　临时测试桌上没有固定端口，只有一个 Recorder、一个测试 Server 和三张状态码卡片。
陈平安直接调用 Handler 得到 nil，真实路由却把 POST 写成了 GET，Content-Type 也不见了。

陈平安
函数没报错，契约应该已经通过了。


阮秀
那路由、方法和响应头是谁验的？


李宝瓶
未知字段和取消路径也会经过同一入口吗？

陈平安
Recorder 验单个处理器，NewServer 验完整链路。

陈平安
逐项断言状态、Content-Type 和稳定 JSON 字段。


宁姚
测试关闭 Server 与 Body，不占固定端口。

收束　三张状态码卡片终于都找到了自己的测试用例。

先有画面，再落到工程

在进程内测完整 HTTP

先懂一句

httptest 能验证路由、状态、头部和响应体，而不依赖固定端口。

直觉类比

像在排练室搭一座迷你城门：不用占真实街道，也能把路由、请求头、状态码和回信正文从头走一遍。

术语落地

- HTTP：客户端和服务端互寄有格式的信
- Handler：门房，负责收信和回信

读代码抓手

- 先看 TestHealth 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 最后核对 TestHealth 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 9.3 · 拆去比喻

在进程内测完整 HTTP

工程要点

- NewRequest 与 NewRecorder 适合快速断言 handler 的状态和响应体。
- NewServer 提供临时 URL，覆盖客户端、路由和真实传输交互。
- 断言 Content-Type、状态码和稳定字段，不只查正文包含某段文字。
- 测试服务器必须 Close；客户端重定向策略应按场景显式配置。

练手

为一个 JSON handler 增加成功、未知字段和方法不允许三组 HTTP 测试。

验收

测试断言状态、Content-Type 与 JSON 契约；不绑定固定端口。

```go
func TestHealth(t *testing.T) {
    req := httptest.NewRequest("GET", "/health", nil)
    rec := httptest.NewRecorder()
    healthHandler(rec, req)
    res := rec.Result()
    defer res.Body.Close()
    if res.StatusCode != http.StatusOK {
        t.Fatalf("status=%d", res.StatusCode)
    }
}
```

章内续问 9.4 · 闯祸与群聊

补页 9.A 身份从哪里来：JWT 与 OIDC

闯祸现场　登录页旁放着一枚拆开的 JWT，header 和 payload 清清楚楚，签名校验灯却还没亮。陈平安看见 token 里写着 tenant-7，便相信了客户端自填的 user_id，冒名请求险些进账。

陈平安



Token 能拆开，身份不就可信了吗？



宁姚

能读，不等于有人为这段内容担保。



李宝瓶

算法、issuer、audience 和过期时间谁核对？

陈平安



我在可信边界验证签名、时效、kid 和受众。

陈平安



验真后生成 Principal，不信请求体里的身份。



宁姚

未知 kid 只做有界刷新，日志不留原 token。

收束　签名校验灯亮起后，那枚 token 才被放进通行盒。

先有画面，再落到工程

补页 9.A 身份从哪里来：JWT 与 OIDC

先懂一句

Token 能拆开，不等于身份可信。

直觉类比

像看学生证：能把塑料卡掰开，不等于卡是真的。要验签名、发行学校、使用场所和有效期，最后再生成系统信得过的身份。

术语落地

- 缓存：为了快而做的副本，不是最终事实
- Trace：一次请求一路经过哪些地方的脚印串
- tenant：同一系统里互相不能偷看账本的一户人家

读代码抓手

- 先看 Verify 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Verify 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 9.4 · 拆去比喻

补页 9.A 身份从哪里来：JWT 与 OIDC

工程要点

- 只在 TLS 与可信代理边界后接收凭证，不能信任请求体或任意 X-Tenant-ID。
- JWT 验证固定算法白名单，并核对 issuer、audience、exp、nbf 与 clock skew。
- JWKS 按 kid 缓存；未知 kid 只触发有界刷新，轮换期同时接受明确的新旧密钥。
- OIDC claim 映射为 typed Principal；原始 claim 不直接成为业务授权结论。
- 认证失败统一返回 401 和稳定 code，日志只留 token 指纹，不留原始 token。

练手

注入伪造签名、错误 audience、过期 token、未知 kid 和轮换中的旧 kid。

验收

五类非法凭证全部返回稳定 401；合法凭证生成 Principal；日志与 trace 不出现原始 token。

```go
func Verify(raw string, now time.Time) (Principal, error) {
    tok, err := verifier.Verify(raw, now, allowedLeeway)
    if err != nil { return Principal{}, ErrUnauthenticated }
    if tok.Iss != wantIss || !tok.HasAudience(wantAud) {
        return Principal{}, ErrUnauthenticated
    }
    if tok.Subject == "" || tok.Tenant == "" {
        return Principal{}, ErrUnauthenticated
    }
    if !membership.Allows(tok.Subject, tok.Tenant) {
        return Principal{}, ErrUnauthenticated
    }
    return Principal{
        UserID: tok.Subject, TenantID: tok.Tenant,
    }, nil
}
```

章内续问 9.5 · 闯祸与群聊

补页 9.B 过了山门，还要问能不能动这本账

闯祸现场　认证门禁已经放行，桌上却并排摆着 tenant-7 的两本账簿，封面主人并不相同。陈平安只确认来人已登录，就把 bill-0827 递了过去，对象权限根本没问。

陈平安



同一 tenant 的登录用户，应该都能看吧？



阮秀

认证只说明是谁，不说明能动哪本账。



李宝瓶

跨租户、降权和对象转移后怎么判？

陈平安



Service 按 Principal、动作和当前对象状态授权。

陈平安



拒绝按统一 403/404 策略，不泄露对象内容。



宁姚

只记 audit_id 和策略版本，别记录秘密。

收束　两本账簿重新合上，各自回到有名字的抽屉。

先有画面，再落到工程

补页 9.B 过了山门，还要问能不能动这本账

先懂一句

认证回答你是谁，授权回答这件事你能不能做。

直觉类比

门卫认出你是小明，只回答了‘你是谁’；小明能不能拿走小红的书，还要再查这本书归谁、这次动作允不允许。

术语落地

- error：事情没办成时一起带回来的原因条
- Handler：门房，负责收信和回信
- Service：真正执行业务规矩的账房
- tenant：同一系统里互相不能偷看账本的一户人家

读代码抓手

- 先看 Get 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Get 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 9.5 · 拆去比喻

补页 9.B 过了山门，还要问能不能动这本账

工程要点

- Handler 只提取 Principal；Service 对 create、read、update、delete 分别做授权。
- 对象级策略至少绑定 tenant、subject、role/policy version 与资源状态。
- 403 表示身份可信但权限不足；为防资源枚举可按既定策略返回 404。
- 拒绝路径写入稳定 error code、audit_id、policy version，不泄露对象内容。
- 角色降权、成员移出租户和资源转移都要让旧授权及时失效。

练手

覆盖同租户无权对象、跨租户 ID、降权旧 token、资源不存在四种读取。

验收

四条路径符合既定 403/404 契约；拒绝均有 audit_id；任何响应都不泄露对象是否存在。

```
01  func (s *Service) Get(
02      ctx context.Context, p Principal, id string,
03  ) (Bill, error) {
04      bill, err := s.repo.Get(ctx, p.TenantID, id)
05      if err != nil { return Bill{}, err }
06      if !s.policy.CanRead(p, bill) {
07          return Bill{}, ErrNotFound
08      }
09      return bill, nil
10  }
```

章内续问 9.6 · 闯祸与群聊

补页 9.C 中间件有顺序，Server 也有收剑时刻

闯祸现场　关服演练开始，request ID 贴纸、限流器和 readiness 开关排成一列，SSE 连接还在另一块屏幕上闪。panic 先于编号中间件发生，客户端只收到半截 JSON；SIGTERM 又直接掐断了在途请求。

陈平安

加了 recovery，怎么还是找不到事故请求？


李宝瓶
编号都没贴上，日志拿什么串起来？


阮秀
慢 body 和 SSE 能共用同一读写时限吗？

陈平安
固定编号、恢复、认证、限流和 Handler 次序。


陈平安
关服先摘 readiness，再有界等待普通请求。



宁姚
SSE 与 hijacked 连接单独治理，不能照搬。

收束　最后一张 request ID 贴纸稳稳留在完整响应旁。

先有画面，再落到工程

补页 9.C 中间件有顺序，Server 也有收剑时刻

先懂一句

门口先编号再验人，关门先摘流量再等在途请求。

直觉类比

像商场关门：先给每位来客发编号，再验身份；打烊时先不放新人进来，然后等正在结账的人办完，不能突然拔总电闸。

术语落地

- Handler：门房，负责收信和回信
- SSE：服务端向浏览器单向喊话的小喇叭

读代码抓手

- 先看 newServer 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 newServer 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 9.6 · 拆去比喻

补页 9.C 中间件有顺序，Server 也有收剑时刻

工程要点

- 可信代理名单决定是否采信 Forwarded/X-Forwarded-For，不能全网信任。
- 统一校验 Content-Type、body 上限、request ID、panic recovery 与错误响应。
- ReadHeaderTimeout 只守 header；非流式 body/write 还要用 ReadTimeout、WriteTimeout 或 Handler 级 deadline。
- 关闭时先让 readiness 失败，再用 Shutdown 等待普通在途请求。
- SSE、流式上传和 hijacked 连接需按路由单独治理，不能照搬统一读写时限。

练手

真实启动 Server，注入慢 header、慢 body、panic、SSE 与 SIGTERM。

验收

非流式 header/body 均按时限收口；SSE 不被误杀；panic 不写半截 JSON；摘流后在途请求有界完成。

记忆钉

1. 可信代理名单决定是否采信 Forwarded/X-Forwarded-For，不能全网信任。
2. 统一校验 Content-Type、body 上限、request ID、panic recovery 与错误响应。
3. ReadHeaderTimeout 只守 header；非流式 body/write 还要用 ReadTimeout、WriteTimeout 或 Handler 级 deadline。
4. SSE、流式上传和 hijacked 连接需按路由单独治理，不能照搬统一读写时限。

长代码已拆到后续专页

章内续问 9.6 · 代码 1/2

补页 9.C 中间件有顺序，Server 也有收剑时刻

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```go
func newServer(cfg Config, router http.Handler) *http.Server {
    // SSE uses a dedicated streaming server with route-level deadlines.
    return &http.Server{
        Addr: cfg.Addr, Handler: middlewareChain(router),
        ReadHeaderTimeout: 5 * time.Second,
        ReadTimeout: 15 * time.Second, WriteTimeout: 15 * time.Second,
        IdleTimeout: 60 * time.Second, MaxHeaderBytes: 1 << 20,
    }
}

func stop(srv *http.Server) error {
    var errs []error
    readiness.Store(false)
    drain, cancelDrain := context.WithTimeout(
        context.Background(), 5*time.Second)
    if err := waitForEndpointRemoval(drain); err != nil {
        errs = append(errs, fmt.Errorf("drain: %w", err))
    }
    cancelDrain()
    closeHijackedConnections()
    stop, cancelStop := context.WithTimeout(
        context.Background(), 10*time.Second)
    if err := srv.Shutdown(stop); err != nil {
        errs = append(errs, err)
```

下一页继续同一段代码，不切换事故对象。

章内续问 9.6 · 代码 2/2

补页 9.C 中间件有顺序，Server 也有收剑时刻

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
25          if closeErr := srv.Close(); closeErr != nil {
26              errs = append(errs, closeErr)
27          }
28      }
29      cancelStop()
30      return errors.Join(errs...)
31  }
```

非流式 header/body 均按时限收口；SSE 不被误杀；panic 不写半截 JSON；摘流后在途请求有界完成。

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 binding 与业务校验如何分工？

陈平安



binding 只检查 HTTP 输入的形状、类型和基础约束，Service 再检查 1500 分奶茶账的业务规则与权限；



宁姚

对，门口验包裹，城内判能不能办。

第 2 问 user_id 应从哪里来？

陈平安



user_id 和 tenant_id 来自验证 JWT/OIDC 后生成的 typed Principal，不能相信 body、path 或任意 header；



宁姚

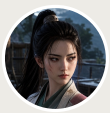
答对，身份必须从可信边界进入。

第 3 问 为何不把所有响应都写成 200？

陈平安



HTTP 状态表达传输结果，稳定 error code 表达业务原因，失败全回 200 会让客户端、代理和监控误判；



宁姚

闭合，201、400、401、404/403 与 409 各守自己的语义。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 10 场 · 闯祸与群聊

第十章 城中三层：Handler、Service、Repository

目标：分清协议、业务与持久化职责，能通过构造函数注入替换依赖。

闯祸现场

白板上画着 Handler、Service、Repository 三个方框，数据库连接串像一把沉甸甸的钥匙挂在陈平安手边。

他嫌绕路，直接在 Handler 里写 SQL，结果授权规则一改，HTTP 测试和数据库测试一起红了。

陈平安



少走两层，代码不是更短吗？



阮秀

通信规则变一次，存储细节也要跟着搬家？



李宝瓶

后台任务以后复用业务规则，也要带着 Gin 吗？

陈平安

Handler 只翻协议，Service 管授权和业务。



陈平安

Repository 只暴露用例所需能力，由 main 注入。





宁姚

业务层接标准库的 Context，不接 Gin 的请求对象。

收束

数据库钥匙被放回了 Repository 方框下面。

先有画面，再落到工程

第十章 城中三层：Handler、Service、Repository

先懂一句

协议在门外，规矩在城中，存取在库房。

直觉类比

门房别一边验信一边钻进库房翻箱子。Handler 收信，Service 判规矩，Repository 存取账本，各站各的岗，坏了才知道该找谁。

术语落地

- Context：沿调用链传下去的取消铃和截止钟
- HTTP：客户端和服务端互寄有格式的信
- Handler：门房，负责收信和回信
- Service：真正执行业务规矩的账房
- Repository：专门和数据库打交道的库管
- 事务：要么整组成功，要么整组撤回

读代码抓手

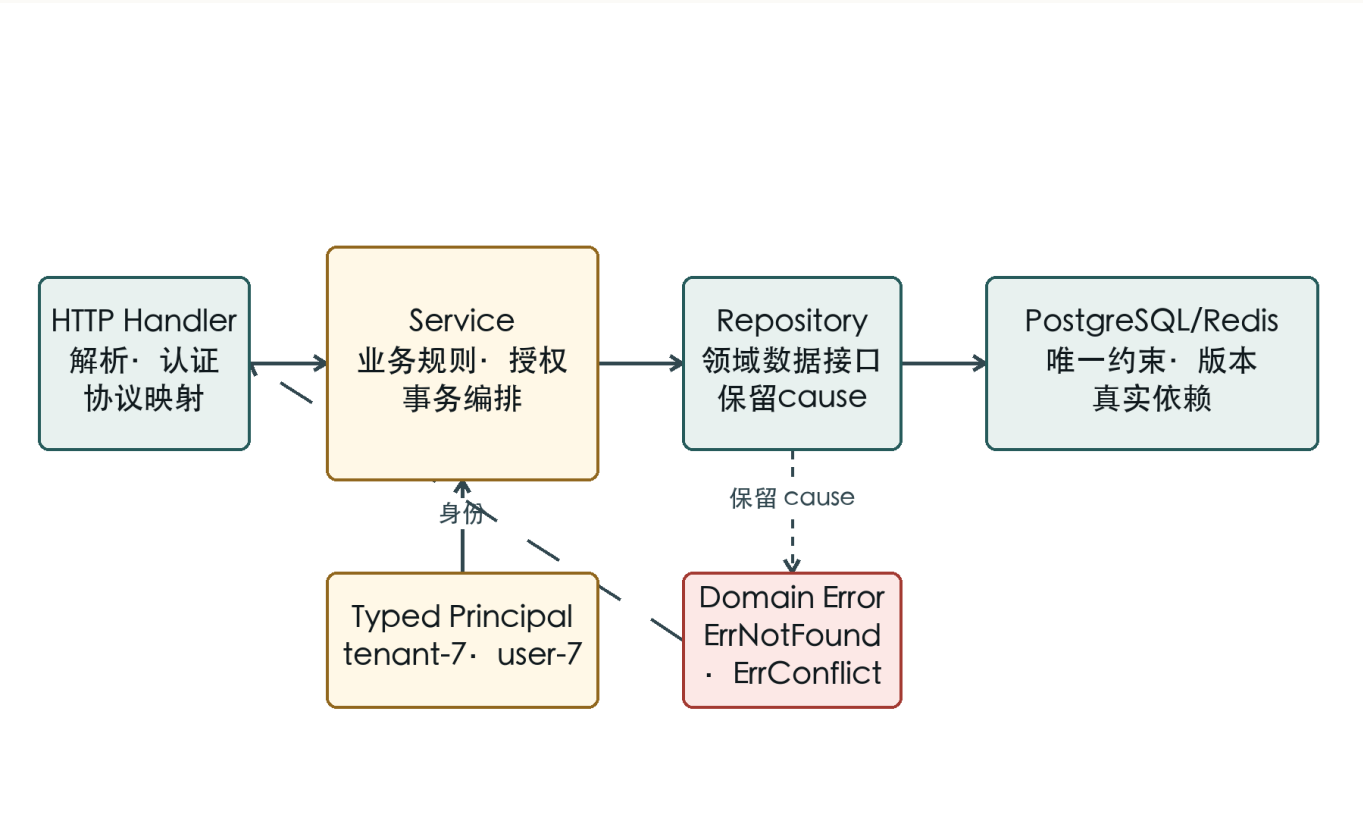
- 先看 Create 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Create 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 10.1

Handler、Service、Repository：职责与依赖方向

小白先懂：协议在门外，规矩在城中，存取在库房。



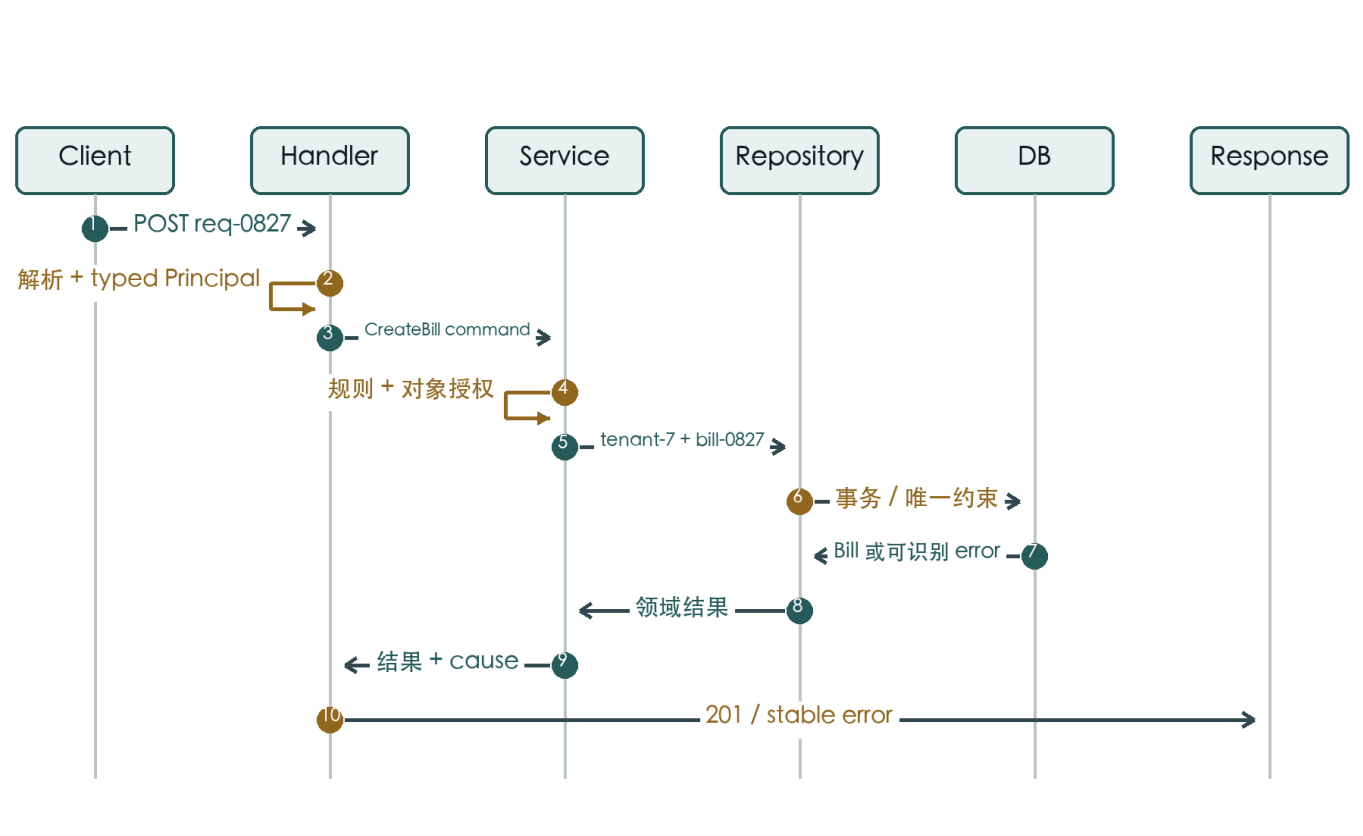
拆去比喻

- Handler 负责协议、认证结果提取和 HTTP 错误翻译，不把 Gin Context 传入业务层。
- Service/use case 负责授权、业务规则和事务编排。
- Repository 只暴露当前用例需要的最小能力，并返回领域或持久化错误。
- composition root 负责注入 SQLRepository 或 FakeRepository，不把依赖选择藏在业务代码里。

图 10.2

req-0827 穿过分层的真实时序

小白先懂：协议在门外，规矩在城中，存取在库房。



拆去比喻

- req-0827 的身份、tenant 与 command 沿调用链显式传递。
- 数据库约束负责最终并发裁决，HTTP 只稳定映射结果。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
type BillRepository interface {
    Save(context.Context, Bill) (Bill, error)
}

type Service struct { repo BillRepository }
func (s *Service) Create(
    ctx context.Context, p Principal, cmd CreateBill,
) (Bill, error) {
    if !p.CanWriteBills() { return Bill{}, ErrForbidden }
    bill, err := NewBill(p.TenantID, p.UserID, cmd)
    if err != nil { return Bill{}, err }
    return s.repo.Save(ctx, bill)
}

type Handler struct {
    service *Service
}
// Only Handler translates domain errors to HTTP.
```

失败路径

- init 中连数据库
- 依赖由业务包偷偷创建
- readiness 永远返回 200

练剑任务

用 FakeRepository 测试 Service 的授权失败、领域校验失败和持久化失败。

验收证据

测试不启动 HTTP/数据库即可证明分层；
Repository 错误只在 Handler 边界翻译为状态码。

收剑自检

- [ ] Repository 可以返回 HTTP 状态吗？
- [ ] Service 为什么不接 Gin Context？
- [ ] 分层价值如何用测试证明？

章内续问 10.1 · 闯祸与群聊

补页 10.A 多租户不是一个 Header

闯祸现场　Repository 查询窗口里只有 bill_id，没有 tenant_id；两张不同租户的同号账单在结果列表里挨到了一起。Service 明明传下了 tenant-7，SQL 却把这道条件丢在门外，测试立即捞回了别人的账。

陈平安：上层已经验过 tenant，SQL 可以省一次吧？

宁姚：上层职责和查询铁门不能互相抵消。

阮秀：连接复用时，RLS 留下的租户状态谁清理？

陈平安：存取接口必须接 tenant，每条查询都带租户条件。

陈平安：复合约束含 tenant，RLS 会话设置后必须清理。

宁姚：管理任务走独立能力、连接角色和审计通路。

收束　两张同号账单被 tenant 标签分进了不同抽屉。

先有画面，再落到工程

补页 10.A 多租户不是一个 Header

先懂一句

应用层守一次，查询层再落一道 tenant 铁门。

直觉类比

像公寓楼的信箱：门牌写 tenant，钥匙来自可信门卫。应用先核一次，数据库查询再带一次门牌条件，不能靠住户自己喊‘我是七楼的’。

术语落地

- Repository：专门和数据库打交道的库管
- tenant：同一系统里互相不能偷看账本的一户人家

读代码抓手

- 先看 Get 的输入和返回值：它接什么，最后必须交回什么。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Get 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 10.1 · 拆去比喻

补页 10.A 多租户不是一个 Header

工程要点

- TenantID 只来自可信 Principal，不来自 header、path 或 body 的自由输入。
- Repository API 强制接收 tenant；查询、更新、删除都在 SQL 中带 tenant 条件。
- 唯一约束与外键优先包含 tenant_id，避免同号对象跨租户串联。
- PostgreSQL RLS 可作纵深防御，但连接复用时必须正确设置并清理会话租户。
- 管理任务与跨租户报表使用独立 capability、连接角色和审计通路。

练手

在普通 API、后台任务和导出查询中各注入一次缺失 tenant 条件。

验收

静态 SQL 断言或集成测试捕获三处遗漏；RLS 再拒绝跨租户访问；审计能区分普通与管理通路。

```
const getBill = `
SELECT id, amount_cents, version
FROM bills
WHERE tenant_id = $1 AND id = $2`

func (r *Repo) Get(ctx context.Context,
    tenant, id string) (Bill, error) {
    row := r.db.QueryRowContext(ctx, getBill, tenant, id)
    return scanBill(row)
}
```

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 Repository 可以返回 HTTP 状态吗？

陈平安



Repository 只返回数据结果和可识别领域/存储错误，不返回 HTTP 状态；



宁姚

对，由 Handler 在协议边界统一映射状态码和错误体。

第 2 问 Service 为什么不接 Gin Context？

陈平安



Service 可以接标准 context.Context，却不接 gin.Context，否则业务会被 Gin 绑住；



宁姚

准确，这样 HTTP、Worker 和测试都能复用同一业务层。

第 3 问 分层价值如何用测试证明？

陈平安



给 Service 注入 fake Repository，给 Handler 注入 fake Service，分别验证业务与协议，不启动无关依赖；



宁姚

这就用可替换测试证明了分层价值。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 11 场 · 闯祸与群聊

第十一章 真账本：SQL、事务与幂等

目标：掌握金额表示、schema、事务、唯一约束、幂等和兼容迁移。

闯祸现场　手机网络转了一圈，req-0827 又落到桌面；账本旁边放着一枚写有 amount_cents=1500 的印章。陈平安先查数据库没看见旧记录，两次并发请求便同时落笔，奶茶账凭空多了一份。


陈平安：我先查再写，怎么还会重复？


阮秀：两个人都能在落笔前看到空白。


李宝瓶：同 key 换了金额，也要返回第一次结果吗？


陈平安：幂等域绑定 tenant、动作和请求指纹。


陈平安：原子 Claim 后同事务写账与事件，唯一约束裁决。


宁姚：同 key 异载荷回 409；旧结果仍按当前权限重验。

收束　那枚印章只落下一次，桌面总算安静了。

先有画面，再落到工程

第十一章 真账本：SQL、事务与幂等

先懂一句

钱不用浮萍称，去重靠最后一道铁门。

直觉类比

数据库像真账房。转账不能只改左边不改右边；同一张付款单拍桌子两次，也不能扣两次钱。事务守整锅，幂等守重复。

术语落地

- 事务：要么整组成功，要么整组撤回
- 幂等：同一件事喊两遍，也只真正办一次
- tenant：同一系统里互相不能偷看账本的一户人家

读代码抓手

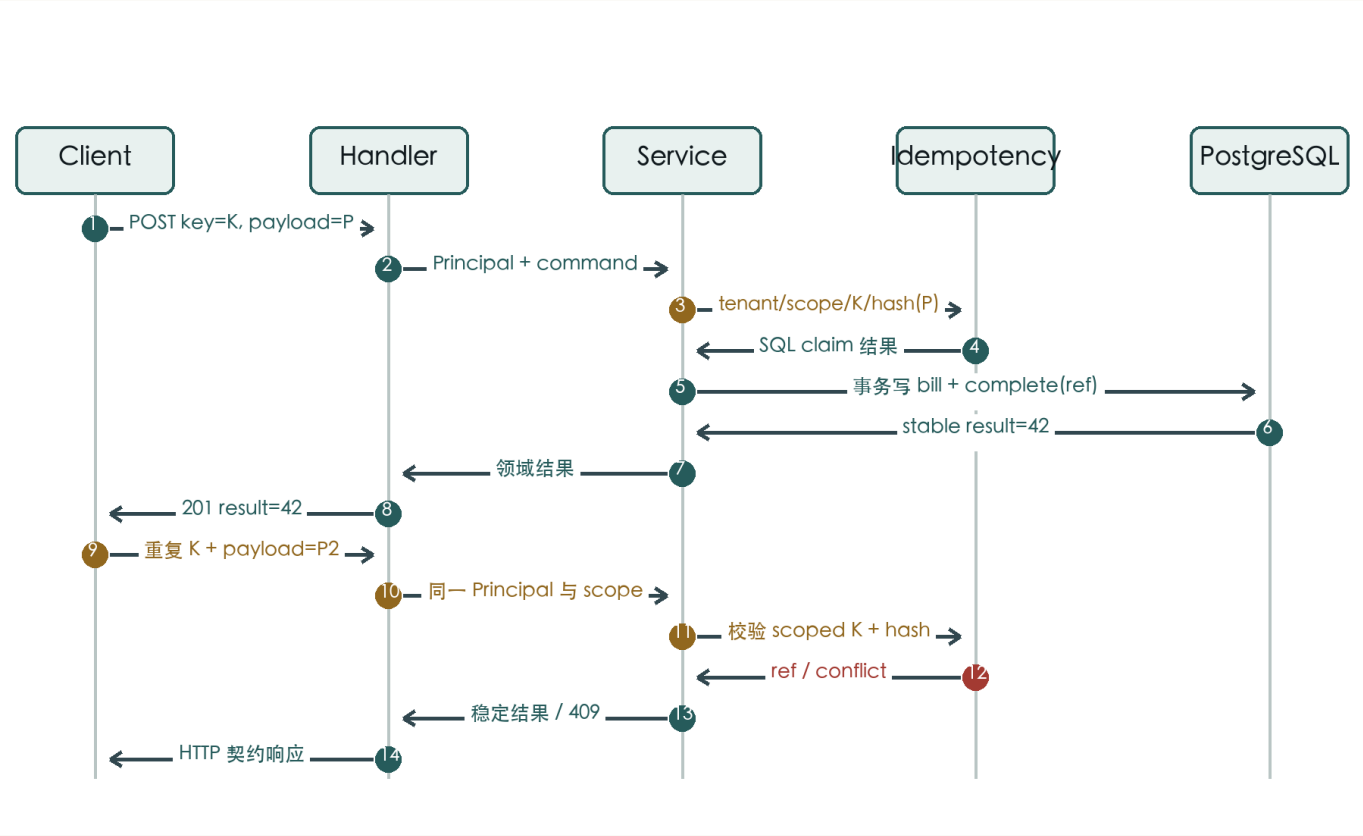
- 先看 Create 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 找到稳定身份：重试时哪个号码绝不能换，回执又和谁绑定。
- 最后核对 Create 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 11.1

事务、唯一约束与幂等写入时序

小白先懂：钱不用浮萍称，去重靠最后一道铁门。



拆去比喻

- 幂等唯一键必须绑定 tenant、认证主体或明确共享范围、动作 scope 与 key；同主体同 key 不同 payload 才稳定返回 409。
- IdempotencyRecord 与 claim 在同一事务里保存 SubjectID 和请求指纹；读取任何 hash/status 前先核对主体边界，避免同租户状态探针。
- 原子 Claim 先落 processing；完成后只保存稳定 result ref，不复制副作用结果。
- 命中 completed 旧记录时仍按当前 Principal 和当前 Bill 状态重新授权，不能把幂等键当读取凭证。
- 数据库唯一约束覆盖 tenant+subject+action+key；跨主体或跨租户碰撞彼此独立，保留期与清理策略书面固定。

代码完整展开，不牺牲阅读字号

小白先懂：钱不用浮萍称，去重靠最后一道铁门。

```
type IdempotencyRecord struct {
    TenantID, SubjectID, Action, Key string
    RequestHash, Status, ResultRef string
}

func (s *Service) Create(
    ctx context.Context, p Principal, cmd CreateBill,
) (Bill, error) {
    if !p.CanWriteBills() { return Bill{}, ErrForbidden }
    claim := IdempotencyClaim{
        TenantID: p.TenantID, SubjectID: p.SubjectID,
        Action: "bill:create", Key: cmd.Key,
        RequestHash: canonicalHash(cmd),
    }
    record, claimed, err := s.idem.Claim(ctx, claim)
    if err != nil { return Bill{}, err }
    if !claimed {
        if record.TenantID != p.TenantID ||
            record.SubjectID != p.SubjectID {
            return Bill{}, ErrNotFound
        }
        if record.RequestHash != claim.RequestHash {
            return Bill{}, ErrIdempotencyConflict
        }
        if record.Status == "processing" {
            return Bill{}, ErrIdempotencyInProgress
        }
        if record.Status != "completed" {
            return Bill{}, ErrIdempotencyRetryable
        }
        bill, err := s.bills.GetByRef(ctx, p.TenantID, record.ResultRef)
        if err != nil { return Bill{}, err }
        if !s.policy.CanRead(p, bill) { return Bill{}, ErrNotFound }
        return bill, nil
    }
    return s.createAndComplete(ctx, p, record, cmd)
}
```

失败要能复现，完成要有证据

失败路径

- 金额用浮点
- 把事务当成跨服务万能锁
- 只在应用层做重复检查

练剑任务

并发提交同主体同 key 同/异 payload、同租户不同 SubjectID 和跨租户碰撞，再模拟 Commit 回包丢失与完成后撤权。

验收证据

同主体同摘要只写一次、异摘要 409；不同主体各自独立且看不到对方 hash/status；旧结果仍按当前对象权限授权。

收剑自检

- [ ] 为何先查再写不能防并发重复？
- [ ] 事务与幂等有何不同？
- [ ] 金额字段为何带单位？

章内续问 11.1 · 闯祸与群聊

补页 11.A 事务模板与 Commit 结果不确定

闯祸现场　事务面板停在 Commit，网络线图标忽然断开，req-0827 的回信没有回来。
陈平安把 Commit error 当成未落账，准备再写一次；数据库里却可能已经有 bill-0827。

陈平安
Commit 报错，不就是事务失败了吗？



阮秀
回信丢失时，印章可能已经盖下。



李宝瓶
Begin、事务体和 Commit 错误能同样重试吗？

陈平安
事务助手统一开始、回滚和提交结果分类。

陈平安
Commit 不明标 Unknown，用幂等键查询结果。



宁姚
不盲重放；原子写必须共用同一个 sql.Tx。

收束　断开的网线旁，多了一张醒目的 Unknown 标签。

先有画面，再落到工程

补页 11.A 事务模板与 Commit 结果不确定

先懂一句

Rollback 能收残局，Commit 失败却未必说明没落账。

直觉类比

像小卖部结账：扣库存、收钱、记流水是一只托盘，要么一起端走，要么一起退回。可收银机吐纸失败时，钱到底扣没扣，不能只凭没看到小票判断。

术语落地

- Repository：专门和数据库打交道的库管
- 事务：要么整组成功，要么整组撤回
- 幂等：同一件事喊两遍，也只真正办一次
- outbox：和业务账一起落库、之后再送出的发件箱

读代码抓手

- 先看 WithTx 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 WithTx 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 11.1 · 拆去比喻

补页 11.A 事务模板与 Commit 结果不确定

工程要点

- 事务 helper 统一 BeginTx、defer Rollback、panic 收口和 Commit 错误分类。
- 所有参与原子性的 Repository 方法必须使用同一个 *sql.Tx。
- 业务错误回滚；Commit 返回错误时结果可能未知，不能盲目重放非幂等写。
- 外部网络调用不放进长事务；先落状态/outbox，再由事务外执行。
- 记录 tx duration、rollback cause 与 commit ambiguity，禁止记录敏感 SQL 参数。

练手

分别在 Begin、业务写、Commit 回包三个位置注入错误，并模拟 Commit 后连接断开。

验收

无连接泄漏；业务失败必回滚；Commit 不明不直接重复副作用，并可凭幂等记录查询稳定结果。

```go
func WithTx(ctx context.Context, db *sql.DB,
    fn func(*sql.Tx) error) (err error) {
    tx, err := db.BeginTx(ctx, nil)
    if err != nil { return TxError{Stage: "begin", Err: err} }
    defer func() {
        if p := recover(); p != nil {
            _ = tx.Rollback(); panic(p)
        }
        if err != nil { _ = tx.Rollback() }
    }()
    if err = fn(tx); err != nil {
        return TxError{Stage: "body", Err: err}
    }
    if err = tx.Commit(); err != nil {
        return TxError{Stage: "commit", Err: err, Unknown: true}
    }
    return nil
}
```

章内续问 11.2 · 闯祸与群聊

补页 11.B 隔离级别不是越高越好

闯祸现场　两台设备同时打开 bill-0827，屏幕上都显示 version=3，一边改金额，一边等保存。
陈平安只看 UPDATE 影响零行就返回冲突，连不存在和跨租户对象也被一起暴露成了 409。

陈平安

零行更新都算版本冲突，不够统一吗？


宁姚
不存在、越界和过期版本不是同一件事。


阮秀
你先在哪个 tenant/owner 范围读当前对象？

陈平安

先在可信作用域锁定并读取当前 version。

陈平安

只在 expected 不同时回 ErrConflict 和 409。


宁姚
不存在或越界回 ErrNotFound；持锁后异常是内部错误。

收束　两台设备一台保存成功，另一台老实地重新读取。

先有画面，再落到工程

补页 11.B 隔离级别不是越高越好

先懂一句

锁住一行、校验版本、重跑事务，三把剑各有代价。

直觉类比

像只剩一个限量玩具：先把那一格锁住，再核对你看到的版本；别人已经买走时叫版本冲突，不存在或不是你的玩具则不能假装成冲突。

术语落地

- HTTP：客户端和服务端互寄有格式的信
- 事务：要么整组成功，要么整组撤回
- tenant：同一系统里互相不能偷看账本的一户人家
- owner：这份资源真正归属的那个人或主体

读代码抓手

- 先看 UpdateBill 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 UpdateBill 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 11.2 · 拆去比喻

补页 11.B 隔离级别不是越高越好

工程要点

- 先在可信 tenant/owner 作用域内读取当前对象；不存在或越界统一返回 ErrNotFound。
- 当前 version 与 expected version 不同才是 ErrConflict，并映射 HTTP 409/bill_version_conflict。
- 持锁后更新仍非一行属于内部一致性错误，不能把不存在或越界伪装成版本冲突。
- 只对白名单 deadlock/serialization failure 重试整个事务；ErrConflict 交给调用方重新读取。

练手

并发修改同一 bill：验一次成功、一次版本冲突；不存在与跨租户 ID 均返回 ErrNotFound。

验收

过期 version 稳定返回 409/bill_version_conflict；不存在与跨租户对象不泄露存在性；成功更新不会覆盖先提交版本。

```
var ErrConflict = errors.New("bill version conflict")

func (r *Repo) UpdateBill(ctx context.Context, tx *sql.Tx,
    tenantID, id string, expected, amount int64) error {
    // Both helpers scope every query by tenantID + id.
    current, err := r.lockVersion(ctx, tx, tenantID, id)
    if errors.Is(err, sql.ErrNoRows) { return ErrNotFound }
    if err != nil { return err }
    if current != expected { return ErrConflict }
    rows, err := r.updateAtVersion(
        ctx, tx, tenantID, id, expected, amount)
    if err != nil { return err }
    if rows != 1 { return ErrInvariant }
    return nil
}
```

章内续问 11.3 · 闯祸与群聊

补页 11.C 连接池是一道总容量预算

闯祸现场　容量白板上画着十个服务副本，每个副本都把 MaxOpenConns 写成了五十。陈平安只看单实例等待很短，数据库门口却被五百条连接和迁移任务一起堵住。

陈平安：每个池都不大，为什么数据库还是满了？



阮秀：你数了小队，没有数整座大厅。



李宝瓶：运维、迁移和后台任务的连接留在哪里？

陈平安：按实例总数计算预算，并预留 job 与 ops。

陈平安：查询设时限，结果集、语句和事务都逐条关闭。



宁姚：池满要拒绝或降级，不能继续借未来容量。

收束　白板上的五百被圈住，旁边终于写上了总预算。

先有画面，再落到工程

补页 11.C 连接池是一道总容量预算

先懂一句

每座院子十口井，十座院子就是一百口。

直觉类比

连接池像食堂座位：每桌只摆十把椅子，十个服务就是一百把。椅子开得太多，厨房先被挤爆；太少，门口又排长队。

术语落地

- goroutine：很轻的小跑堂，可以同时派出很多个
- Context：沿调用链传下去的取消铃和截止钟

读代码抓手

- 先看 ConfigurePool 的输入和返回值：它接什么，最后必须交回什么。
- 最后核对 ConfigurePool 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 11.3 · 拆去比喻

补页 11.C 连接池是一道总容量预算

工程要点

- 连接池不是吞吐开关；数据库 CPU、锁和 max_connections 才是总边界。
- 配置 MaxOpen、MaxIdle、ConnMaxLifetime 与 ConnMaxIdleTime，并加入随机抖动。
- 查询必须带 context deadline；rows、stmt 与 tx 都要在所有路径关闭。
- 扩容前计算实例总池、迁移 job、只读副本和运维保留连接。
- 池饱和时明确拒绝或降级，不能让请求无限等待并拖垮 goroutine。

练手

用多实例预算模拟池饱和、慢查询、请求取消与连接生命周期集中到期。

验收

总连接不越数据库预算；取消后连接归还；等待时间有指标；连接轮换不形成同刻重连尖峰。

```go
func ConfigurePool(db *sql.DB, c PoolConfig) {
    db.SetMaxOpenConns(c.MaxOpen)
    db.SetMaxIdleConns(c.MaxIdle)
    db.SetConnMaxLifetime(c.MaxLifetime)
    db.SetConnMaxIdleTime(c.MaxIdleTime)
}

// instances*MaxOpen + jobs + ops < DB max_connections
```

章内续问 11.4 · 闯祸与群聊

补页 11.D 事务外盒：Outbox、Inbox 与至少一次投递

闯祸现场　账本已经写下 bill-0827，旁边那封 BillCreated 还压在发送盘底下。进程在 Publish 前退出，数据库里有账，Broker 却永远等不到消息。

陈平安：提交后立刻 Publish，窗口应该很小。



阮秀：窗口再小，也足够进程在里面倒下。



李宝瓶：重复投递时，消费副作用怎么只落一次？

陈平安：业务行和 outbox 在同一事务写入。

陈平安：consumer 用 inbox(event_id) 与本地副作用同事务去重。



宁姚：至少一次投递可重复，副作用必须认同一个身份。

收束　那封信被夹进账本夹层，再也不怕落在桌下。

先有画面，再落到工程

补页 11.D 事务外盒：Outbox、Inbox 与至少一次投递

先懂一句

账与信同炉落地，送信可以重来，副作用只能认一次。

直觉类比

像生日礼物和发件卡一起锁进柜子：礼物入账时卡片也必须在。邮差可以重送卡片，收件箱却要记住已经签收，不能因此再发一份礼物。

术语落地

- 事务：要么整组成功，要么整组撤回
- 幂等：同一件事喊两遍，也只真正办一次
- Replay：拿旧证据重新走一遍，但要先说清是哪种重走
- outbox：和业务账一起落库、之后再送出的发件箱
- inbox：收件方用来记住这封信已经处理过的登记簿

读代码抓手

- 先看 createBill 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 createBill 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 11.4 · 拆去比喻

补页 11.D 事务外盒：Outbox、Inbox 与至少一次投递

工程要点

- 业务写与 outbox insert 共用事务，消除数据库与 Broker 的双写窗口。
- Relay 用 FOR UPDATE SKIP LOCKED 有界认领，失败按 available_at 退避重试。
- consumer 在同一事务写 inbox(event_id) 与本地副作用；外部副作用改写本地 outbox。
- aggregate_id 配合单调 sequence；只认领队首事件，前序失败会阻塞后继。
- 毒消息进入 DLQ；人工 replay 经过权限、租户、幂等与审计门禁。

练手

在 Commit 前、Commit 后 Publish 前、Publish 后 Ack 前、消费写入后 Ack 前分别 kill 进程。

验收

事件最终不丢；重复投递不重复副作用；同 aggregate 顺序可解释；DLQ 回放不绕过授权。

```go
func createBill(tx *sql.Tx, b Bill, e Event) error {
    if err := bills.Insert(tx, b); err != nil { return err }
    if err := outbox.Insert(tx, e); err != nil { return err }
    return nil
}

func consume(tx *sql.Tx, e Event) error {
    fresh, err := inbox.InsertOnce(tx, e.ID)
    if err != nil || !fresh { return err }
    if err = projector.Apply(tx, e); err != nil { return err }
    return localOutbox.Insert(tx, e.FollowUp())
}
```

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 为何先查再写不能防并发重复？

陈平安



先查再写之间仍有并发窗口，两个 req-0827 都可能查到不存在；



宁姚

对，必须靠同事务幂等占位和数据库唯一约束作最后裁决。

第 2 问 事务与幂等有何不同？

陈平安



事务保证单次尝试原子，幂等识别多次尝试是否同一请求；



宁姚

答对，所以 Commit unknown 不能盲重放，要拿 req-0827 查稳定结果。

第 3 问 金额字段为何带单位？

陈平安



amount_cents 明写最小整数单位，1500 就是 15 元，既避开浮点误差也避免单位歧义；



宁姚

正确，跨 API、库表和事件都保持同一口径。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 12 场 · 闯祸与群聊

第十二章 两座库房：Redis 与缓存一致性

目标：掌握 cache-aside、TTL、singleflight、穿透/击穿和分布式锁边界。

闯祸现场　窗边的小风扇吹着一叠 Redis 缓存卡片，最上面仍写着“没有 bill-0827”。
数据库已经提交新账，陈平安却先删缓存再 Commit；并发读请求趁空回源，又把旧列表填了回来。

陈平安

删过缓存了，旧数据怎么又回来了？


阮秀
真账没提交时，回源读到的仍是旧版本。


李宝瓶
角色刚降权，缓存命中还能直接返回吗？

陈平安
命中和回源都重新做对象授权。


陈平安
Commit 后失效，并用版本 CAS 拒绝迟到回填。



宁姚
缓存是投影；故障也不能伪装成普通 miss。

收束　旧卡片被风吹进回收盒，没有再飘回桌上。

先有画面，再落到工程

第十二章 两座库房：Redis 与缓存一致性

先懂一句

真本只有一个，缓存只是拓印。

直觉类比

Redis 像贴在冰箱门上的便签：看得快，但可能过时。真配方还在食谱里；便签丢了可以重抄，不能反过来逼食谱迁就便签。

术语落地

- 事务：要么整组成功，要么整组撤回
- Redis：跑得快的临时小抄，不是真账本
- 缓存：为了快而做的副本，不是最终事实
- tenant：同一系统里互相不能偷看账本的一户人家
- outbox：和业务账一起落库、之后再送出的发件箱

读代码抓手

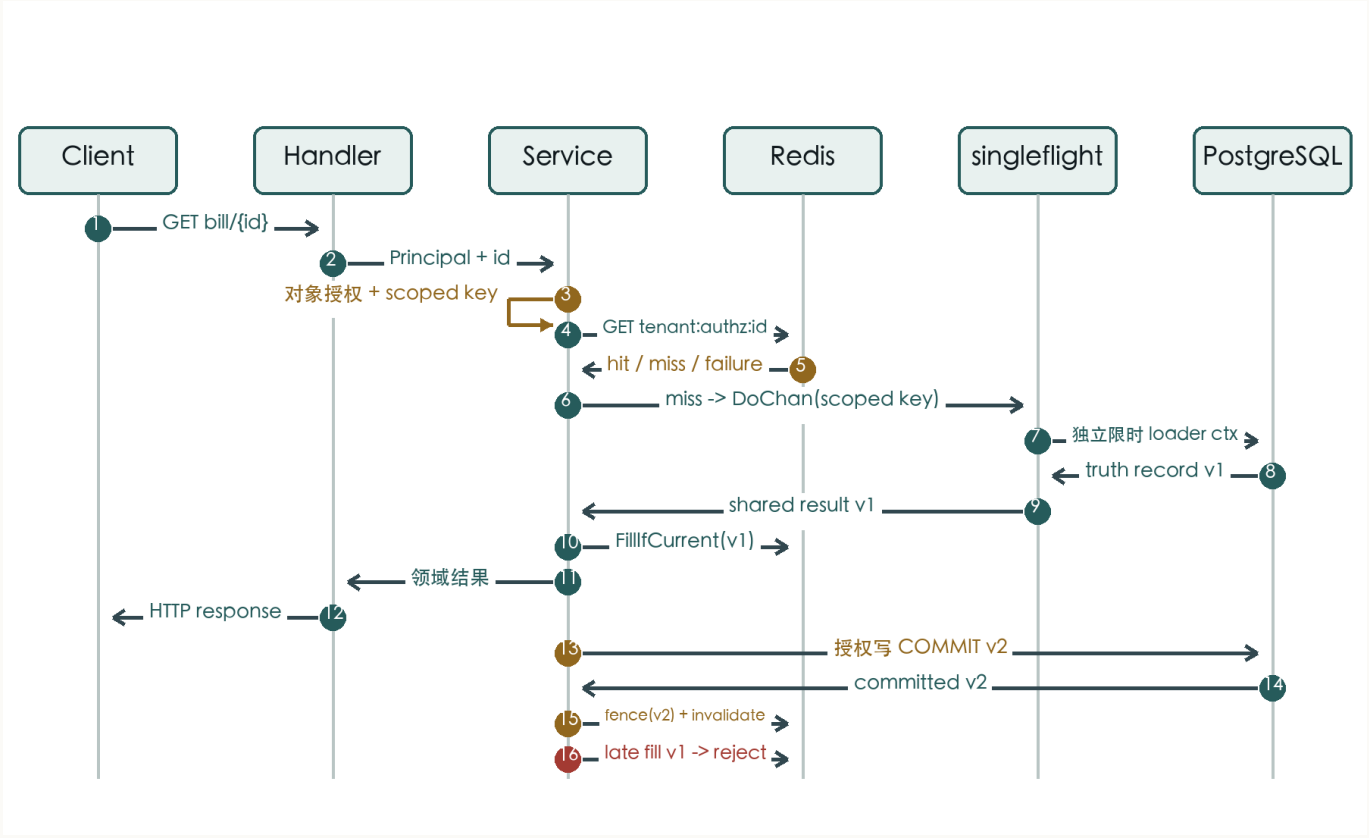
- 先看 GetBill 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 GetBill 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 12.1

Cache-Aside 读写与缓存失效时序

小白先懂：真本只有一个，缓存只是拓印。



拆去比喻

- cache miss 是明确的 key 不存在；超时、断连和饱和是 cache fault，必须进入可观测的降级或拒绝路径，不能伪装成普通 miss 撞向数据库。
- 缓存命中必须对 entry.Bill 执行 policy.CanRead(p, bill)，tenant key 不能替代对象级授权。
- 数据库回源也先得到 Bill，再执行同一 policy.CanRead；两条路径统一按既定 403/404 策略拒绝。
- 缓存条目绑定 membership、policy、object ACL 与 Bill data version；任一版本变化都使旧条目失效。
- 写事务原子递增 data/authz version 并写 outbox；SetIfCurrent 用 durable version ledger 做 CAS，拒绝迟到旧回填。

代码完整展开 1/2

保持字号，按安全行数拆页

小白先懂：真本只有一个，缓存只是拓印。

```
type CacheVersions struct {
    Membership, Policy, ObjectACL, Data uint64
}

type CachedBill struct {
    Bill Bill
    Versions CacheVersions
}

func (s *Service) GetBill(
    ctx context.Context, p Principal, id string,
) (Bill, error) {
    key := "bill:" + p.TenantID + ":" + id
    entry, cacheErr := s.cache.Get(ctx, key)
    switch {
    case cacheErr == nil:
        current, err := s.versions.Current(ctx, p, id)
        if err != nil { return Bill{}, err }
        if entry.Versions == current {
            if !s.policy.CanRead(p, entry.Bill) {
                return Bill{}, ErrNotFound
            }
            return entry.Bill, nil
        }
        if err := s.cache.Delete(ctx, key); err != nil {
            s.metrics.CacheDeleteFault(err)
        }
    case errors.Is(cacheErr, ErrCacheMiss):
        // A true miss may use the bounded DB fill path below.
    default:
        s.metrics.CacheReadFault(cacheErr)
        return Bill{}, errors.Join(ErrCacheUnavailable, cacheErr)
```

下一页继续同一段代码，最后一页再统一核对失败路径与验收证据。

代码完整展开 2/2

保持字号，按安全行数拆页

小白先懂：真本只有一个，缓存只是拓印。

```
33      }
34
35      bill, err := s.repo.Get(ctx, p.TenantID, id)
36      if err != nil { return Bill{}, err }
37      if !s.policy.CanRead(p, bill) { return Bill{}, ErrNotFound }
38      snapshot, err := s.versions.Current(ctx, p, bill.ID)
39      if err != nil { return Bill{}, err }
40      snapshot.Data = bill.Version
41      if err := s.cache.SetIfCurrent(
42          ctx, key, CachedBill{bill, snapshot},
43      ); err != nil { s.metrics.CacheFillFault(err) }
44      return bill, nil
45  }
```

代码到此完整展开；下一页统一核对失败路径、任务、证据与自检。

失败要能复现，完成要有证据

失败路径

- 把 Redis 当唯一事实源
- 缓存 miss 与网络故障混为一谈
- 静态锁值配合直接 DEL

练剑任务

为 cache Get 注入 key 不存在、超时、连接重置和饱和四种结果；只有 ErrCacheMiss 可以进入有界 DB 回填，其余故障按容量契约拒绝并记指标。

验收证据

正常命中和真实 miss 都保持对象授权；cache fault 不伪装成 miss、不造成无界 DB 扇出；删除和回填故障均可观测，迟到旧值仍被版本 CAS 拒绝。

收剑自检

- [ ] 缓存 miss 与 Redis 故障如何区分？
- [ ] 为何先提交数据库再失效缓存？
- [ ] 锁为什么要唯一 token？

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 缓存 miss 与 Redis 故障如何区分？

陈平安



cache miss 是明确的 key 不存在，Redis 故障是超时或连接错误，前者可回源，后者要按降级与容量契约处理；



宁姚

对，不能把故障伪装成普通 miss 去撞库。

第 2 问 为何先提交数据库再失效缓存？

陈平安



若先删缓存再提交，别的请求可能回源旧数据并重新填回；



宁姚

答对，先 Commit 真账，再失效抄本，并让失败可重试可观测。

第 3 问 锁为什么要唯一 token？

陈平安



唯一 token 证明锁的 owner，释放时原子 compare-and-delete，才不会删掉过期后别人刚拿到的锁；



宁姚

闭合，固定 value 加直接 Del 不能过关。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 13 场 · 闯祸与群聊

第十三章 山洪过窄桥：并发闸门与过载拒绝

目标：掌握并发上限、非阻塞拒绝、取消竞态和过载证据；明确它与真正 Worker Pool 的差异。

闯祸现场　夜班桌上排着一列 BillCreated 纸条，队尾已经绕过了咖啡机，Worker 的指示灯却只亮着两盏。陈平安把 channel 容量调得更大，内存曲线随即抬头，超时请求还在队尾继续排队。

陈平安

队列再大一点，总能先装下吧？


阮秀
装下压力，不等于处理掉压力。


李宝瓶
队满时，是等、拒绝，还是悄悄丢掉？

陈平安

我固定并发槽位；满载就明确拒绝，不让请求暗中排队。

陈平安

只给可恢复且幂等安全的错误统一重试预算。


宁姚
背压必须可见；无界排队不在选项里。

收束　纸条停在警戒线内，咖啡机重新露出了门。

先有画面，再落到工程

第十三章 山洪过窄桥：并发闸门与过载拒绝

先懂一句

闸门限制同时出剑，队列才负责排队等候。

直觉类比

窄桥一次只能过十个人。闸门管桥上有几个，队列管桥头排多长；人太多时要明确说‘现在进不去’，不能让整座山的人一起挤上桥。

术语落地

- channel：小跑堂之间传东西的窗口

读代码抓手

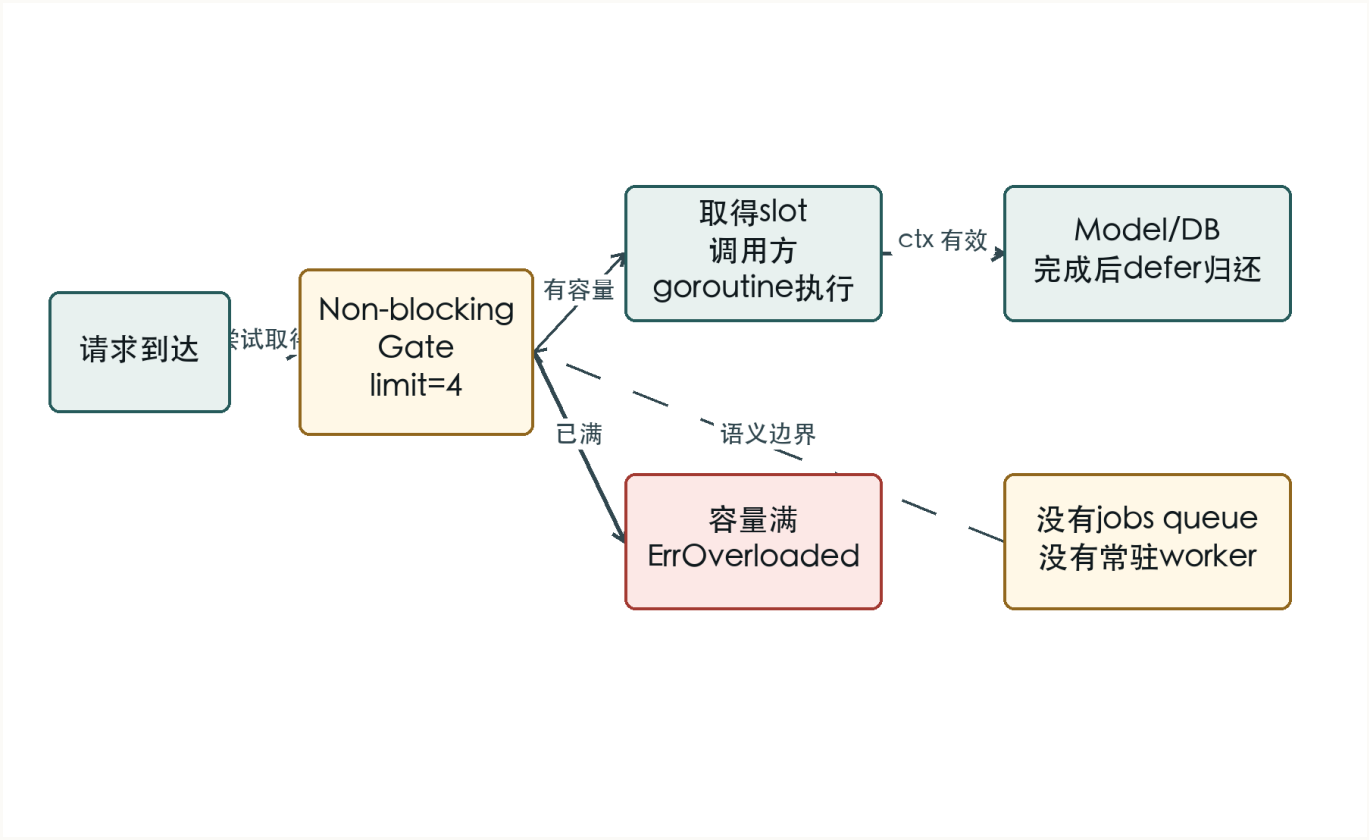
- 先看 NewGate 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 NewGate 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 13.1

并发闸门、取消竞态与过载拒绝

小白先懂：闸门限制同时出剑，队列才负责排队等候。



拆去比喻

- semaphore/gate 只限制同时执行数；它不是 jobs queue，也没有常驻 worker 生命周期。
- 容量已满时立即拒绝并返回稳定 overload 错误，调用方据 Retry-After 或预算决定是否重试。
- 取得 slot 后的复检定义准入点；准入后若再取消，由 fn 继续协作收口。
- 真正 Worker Pool 还需要有界队列、关闭所有权、等待策略和 worker 收口证据。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
type Gate struct { slots chan struct{} }

func NewGate(limit int) (*Gate, error) {
    if limit <= 0 { return nil, ErrInvalidLimit }
    return &Gate{slots: make(chan struct{}, limit)}, nil
}
func (g *Gate) Do(
    ctx context.Context, fn func(context.Context) error,
) error {
    select {
    case <-ctx.Done(): return ctx.Err()
    case g.slots <- struct{}{}: defer func(){ <-g.slots }()
    default:
        if err := ctx.Err(); err != nil { return err }
        return ErrOverloaded
    }
    if err := ctx.Err(); err != nil { return err }
    return fn(ctx)
}
```

失败路径

- 无界 channel 充当队列
- 所有错误都重试
- 多个服务层叠重试造成放大

练剑任务

实现 non-blocking concurrency gate，记录 active/rejected，并注入容量满与取消竞态。

验收证据

active 不超上限；准入时已取消则不启动；准入后取消由 fn 收口；过载与取消可区分。

收剑自检

- [ ] 并发槽位满时，本章的 Gate 如何处理？
- [ ] 哪些错误不应重试？
- [ ] 为什么多层重试会放大流量？

章内续问 13.1 · 闯祸与群聊

补页 13.A 真正的有界 Worker Pool

闯祸现场　一只放大的 channel 像空仓库摆在屏幕中央，却没有门卫、班表和关门铃。
Stop 与 Submit 同时发生，发送方撞上已关闭的入口，Worker 还有两个没退出。

陈平安

有个带缓冲的队列，不就算工作池了吗？


阮秀
谁提交、谁消费、谁关门，你一个都没写。


李宝瓶
队满和停止时，调用方能分清结果吗？

陈平安

补固定数量的执行者、有界队列、可取消提交和异常隔离。

陈平安

提交侧关入口，停止时明确排空还是中止。


宁姚
状态机要保证无 send-on-closed，也无泄漏。

收束　关门铃响过一声，所有工位都按编号熄了灯。

先有画面，再落到工程

补页 13.A 真正的有界 Worker Pool

先懂一句

闸门管同时过桥的人，队列才管桥头等候的人。

直觉类比

像游乐园小火车：车上只有固定座位，站台也只有有限排队区。座位是 worker，队伍是 queue；只加检票闸门，不会凭空多出候车空间。

术语落地

- Context：沿调用链传下去的取消铃和截止钟

读代码抓手

- 设计练习 13.1：提交状态表、测试矩阵、命令输出或验收证据。
- 本节不提供可运行实现，不用漂亮伪代码冒充实验结果。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 13.1 · 拆去比喻

补页 13.A 真正的有界 Worker Pool

工程要点

- Queue 容量、worker 数和单任务上限共同决定内存与下游并发。
- Submit 接受 context，并返回 accepted、queue_full、stopped 等稳定结果。
- 只有提交方关闭输入；关闭与提交并发必须由状态机或锁协调，避免 send-on-closed。
- Worker 隔离 panic 并记录 job_id；结果通道的关闭权只能归拥有者。
- Stop(ctx) 明确 drain 或 abort，等待全部 worker 在期限内退出。

练手

以 1000/s 生产、100/s 消费，注入慢任务、取消、worker panic 与 shutdown。

验收

内存有界；队列深度不越界；无 send-on-closed；panic 不杀进程；所有 worker 按策略在期限内退出。

记忆钉

1. Queue 容量、worker 数和单任务上限共同决定内存与下游并发。
2. Submit 接受 context，并返回 accepted、queue_full、stopped 等稳定结果。
3. 只有提交方关闭输入；关闭与提交并发必须由状态机或锁协调，避免 send-on-closed。
4. Stop(ctx) 明确 drain 或 abort，等待全部 worker 在期限内退出。

章内续问 13.2 · 闯祸与群聊

补页 13.B 限流、背压、重试与熔断不是一回事

闯祸现场　故障注入器让下游连续超时，入口和 Worker 的重试计数器同时开始飞快转动。一笔 bill-0827 被层层重试放大成一群并发请求，半开的熔断器也被瞬间挤满。

陈平安：每层多试几次，成功率不是更高吗？



阿良：每个人都来救火，走廊先被水桶堵住了。



李宝瓶：校验失败和 Unknown 副作用也能自动重试吗？

陈平安：外层统一重试额度，只重试幂等的暂时故障。

陈平安：队满稳定拒绝；半开有界探测，退避带 full jitter。



宁姚：限流、背压、重试、熔断各守自己的失败。

收束　疯狂转动的计数器停下，只剩一条可解释的 Retry-After。

先有画面，再落到工程

补页 13.B 限流、背压、重试与熔断不是一回事

先懂一句

限流管来客，背压管排队，重试管再来，熔断管暂不再来。

直觉类比

四块牌子别混用：限流是‘每分钟只进十人’，背压是‘队满请稍等’，重试是‘这次临时失败可再来’，熔断是‘机器坏了，先暂停营业’。

术语落地

- 幂等：同一件事喊两遍，也只真正办一次

读代码抓手

- 设计练习 13.2：提交状态表、测试矩阵、命令输出或验收证据。
- 本节不提供可运行实现，不用漂亮伪代码冒充实验结果。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 13.2 · 拆去比喻

补页 13.B 限流、背压、重试与熔断不是一回事

工程要点

- Token Bucket 管速率与突发；并发 Gate 管同时执行；有界 Queue 管等待容量。
- 过载返回 429 或 503，并提供可解释的 Retry-After；取消不伪装成失败。
- 重试只用于幂等/安全操作和暂态错误，外层统一预算，避免跨层放大。
- 指数退避配 full jitter，并受次数、总时长和额外流量比例约束。
- Circuit Breaker 有 closed/open/half-open 状态，但不能替代请求 timeout。

练手

注入下游 50% timeout、持续故障和恢复抖动，比较无预算重试与有预算策略。

验收

额外重试量不越预算；下游持续故障时 breaker 打开；half-open 探测有界；恢复后自动关闭且无流量尖峰。

记忆钉

1. Token Bucket 管速率与突发；并发 Gate 管同时执行；有界 Queue 管等待容量。
2. 过载返回 429 或 503，并提供可解释的 Retry-After；取消不伪装成失败。
3. 重试只用于幂等/安全操作和暂态错误，外层统一预算，避免跨层放大。
4. Circuit Breaker 有 closed/open/half-open 状态，但不能替代请求 timeout。

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 并发槽位满时，本章的 Gate 如何处理？

陈平安



并发槽位满时立即返回稳定的过载错误，不等待、不暗中排队；



宁姚

对，队列策略留到真正的 Worker Pool 再讲。

第 2 问 哪些错误不应重试？

陈平安



校验、权限、永久业务拒绝和结果 unknown 的非幂等写都不能随手重试；



宁姚

准确，只给已分类、可恢复且满足幂等语义的错误预算。

第 3 问 为什么多层重试会放大流量？

陈平安



入口和下游若各重试 n 次，总调用会相乘而不是相加；



宁姚

答对，重试预算必须由外层统一，避免山门口越救越堵。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 14 场 · 闯祸与群聊

第十四章 流光传书：SSE、断线与跨层取消

目标：理解流式响应、连接生命周期、deadline 和浏览器断线后的资源回收。

闯祸现场　浏览器开着 bill-0827 的 SSE 状态页，窗外雷声一响，网络图标突然变灰。
陈平安看见请求 Context 取消，差点顺手把后台 Bill 处理也标成 cancelled。

陈平安

订阅断了，后台任务也该停吧？


李宝瓶
听众离席，账本为什么要跟着合上？


阮秀
真要取消，谁来持久化 bill_id 的状态变化？

陈平安
断线只关闭订阅和连接资源。


陈平安
业务取消走显式 bill_id API，再沿调用链收口。



宁姚
边界不能混；重连游标也只能属于同一 Bill。

收束　网络图标恢复时，账单仍在原来的状态线上前进。

先有画面，再落到工程

第十四章 流光传书：SSE、断线与跨层取消

先懂一句

SSE 断线只停订阅；业务取消要显式发出，再沿调用链收口。

直觉类比

SSE 像学校广播。某个同学摘下耳机，只代表他不听了，不代表广播站停播；真要取消后台任务，要拿着 bill_id 去按明确的停止按钮。

术语落地

- struct：把一件东西要填的格子做成固定模具
- Context：沿调用链传下去的取消铃和截止钟
- SSE：服务端向浏览器单向喊话的小喇叭

读代码抓手

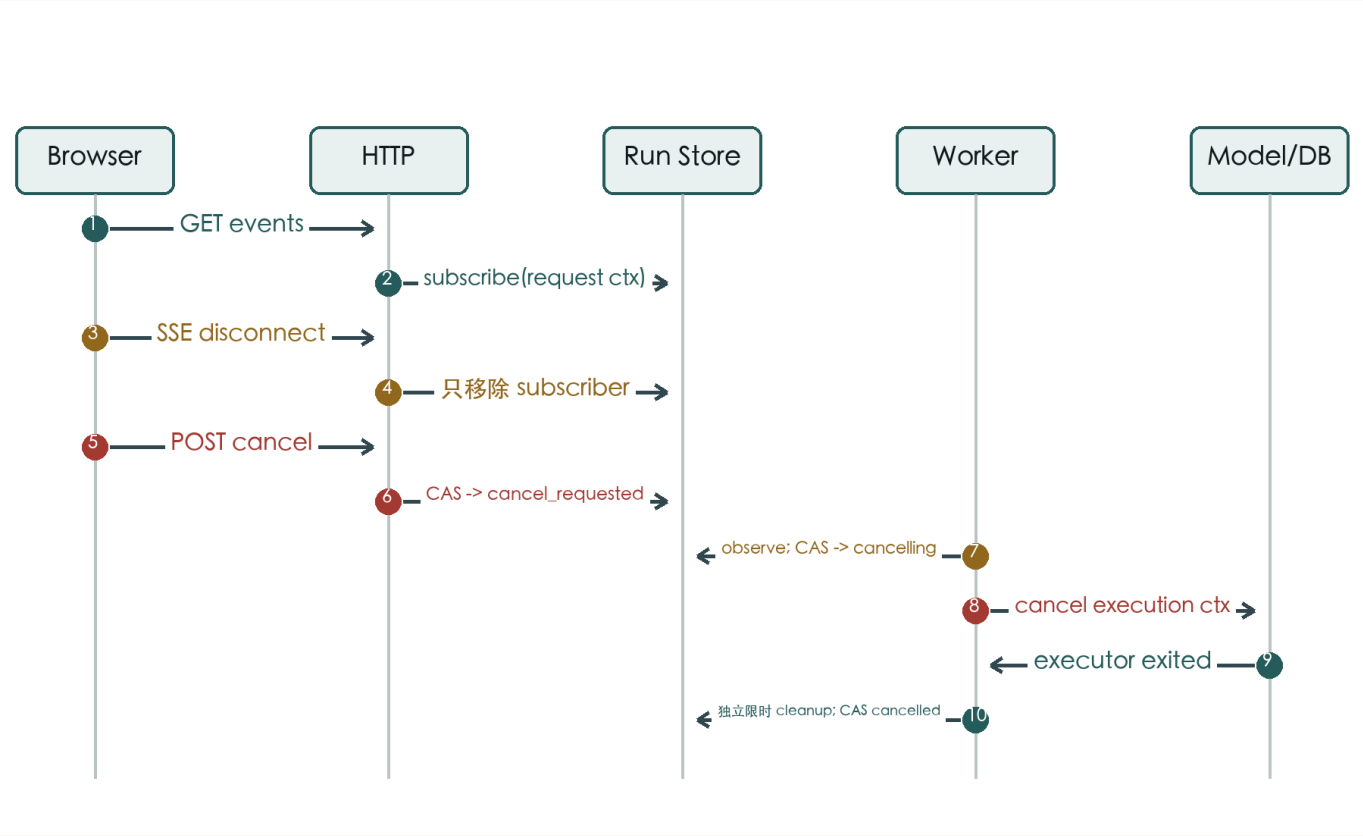
- 先看 Stream 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Stream 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 14.1

回顾 · Context 取消信号跨层传递时序

小白先懂：SSE 断线只停订阅；业务取消要显式发出，再沿调用链收口。



拆去比喻

- 本章只讨论 Bill 事件；路由、授权、事件存储、游标和取消契约都以 bill_id 为键。
- 连接前执行对象级授权；连接后订阅授权版本变化，降权、移除或对象转移时立即重验并断开。
- 断线只结束订阅，不取消 Bill 后台推进；改变业务状态必须调用显式 bill_id API。
- SSE 需要稳定 event ID、Last-Event-ID、心跳、Flush、代理禁缓冲和慢客户端上限。

代码完整展开，不牺牲阅读字号

小白先懂：SSE 断线只停订阅；业务取消要显式发出，再沿调用链收口。

```go
func (h *BillEventHandler) Stream(c *gin.Context) {
    p, ok := PrincipalFrom(c)
    if !ok { h.writeError(c, ErrUnauthorized); return }
    ctx, billID := c.Request.Context(), c.Param("bill_id")

    // WatchBill atomically returns the current authz version and
    // reports every later change; then revalidate before subscribe.
    access, err := h.authz.WatchBill(ctx, p, billID)
    if err != nil { h.writeError(c, err); return }
    defer access.Close()
    allowed, err := access.Revalidate(ctx)
    if err != nil || !allowed { h.writeError(c, ErrNotFound); return }

    events, err := h.events.SubscribeBill(
        ctx, p.TenantID, billID, c.GetHeader("Last-Event-ID"))
    if err != nil { h.writeError(c, err); return }
    defer events.Close()
    for {
        select {
        case <-ctx.Done(): return
        case <-access.Changed():
            allowed, err := access.Revalidate(ctx)
            if err != nil || !allowed { return }
        case event, open := <-events.C():
            if !open || h.writeBillSSE(c.Writer, event) != nil { return }
        }
    }
}
```

下一页核对失败路径、练剑任务、验收证据与自检问题。

失败要能复现，完成要有证据

失败路径

- 把 context 放进 struct 长期保存
- 把业务参数塞进 context
- 忽略 ctx.Err

练剑任务

测试 bill-0827 的授权、重连、游标过期、慢读和连接中途降权，并把另一账单的游标提交给它。

验收证据

全章只用 Bill/bill_id；跨账单游标被拒；权限撤销后连接及时断开；网络断线不改变 Bill 状态。

收剑自检

- [ ] SSE 断线后模型调用如何停止？
- [ ] 数据库驱动一定支持取消吗？
- [ ] 取消状态在哪里持久化？

章内续问 14.1 · 闯祸与群聊

补页 14.A SSE、WebSocket 与慢客户端

闯祸现场　SSE 管道持续冒出 BillEvent，一位慢客户端的 outbound queue 像水桶一样越装越满。
陈平安舍不得丢一条消息，服务端内存便替这个连接无限囤积，断线重连还收到了重复事件。

陈平安

消息都留着，客户端迟早能读完吧？


李宝瓶
水能一直流，桶不能无限长高。


阮秀
队满时合并、丢弃还是断开，规则写在哪里？

陈平安
每连接设有界队列，按 Bill 事件策略处理队满。


陈平安
带事件 ID、心跳和只属于该 bill_id 的续传游标。



宁姚
断线只收订阅；显式 cancel 才改变 Bill 状态。

收束　水桶停在刻度线下，慢连接被礼貌请出了队伍。

先有画面，再落到工程

补页 14.A SSE、WebSocket 与慢客户端

先懂一句

水能一直流，桶却不能无限大。

直觉类比

像学校广播配耳机：广播可以一直说，某个同学却可能听得很慢。耳机缓存不能无限长，满了就合并消息或请他重新连接，广播站不用跟着停课。

术语落地

- SSE：服务端向浏览器单向喊话的小喇叭

读代码抓手

- 先看 Stream 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Stream 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 14.1 · 拆去比喻

补页 14.A SSE、WebSocket 与慢客户端

工程要点

- 入口先从认证中间件取得 Principal；未认证请求在 WatchBill 和 SubscribeBill 之前拒绝。
- 本章事件聚合只有 Bill；路由、授权、事件存储与游标全部以 bill_id 为键。
- SSE 设置 no-cache、X-Accel-Buffering:no、心跳、event ID 与游标过期契约。
- 每连接 outbound queue 有界；慢客户端按 Bill 事件策略 drop/coalesce/disconnect，不无限占内存。
- 订阅授权版本变化；连接中途降权、成员移除或对象转移时重验并断开，不能只在握手时授权。

练手

测试未认证、合法 Principal、对象降权、跨 bill 游标、断线重连、游标过期和慢读。

验收

未认证请求零订阅；bill_id 不能加载别的对象，跨账单游标被拒；断线只收口订阅，显式取消才改变 Bill 状态。

记忆钉

1. 入口先从认证中间件取得 Principal；未认证请求在 WatchBill 和 SubscribeBill 之前拒绝。
2. 本章事件聚合只有 Bill；路由、授权、事件存储与游标全部以 bill_id 为键。
3. SSE 设置 no-cache、X-Accel-Buffering:no、心跳、event ID 与游标过期契约。
4. 断线只结束 Bill 事件订阅；显式取消账单处理使用 bill_id API，并持久化 Bill 状态。

长代码已拆到后续专页

章内续问 14.1 · 代码 1/1

补页 14.A SSE、WebSocket 与慢客户端

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
01  func (h *BillEventHandler) Stream(c *gin.Context) {
02      p, ok := PrincipalFrom(c)
03      if !ok { h.writeError(c, ErrUnauthorized); return }
04      ctx := c.Request.Context()
05      billID := c.Param("bill_id")
06      // Atomic snapshot+watch closes the check/register race.
07      access, err := h.authz.WatchBill(ctx, p, billID)
08      if err != nil { h.writeError(c, err); return }
09      defer access.Close()
10      allowed, err := access.Revalidate(ctx)
11      if err != nil || !allowed {
12          h.writeError(c, ErrNotFound); return
13      }
14
15      events, err := h.events.SubscribeBill(
16          ctx, p.TenantID, billID, c.GetHeader("Last-Event-ID"))
17      if err != nil { h.writeError(c, err); return }
18      defer events.Close()
19
20      // Select on events, ctx and access changes.
21      // Revocation revalidates current Bill state, then disconnects.
22      h.writeBillSSE(ctx, c.Writer, events, access)
23  }
```

未认证请求零订阅；bill_id 不能加载别的对象，跨账单游标被拒；断线只收口订阅，显式取消才改变 Bill 状态。

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 SSE 断线后模型调用如何停止？

陈平安



本章只有 Bill 事件流；`/bills/{bill_id}/events` 断线只结束订阅，不自动取消 bill-0827 的后台状态推进；



宁姚

对，停止处理必须走显式的 bill_id 取消契约。

第 2 问 数据库驱动一定支持取消吗？

陈平安



数据库驱动不一定响应 context 取消，必须用真实驱动测试 deadline 后连接是否归还；



宁姚

准确，传了 ctx 不等于下游真的听见。

第 3 问 取消状态在哪里持久化？

陈平安



取消请求和 Bill 处理状态按 bill_id 持久化，不能只放在 SSE 连接内存里；



宁姚

闭合，重连游标也只能续同一个 bill_id 的事件序列。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 15 场 · 闯祸与群聊

第十五章 夜行留灯：Log、Metric 与 Trace

目标：建立 request_id、run_id、trace_id 的区分和日志、指标、trace 三件套。

闯祸现场　清晨的大屏上，bill-0827 已经 completed，却比平时慢了几秒；日志、指标和 Trace 分占三块屏幕。陈平安只搜 access log，HTTP、数据库、Outbox 和 Worker 各自都说自己没问题，事故像凭空消失。

陈平安



有日志还找不到慢点，是日志不够多吗？



李宝瓶

你缺的是跨同步与异步步骤的因果线。



阮秀

bill_id 能直接当指标标签吗？

陈平安



我把调用链信息传下去，异步步骤按真实因果关联。

陈平安



高基数 ID 留在日志、Trace 或可查询账本。



宁姚

日志讲事件，指标量影响，Trace 解释路径。

收束　三块屏幕终于指向了同一段 Worker 等待。

先有画面，再落到工程

第十五章 夜行留灯：Log、Metric 与 Trace

先懂一句

日志记事，指标看势，Trace 串因果。

直觉类比

系统生病时，Log 是病历，Metric 是体温计，Trace 是造影路线。只看一张病历就宣布找到病因，和看见一个喷嚏就开刀差不多。

术语落地

- error：事情没办成时一起带回来的原因条
- goroutine：很轻的小跑堂，可以同时派出很多个
- Context：沿调用链传下去的取消铃和截止钟
- HTTP：客户端和服务端互寄有格式的信
- Metric：系统现在胖瘦快慢的仪表盘数字
- Trace：一次请求一路经过哪些地方的脚印串
- Run：从开始到结束的一整趟任务

读代码抓手

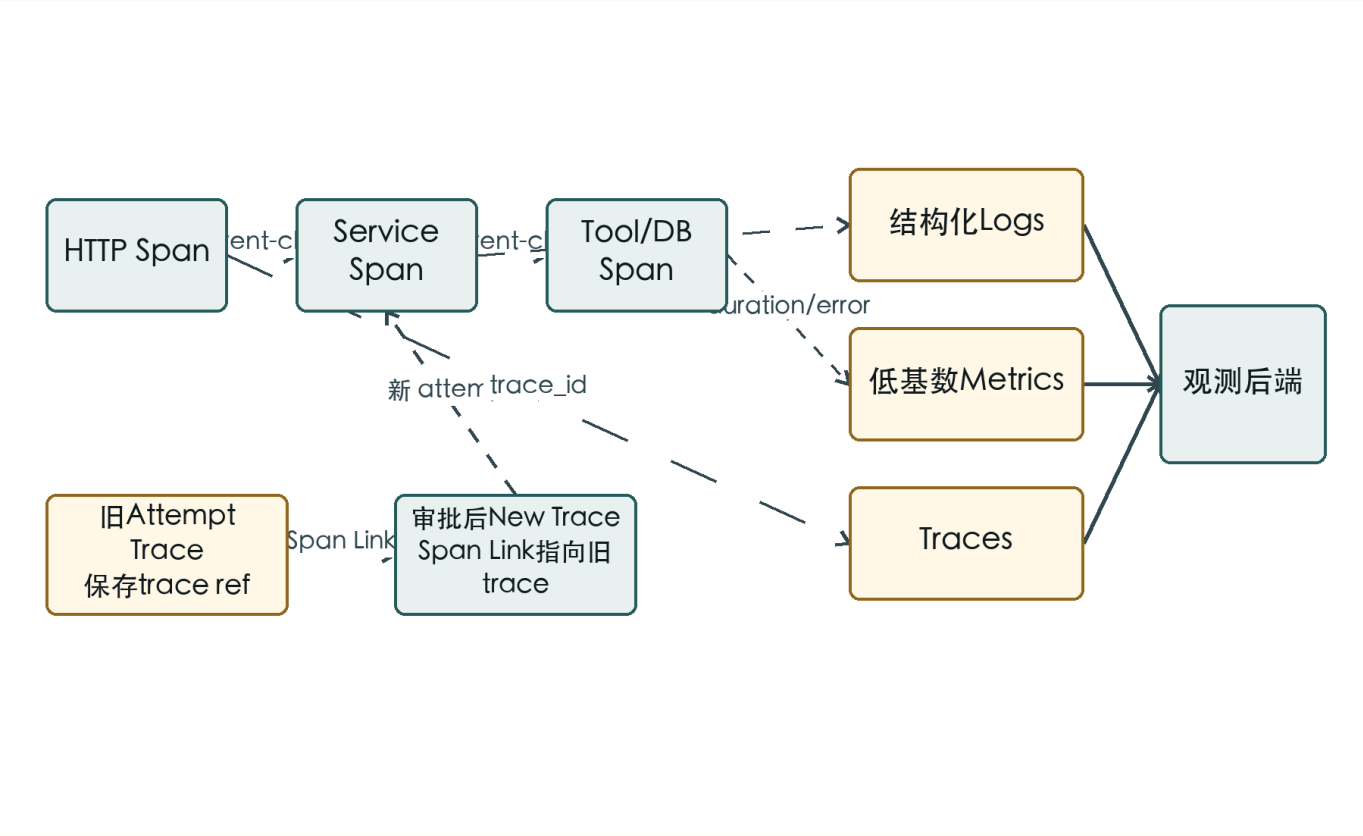
- 先看 Create 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Create 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 15.1

Log、Metric、Trace 与重放证据

小白先懂：日志记事，指标看势，Trace 串因果。



拆去比喻

- 日志记离散事件，指标看趋势，Trace 串同步因果；三者通过 request_id/run_id 关联。
- HTTP 边界提取/注入 trace context，各层创建 span 并记录 error/status。
- 跨审批恢复不是原请求的同步子调用，用 Span Link 连接旧 trace 与新 attempt。
- run_id 可作受控属性，不作为高基数 metric label；敏感 prompt 保存引用或脱敏摘要。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
func (s *Service) Create(ctx context.Context, cmd Command) (Bill, error) {
    ctx, span := tracer.Start(ctx, "bill.create")
    defer span.End()
    span.SetAttributes(attribute.String("run.id", cmd.RunID))
    bill, err := s.repo.Save(ctx, cmd.Bill)
    if err != nil {
        span.RecordError(err)
        span.SetStatus(codes.Error, "save failed")
    }
    return bill, err
}

// After an approval pause, start a new trace.
// Add a Span Link to the saved run trace.
```

失败路径

- 只看 goroutine 数上涨就断言泄漏
- 高基数字段直接做 metric label
- 每层重复打印同一错误

练剑任务

为请求、数据库和工具调用建立 span；模拟审批暂停后恢复，并通过 Span Link 关联。

验收证据

Trace 展示同步父子关系与异步 link；错误状态正确；指标 label 不含 run_id/用户输入。

收剑自检

- [ ] 三个 ID 各解决什么问题？
- [ ] 高基数字段能做 metric label 吗？
- [ ] 敏感 prompt 应如何记录？

章内续问 15.1 · 闯祸与群聊

先量，再看 CPU，再看等待

闯祸现场　Trace 把慢点圈在 Worker 函数上，旁边摆着 benchmark、pprof 和 execution trace 三份工具报告。陈平安凭代码长相重写了最复杂的一段，复测后延迟没变，分配次数反而升了。

陈平安

最复杂的函数，不就是热点吗？


李宝瓶
长得累，不等于机器真的在这里累。


阮秀
CPU、分配和调度等待能用同一份证据回答吗？

陈平安
benchmark 量局部成本，pprof 找资源热点。


陈平安
execution trace 看调度时间线，同环境做前后对比。



宁姚
单次 ns/op 不是生产结论，证据要可重复。

收束　三份报告各回到自己的夹子里，不再互相冒名。

先有画面，再落到工程

先量，再看 CPU，再看等待

先懂一句

benchmark、pprof 和 trace 回答三类不同的性能问题。

直觉类比

给小车调速要三样东西：秒表告诉你快没快，发动机热成像告诉你 CPU 花在哪，路线记录告诉你时间耗在等待哪一段。

术语落地

- goroutine：很轻的小跑堂，可以同时派出很多个
- Trace：一次请求一路经过哪些地方的脚印串
- Tool：真正会读文件、查库或调用服务的手脚

读代码抓手

- 先看 BenchmarkDecode 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 BenchmarkDecode 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 15.1 · 拆去比喻

先量，再看 CPU，再看等待

工程要点

- Benchmark 使用固定输入和 b.N，避免把准备工作算进被测循环。
- 用 b.ReportAllocs 观察分配；比较前固定机器、版本和负载条件。
- pprof 用采样定位 CPU、堆和阻塞热点，结论要回到源码验证。
- go tool trace 展示 goroutine、网络阻塞和调度事件的时间关系。
- 优化后重跑同一基准，并保留前后数据与统计波动。

练手

为一个热点函数建立基准，采集 CPU profile 和 trace，并分别写一句结论。

验收

基准可重复且含分配数据；pprof 与 trace 结论不混用；优化有前后对比。

```
01  func BenchmarkDecode(b *testing.B) {
02      data := []byte("{\"id\":1}")
03      b.ReportAllocs()
04      b.ResetTimer()
05      for i := 0; i < b.N; i++ {
06          var v Bill
07          if err := json.Unmarshal(data, &v); err != nil {
08              b.Fatal(err)
09          }
10      }
11  }
```

章内续问 15.2 · 闯祸与群聊

补页 15.A 从三类信号走到 SLO

闯祸现场　值班台只为一条慢账单敲响警报，HTTP 202 曲线依旧漂亮，异步终态面板却空着一块。陈平安准备凭单次成功继续 Canary，Worker 停摆和 Unknown 积压都被快速响应掩住。

陈平安：HTTP 很快，Canary 应该还能继续。



李宝瓶：收件快，不等于账单按时办完。



阮秀：零样本、旧快照和配置错版也算通过吗？

陈平安：固定 eligible 分母、窗口、终态期限和配置版本。

陈平安：同时看成功终态、完成耗时和各类积压的最老等待时间。



宁姚：证据不足或任一异步 SLO 越界，都失败关闭。

收束　警报不再追着单条日志跑，而是守在用户结果旁。

先有画面，再落到工程

补页 15.A 从三类信号走到 SLO

先懂一句

日志讲发生了什么，指标说影响多大，Trace 解释为何走到这里。

直觉类比

像看校医记录：日志写‘小明摔了’，指标说‘今天摔了多少人’，Trace 画出他从楼梯到医务室的路线，SLO 决定严重到什么程度必须停课检查。

术语落地

- SLO：答应用户大多数时候要达到的服务线
- Run：从开始到结束的一整趟任务
- Unknown：事情可能做了，也可能没做，账还没对清
- outbox：和业务账一起落库、之后再送出的发件箱

读代码抓手

- 先看 AllowCanary 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 AllowCanary 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 15.2 · 拆去比喻

补页 15.A 从三类信号走到 SLO

工程要点

- eligible Run 的分母、排除项、统计窗口与版本必须固定；主动取消和演练流量只能按书面规则排除。
- terminal success 是 eligible Run 在 TerminalDeadline 内进入成功终态的比例；超时仍未终结计失败。
- time-to-terminal 从接受异步请求计到成功或失败终态，至少跟踪 p50/p95/p99。
- queue、outbox、unknown、reconciling 同时监控 oldest age 与 backlog，避免低流量下只看深度漏报。
- Canary 对零样本、样本不足、快照过期和配置版本不一致一律失败关闭，再检查异步 SLO 与多窗口 burn rate。

练手

依次注入好 p95/坏 p99、短窗和长窗 burn-rate 超限、Worker 停摆、outbox 不发布、unknown/reconciling 堆积，以及窗口或配置缺失。

验收

零样本、过期/错版快照、缺失 burn-rate 窗口或非法阈值返回 ErrInsufficientEvidence；p99、任一 burn rate 或异步门槛越界返回 ErrAsyncSLO。

记忆钉

1. eligible Run 的分母、排除项、统计窗口与版本必须固定；主动取消和演练流量只能按书面规则排除。
2. terminal success 是 eligible Run 在 TerminalDeadline 内进入成功终态的比例；超时仍未终结计失败。
3. time-to-terminal 从接受异步请求计到成功或失败终态，至少跟踪 p50/p95/p99。
4. Canary 对零样本、样本不足、快照过期和配置版本不一致一律失败关闭，再检查异步 SLO 与多窗口 burn rate。

长代码已拆到后续专页

章内续问 15.2 · 代码 1/3

补页 15.A 从三类信号走到 SLO

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
type AsyncSLOGate struct {
    ConfigVersion string
    Window, TerminalDeadline, MaxSnapshotAge time.Duration
    BurnShortWindow, BurnLongWindow time.Duration
    MinEligible uint64
    MinTerminalSuccess float64
    MaxTimeToTerminalP95, MaxTimeToTerminalP99 time.Duration
    MaxBurnRateShort, MaxBurnRateLong float64
    MaxQueueAge, MaxOutboxAge time.Duration
    MaxUnknownAge, MaxReconcileAge time.Duration
    MaxBacklogs [4]uint64
}

type AsyncSnapshot struct {
    ConfigVersion string
    CapturedAt time.Time
    Window, TerminalDeadline time.Duration
    BurnShortWindow, BurnLongWindow time.Duration
    Eligible, SuccessWithinDeadline uint64
    TimeToTerminalP95, TimeToTerminalP99 time.Duration
    BurnRateShort, BurnRateLong float64
    OldestQueueAge, OldestOutboxAge time.Duration
    OldestUnknownAge, OldestReconcileAge time.Duration
    Backlogs [4]uint64
```

下一页继续同一段代码，不切换事故对象。

章内续问 15.2 · 代码 2/3

补页 15.A 从三类信号走到 SLO

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
}

func (g AsyncSLOGate) AllowCanary(
    now time.Time, s AsyncSnapshot,
) error {
    if g.MinEligible == 0 || g.MaxSnapshotAge <= 0 ||
        g.MaxTimeToTerminalP95 <= 0 ||
        g.MaxTimeToTerminalP99 < g.MaxTimeToTerminalP95 ||
        g.BurnShortWindow <= 0 ||
        g.BurnLongWindow <= g.BurnShortWindow ||
        g.MaxBurnRateShort <= 0 || g.MaxBurnRateLong <= 0 ||
        s.Eligible == 0 || s.SuccessWithinDeadline > s.Eligible ||
        s.ConfigVersion != g.ConfigVersion || s.Window != g.Window ||
        s.TerminalDeadline != g.TerminalDeadline ||
        s.BurnShortWindow != g.BurnShortWindow ||
        s.BurnLongWindow != g.BurnLongWindow ||
        s.CapturedAt.After(now) ||
        now.Sub(s.CapturedAt) > g.MaxSnapshotAge ||
        s.Eligible < g.MinEligible {
        return ErrInsufficientEvidence
    }
    for i := range s.Backlogs {
        if s.Backlogs[i] > g.MaxBacklogs[i] { return ErrAsyncSLO }
    }
```

下一页继续同一段代码，不切换事故对象。

章内续问 15.2 · 代码 3/3

补页 15.A 从三类信号走到 SLO

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
49      successRate := float64(s.SuccessWithinDeadline) / float64(s.Eligible)
50      if successRate < g.MinTerminalSuccess ||
51          s.TimeToTerminalP95 > g.MaxTimeToTerminalP95 ||
52          s.TimeToTerminalP99 > g.MaxTimeToTerminalP99 ||
53          s.BurnRateShort > g.MaxBurnRateShort ||
54          s.BurnRateLong > g.MaxBurnRateLong ||
55          s.OldestQueueAge > g.MaxQueueAge ||
56          s.OldestOutboxAge > g.MaxOutboxAge ||
57          s.OldestUnknownAge > g.MaxUnknownAge ||
58          s.OldestReconcileAge > g.MaxReconcileAge {
59          return ErrAsyncSLO
60      }
61      return nil
62  }
```

零样本、过期/错版快照、缺失 burn-rate 窗口或非法阈值返回 ErrInsufficientEvidence；p99、任一 burn rate 或异步门槛越界返回 ErrAsyncSLO。

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 三个 ID 各解决什么问题？

陈平安

request_id 标识一次 HTTP 来回，Run ID 标识可跨请求恢复的异步执行，trace_id 串起一次被采样的因果路径；



宁姚

对，bill_id 仍是领域对象 ID，四者不能互换。

第 2 问 高基数字段能做 metric label 吗？

陈平安

user_id、bill_id、run_id 这类高基数字段不能做 metric label，应放日志、trace 或可查询账本；



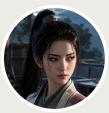
宁姚

正确，指标只保留受控维度。

第 3 问 敏感 prompt 应如何记录？

陈平安

敏感输入不记原文，只留脱敏摘要、引用和访问审计，token、cookie 与 PII 统一遮蔽；



宁姚

答对，排障证据不能变成第二处泄露。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 16 场 · 闯祸与群聊

第十六章 下山之前：配置、Docker、迁移与回滚

目标：掌握配置与密钥、探针、多阶段镜像、CI、数据库迁移、灰度和回滚证据。

闯祸现场　发布室里摆着旧镜像、迁移清单和一只写着 RTO/RPO 的计时器，Canary 曲线刚越过红线。陈平安准备一键切回旧二进制，却发现新版本已经写入新字段，旧代码未必还能读。

陈平安



镜像退回去，不就是完整回滚吗？



宁姚

配置、密钥、数据契约和副作用会一起倒退吗？



阮秀

旧新版本是否都能读写，恢复演练做过没有？

陈平安

迁移先 expand，回填校验后再 contract。



陈平安

Canary 越界时按迁移状态选回滚或前滚。





宁姚

发布门禁要绑定版本、SLO、不变量和恢复证据。

收束　计时器重新归零，这次旁边多了一份真实恢复记录。

先有画面，再落到工程

第十六章 下山之前：配置、Docker、迁移与回滚

先懂一句

下山六件：配、密、迁、探、镜、退。

直觉类比

上线像全班搬家：钥匙别写在纸箱外面，家具要编号，旧门要能关，新门要能探活，搬坏了还得知道哪些能搬回、哪些数据只能向前修。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

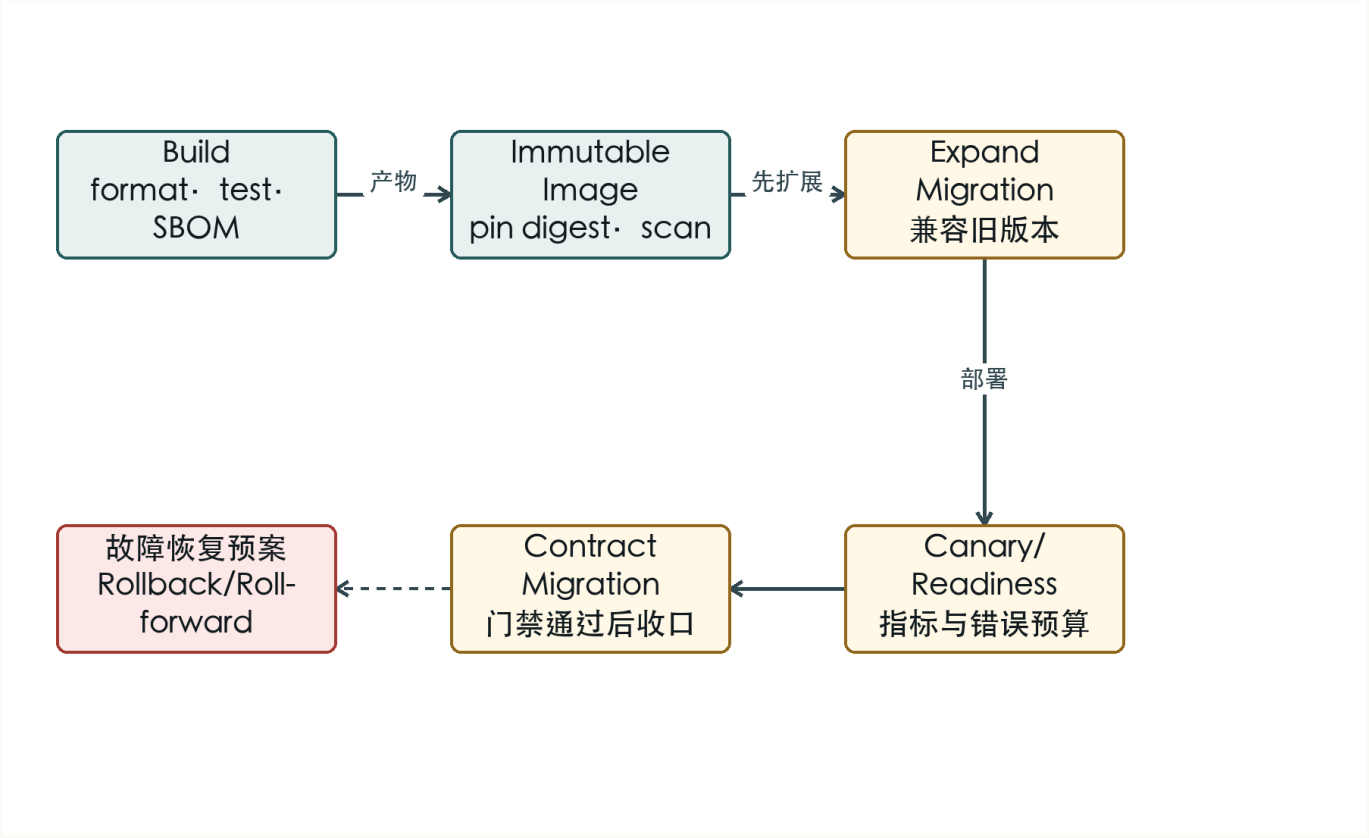
- 先看这段代码在读写什么状态，再找它对外承诺的结果。
- 最后核对状态出口：成功改变什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 16.1

部署、兼容迁移与回滚流程

小白先懂：下山六件：配、密、迁、探、镜、退。



拆去比喻

- 配置与密钥分离；镜像使用多阶段构建并 pin digest，记录 toolchain、依赖、SBOM 与构建参数。
- readiness 表示能否接流量，liveness 只判断是否需要重启，二者不能互相替代。
- 迁移先 expand 后 contract；破坏性 contract 前有备份、校验、稳定窗口和恢复演练。
- 回滚策略可能是镜像回滚、可逆迁移或 roll-forward；不能承诺镜像能恢复已删除数据。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```
# CI passes verified image references in name@sha256 form.
ARG GO_IMAGE
ARG RUN_IMAGE

FROM ${GO_IMAGE} AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -trimpath -o /out/server ./cmd/server

FROM ${RUN_IMAGE}
COPY --from=build /out/server /server
USER nonroot:nonroot
ENTRYPOINT ["/server"]
```

失败路径

- latest 标签不可追溯
- 把 .env 打进镜像
- readiness 通过却依赖尚未就绪

练剑任务

构建 pin digest 的镜像，设计 expand/contract 迁移并演练镜像回滚与数据库恢复。

验收证据

同一源码可追溯 toolchain/SBOM；探针语义正确；破坏性迁移有恢复证据或明确 roll-forward。

收剑自检

- [ ] readiness 与 liveness 有何区别？
- [ ] 为何先 expand 再 contract？
- [ ] 镜像回滚能恢复破坏性迁移吗？

章内续问 16.1 · 闯祸与群聊

把日常门禁固定下来

闯祸现场　提交按钮旁排着 gofmt、vet、test、race、coverage 和 govulncheck 六盏灯。陈平安只看 go test 亮绿就准备合并，未执行的并发路径和格式差异仍藏在暗处。

陈平安



单测全绿，其他灯可以下次再看。



宁姚

每盏灯覆盖的失败边界都不同。



李宝瓶

race 没报错，能证明没执行到的路径安全吗？

陈平安

把检查写进可复现流水线，快慢门禁分阶段。



陈平安

关键并发包跑 race，失败全部阻断合并。





宁姚

覆盖率和 vet 都不能替代审查与业务断言。

收束　六盏灯依次亮起，提交按钮才解除灰色。

先有画面，再落到工程

把日常门禁固定下来

先懂一句

工具链的价值在于每次都执行，并且失败能定位到具体责任。

直觉类比

像每天出门摸四个口袋：钥匙、手机、卡、药。清单的价值不是写得漂亮，而是每次都摸；某个口袋空了，还要立刻知道少的是哪一件。

术语落地

- race：两个人同时改一本账，谁先落笔说不清

读代码抓手

- 设计练习 16.1：提交状态表、测试矩阵、命令输出或验收证据。
- 本节不提供可运行实现，不用漂亮伪代码冒充实验结果。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 16.1 · 拆去比喻

把日常门禁固定下来

工程要点

- gofmt -w 统一格式；CI 用格式差异检查阻止未格式化代码。
- go vet ./... 检查常见可疑结构，但不能替代测试和代码审查。
- go test ./... 守功能；go test -race ./... 覆盖关键并发包或定时全跑。
- go test -coverprofile=cover.out ./... 记录覆盖变化，不追逐单一百分比。
- govulncheck ./... 结合实际调用链检查已知漏洞，并审查依赖变更。

练手

把五项命令写入本地脚本或 CI，并为快门禁与慢门禁划分阶段。

验收

提交门禁可一键复现；任何失败会阻断合并并留下命令与日志。

记忆钉

1. gofmt -w 统一格式；CI 用格式差异检查阻止未格式化代码。
2. go vet ./... 检查常见可疑结构，但不能替代测试和代码审查。
3. go test ./... 守功能；go test -race ./... 覆盖关键并发包或定时全跑。
4. govulncheck ./... 结合实际调用链检查已知漏洞，并审查依赖变更。

章内续问 16.2 · 闯祸与群聊

启动和退出是一份契约

闯祸现场　滚动发布倒计时开始，旧进程里还有 req-0827、后台 Worker 和一条 SSE 长连接。陈平安收到 SIGTERM 就直接退出，事务中断，长连接也没收到任何收口信号。

陈平安



新副本起来了，旧请求会自己搬过去吧？



阮秀

正在执行的工作不会跨进程瞬移。



李宝瓶

普通关服会替你等待后台任务和劫持连接吗？

陈平安

main 登记资源，先摘 readiness，再关普通请求。



陈平安

显式通知长连接与 Worker，并逆序有界清理。





宁姚

每阶段收集错误，不能首错就跳过后续关闭。

收束　倒计时归零时，最后一个工位也留下了退出记录。

先有画面，再落到工程

启动和退出是一份契约

先懂一句

组合根创建依赖，信号触发取消，关闭流程等待在途工作收口。

直觉类比

像放学关校门：先挂‘停止入校’，再等教室里的孩子出来，通知值日生收尾，最后按反顺序关灯锁门。

术语落地

- HTTP：客户端和服务端互寄有格式的信
- Handler：门房，负责收信和回信

读代码抓手

- 先看 run 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 run 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 16.2 · 拆去比喻

启动和退出是一份契约

工程要点

- main 作为 composition root 登记资源，并按启动的逆序执行关闭。
- Shutdown 只等待普通 HTTP 请求；长连接与后台 worker 需要显式通知和等待。
- 监听异常、Shutdown 超时或正常取消都必须进入同一套后续有界清理阶段。
- 每阶段使用明确时限，收集所有失败并用 errors.Join 返回，不能首错即提前退出。

练手

注入超时 Handler、Shutdown 错误、worker 与资源关闭错误，记录各阶段仍被调用及其顺序。

验收

监听失败或 Shutdown 失败后仍执行后续清理；总时长有界；最终错误聚合每个失败阶段。

记忆钉

1. main 作为 composition root 登记资源，并按启动的逆序执行关闭。
2. Shutdown 只等待普通 HTTP 请求；长连接与后台 worker 需要显式通知和等待。
3. 监听异常、Shutdown 超时或正常取消都必须进入同一套后续有界清理阶段。
4. 每阶段使用明确时限，收集所有失败并用 errors.Join 返回，不能首错即提前退出。

长代码已拆到后续专页

章内续问 16.2 · 代码 1/2

启动和退出是一份契约

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```go
func run(ctx context.Context, srv *http.Server) error {
    errc := make(chan error, 1)
    go func() { errc <- srv.ListenAndServe() }()
    var errs []error
    serveDone := false
    recordServe := func(err error) {
        if err != nil && !errors.Is(err, http.ErrServerClosed) {
            errs = append(errs, fmt.Errorf("serve HTTP: %w", err))
        }
    }

    select {
    case err := <-errc:
        serveDone = true
        recordServe(err)
    case <-ctx.Done():
    }

    readiness.Store(false)
    drainCtx, cancelDrain := context.WithTimeout(
        context.Background(), 5*time.Second)
    if err := waitForEndpointRemoval(drainCtx); err != nil {
        errs = append(errs, fmt.Errorf("drain readiness: %w", err))
    }
```

下一页继续同一段代码，不切换事故对象。

章内续问 16.2 · 代码 2/2

启动和退出是一份契约

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
25      cancelDrain()
26      notifyLongLivedClients()
27      shutdownCtx, cancelShutdown := context.WithTimeout(
28          context.Background(), 5*time.Second)
29      if err := srv.Shutdown(shutdownCtx); err != nil {
30          errs = append(errs, fmt.Errorf("shutdown HTTP: %w", err))
31          if closeErr := srv.Close(); closeErr != nil {
32              errs = append(errs,
33                  fmt.Errorf("force close HTTP: %w", closeErr))
34          }
35      }
36      cancelShutdown()
37      if !serveDone { recordServe(<-errc) }
38
39      cleanupCtx, cancelCleanup := context.WithTimeout(
40          context.Background(), 5*time.Second)
41      if err := waitAndClose(cleanupCtx); err != nil {
42          errs = append(errs, fmt.Errorf("close resources: %w", err))
43      }
44      cancelCleanup()
45      return errors.Join(errs...)
46  }
```

监听失败或 Shutdown 失败后仍执行后续清理；总时长有界；最终错误聚合每个失败阶段。

章内续问 16.3 · 闯祸与群聊

补页 16.A 配置、密钥与容器硬边界

闯祸现场　镜像检查台上摊着 config.yaml，一串数据库密码正躺在可复制的层历史里。陈平安为了少一步部署，把 Secret 和普通配置一起打包，回滚包也成了泄密源。

陈平安

都放配置文件里，部署不是最简单吗？


宁姚
可校验配置和可轮换秘密不是同一种东西。


阮秀
日志、诊断和旧镜像会不会留下明文？

陈平安

配置按类型启动校验，Secret 只保存引用。

陈平安

KMS 或密钥服务负责注入，不进发布物。


宁姚
容器最小权限，资源上限与应用容量一起预算。

收束　密码从镜像层消失，只留下一个受控引用。

先有画面，再落到工程

补页 16.A 配置、密钥与容器硬边界

先懂一句

配置可以进册，密钥只该临时递到手里。

直觉类比

配置像饭盒标签，可以写今天吃什么；密钥像家门钥匙，只在需要时递到手里，不能印在饭盒、镜像或日志上。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 Validate 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Validate 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 16.3 · 拆去比喻

补页 16.A 配置、密钥与容器硬边界

工程要点

- 配置来源优先级固定，启动时 fail-fast，并记录不含敏感值的配置指纹。
- Secret 使用引用而非明文，支持轮换与新旧密钥双窗口，禁止进入仓库和镜像层。
- 日志、panic dump、环境诊断和 metrics 都执行敏感字段脱敏。
- 容器采用 non-root、read-only rootfs、drop capabilities、tmpfs 与 seccomp。
- CPU/内存/PID requests 与 limits 要同 Go GC、连接池、队列容量一起预算。

练手

注入缺失配置、未知字段、日志泄密、只读文件系统、密钥轮换和内存上限。

验收

错误配置启动即失败；镜像/日志无密钥；轮换不中断；容器在只读根目录和资源限制下正常运行。

```go
type Config struct {
    Addr      string
    DBURLRef  string
    MaxOpen   int
}

func (c Config) Validate() error {
    if c.Addr == "" || c.DBURLRef == "" {
        return ErrInvalidConfig
    }
    if c.MaxOpen < 1 { return ErrInvalidConfig }
    return nil
}
```

章内续问 16.4 · 闯祸与群聊

补页 16.B 迁移、灰度（Canary）、回滚与恢复演练

闯祸现场　迁移看板上，旧列已经被删除，旧镜像却还放在回滚按钮旁；恢复计时器没有一次真实记录。Canary 出错后陈平安切回旧版，旧代码刚启动就读不懂 bill-0827 的新数据。

陈平安

镜像还在，为什么回滚按钮没用？


阮秀
你先 contract，旧版本的路已经拆了。


李宝瓶
备份文件存在，谁证明它真能恢复到目标时间？

陈平安
先 expand，回填可暂停可重跑，校验后再 contract。


陈平安
迁移独立加锁执行，并用真恢复量出 RTO/RPO。



宁姚
按迁移状态选 rollback 或 roll-forward。

收束　回滚按钮旁终于放上了可用的恢复路线图。

先有画面，再落到工程

补页 16.B 迁移、灰度（Canary）、回滚与恢复演练

先懂一句

镜像能退，数据未必退；备份存在，也不等于恢复可用。

直觉类比

发布像换一座积木城堡：新城堡不好可以换回旧模型，但已经搬进去的数据不会自动倒带。备份只是仓库里有箱子，恢复演练才证明箱子真能打开。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 设计练习 16.4：提交状态表、测试矩阵、命令输出或验收证据。
- 本节不提供可运行实现，不用漂亮伪代码冒充实验结果。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 16.4 · 拆去比喻

补页 16.B 迁移、灰度（Canary）、回滚与恢复演练

工程要点

- Migration 用独立 job、advisory lock、statement/lock timeout，禁止每副本抢跑。
- Expand/contract 保证旧新版本同时读写；回填可暂停、可重跑并有校验。
- 发布先摘除异常实例，再依据错误预算、业务不变量和迁移状态决定动作。
- 灰度（Canary）/blue-green 门禁要说明哪些变化可回滚、哪些只能前滚。
- 备份验校验和、加密与保留；restore drill 量出 RTO/RPO 并验证应用可读。

练手

模拟滚动升级、回填中断、不可逆 contract、Canary 超预算及从备份恢复。

验收

旧新版本并存无契约错误；迁移可续跑；发布越界能收口；恢复后的数据与应用校验通过 RTO/RPO。

记忆钉

1. Migration 用独立 job、advisory lock、statement/lock timeout，禁止每副本抢跑。
2. Expand/contract 保证旧新版本同时读写；回填可暂停、可重跑并有校验。
3. 发布先摘除异常实例，再依据错误预算、业务不变量和迁移状态决定动作。
4. 备份验校验和、加密与保留；restore drill 量出 RTO/RPO 并验证应用可读。

章内续问 16.5 · 闯祸与群聊

补页 16.C 上线之前，证据要一层层过门

闯祸现场　发布门前堆着单测报告，真实 PostgreSQL、Redis、Broker 和故障注入器却还没接上电。陈平安拿着全绿覆盖率准备放行，跨租户拒绝和 Worker 停摆从未被测试问到。

陈平安：覆盖率很高，还需要这么多门吗？



左右：单测证明一招，不证明整条运行链。

李宝瓶：HTTP 202 很快，异步一直不终结也能发布？

陈平安：补契约、真实依赖、迁移矩阵、并发和安全回归。

陈平安：负载与故障同时检查同步和异步 SLO。



宁姚：零样本或任一门槛越界都阻断，并保留恢复证据。

收束　门前的绿灯不再只照着覆盖率那一页。

先有画面，再落到工程

补页 16.C 上线之前，证据要一层层过门

先懂一句

单测证明一招，发布门禁证明整条链在真实风雨里站得住。

直觉类比

像闯关卡：单测是检查一招，集成测试是两人配合，迁移演练、压测、灰度和回滚证明整支队伍在真实风雨里还能回家。

术语落地

- goroutine：很轻的小跑堂，可以同时派出很多个
- race：两个人同时改一本账，谁先落笔说不清
- HTTP：客户端和服务端互寄有格式的信
- Service：真正执行业务规矩的账房
- Redis：跑得快的临时小抄，不是真账本
- SLO：答应用户大多数时候要达到的服务线
- Unknown：事情可能做了，也可能没做，账还没对清
- outbox：和业务账一起落库、之后再送出的发件箱

读代码抓手

- 设计练习 16.5：提交状态表、测试矩阵、命令输出或验收证据。
- 本节不提供可运行实现，不用漂亮伪代码冒充实验结果。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 16.5 · 拆去比喻

补页 16.C 上线之前，证据要一层层过门

工程要点

- Unit/Service 测业务不变量；HTTP 契约测试覆盖 schema、状态码和稳定错误 code。
- PostgreSQL/Redis/Broker 集成测试使用真实版本，并执行 migration 兼容矩阵。
- race、goroutine leak、fuzz、property 与安全回归覆盖并发和输入边界。
- 负载与故障注入同时观察 HTTP p95/p99、terminal success、time-to-terminal、queue/outbox/unknown/reconcile age。
- 故障注入必须证明同步 SLO、异步 SLO、业务不变量与恢复证据同时闭合；零样本或任一越界都不能放行。

练手

在 HTTP 202 仍快速成功时分别停 Worker、阻塞 outbox、丢失回执并制造 reconcile 积压，验证 Canary 门禁和恢复动作。

验收

同步接口健康不能掩盖异步未终结；所有阈值引用版本化 SLO 配置，越界可判定地阻断发布并选择 rollback 或 roll-forward。

记忆钉

1. Unit/Service 测业务不变量；HTTP 契约测试覆盖 schema、状态码和稳定错误 code。
2. PostgreSQL/Redis/Broker 集成测试使用真实版本，并执行 migration 兼容矩阵。
3. race、goroutine leak、fuzz、property 与安全回归覆盖并发和输入边界。
4. 故障注入必须证明同步 SLO、异步 SLO、业务不变量与恢复证据同时闭合；零样本或任一越界都不能放行。

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 readiness 与 liveness 有何区别？

陈平安



readiness 决定实例是否继续接流量，liveness 判断进程是否需要重启；



宁姚

对，依赖短抖动不该让 liveness 制造重启风暴。

第 2 问 为何先 expand 再 contract？

陈平安



先 expand 保持旧新版本都能读写，再回填校验，最后才 contract；



宁姚

正确，这样 Canary 越界时旧版本仍有路可退。

第 3 问 镜像回滚能恢复破坏性迁移吗？

陈平安



镜像回滚只退代码，删列、坏数据和已执行副作用不会跟着复原；



宁姚

闭合，必须依据迁移状态选择前滚或回滚，并用 restore drill 证明恢复。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

卷 三

成为 Harness AI 工程师

让模型在工具、状态、权限、预算、评测和人工审批中可靠做事。

第 17-24 章



第 17 场 · 闯祸与群聊

第十七章 会说不等于会做：LLM、Workflow 与 Tool Calling

目标：理解模型不确定性、结构化输出、确定性工作流与受限 Agent 的边界。

闯祸现场　傍晚的办公室只剩空调声，陈平安把写着 bill-0827 的奶茶小票压在键盘边，让模型先整理一份创建草稿。草稿刚出现在屏幕上，他就把它当成了已执行结果，伸手去账本里找那笔并不存在的新记录。

陈平安



模型已经写好草稿，账本应该也多了一笔。



阿良

屏幕上有字，数据库可没替你按确认。



李宝瓶

字段缺失时，谁负责停下来追问而不是写库？

陈平安



我让 Workflow 固定校验和停点，只接收候选参数。



宁姚

候选不是执行；确认前不得调用 ledger.create_bill。

收束　奶茶小票仍压在键盘边，账本一页也没有多。

先有画面，再落到工程

第十七章 会说不等于会做：LLM、Workflow 与 Tool Calling

先懂一句

模型只拟剑谱，系统决定能否出剑。

直觉类比

模型像一位反应极快但不承担后果的顾问：建议可以听，事实仍要核，真正开门、花钱、改账必须由外层系统决定。

术语落地

- Workflow：预先写好的步骤编排，按固定顺序推进和停下
- Agent：在明确边界内观察结果并选择下一步的执行者
- Tool Calling：模型提出结构化调用，外层校验后才真正执行
- LLM：很会猜下一句话、但不会替系统担责的模型

读代码抓手

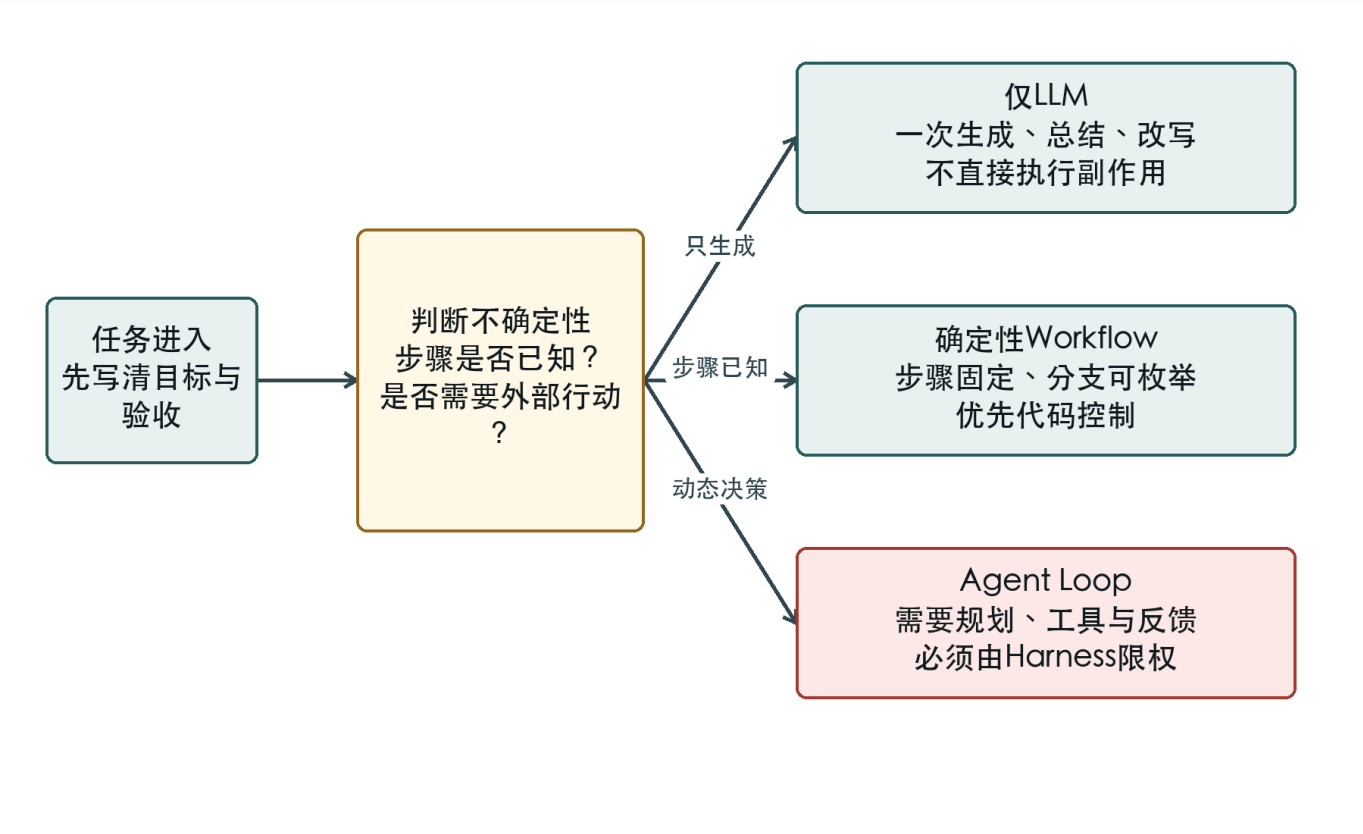
- 先看 PrepareBill 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 PrepareBill 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 17.1

LLM、确定性 Workflow 与 Agent 的选择

小白先懂：模型只拟剑谱，系统决定能否出剑。



拆去比喻

- LLM 生成文本；workflow 按确定步骤编排；Agent 在约束内选择动作。不要为了时髦把确定流程改成开放循环。
- 自然语言解析的输出必须进入 typed draft，不能直接产生不可逆副作用。
- 模型输出是不可信输入：校验 schema、枚举、金额、日期、租户与权限。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
type DraftTool interface {
    Draft(
        context.Context, NaturalLanguageInput,
    ) (BillDraft, error)
}

func PrepareBill(
    ctx context.Context,
    tool DraftTool,
    input NaturalLanguageInput,
) (BillDraft, error) {
    draft, err := tool.Draft(ctx, input)
    if err != nil { return BillDraft{}, err }
    if err := ValidateDraft(draft); err != nil {
        return BillDraft{}, err
    }
    return draft, nil // policy and approval still follow
}
```

失败路径

- 模型自由生成 SQL
- 输出能解析就视为正确
- 把低置信度结果静默写库

练剑任务

设计 parse -> validate -> confirm -> execute 四步接口。

验收证据

未经确认的请求不会产生账单副作用。

收剑自检

- [ ] 固定流程何时不该用 Agent？
- [ ] 模型输出为何属于不可信输入？
- [ ] 草稿模式阻断了什么风险？

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 固定流程何时不该用 Agent？

陈平安



步骤和分支都能预先写清时用 Workflow，不让 Agent 临场发挥。



宁姚

对，不确定判断只放在真正含糊的节点。

第 2 问 模型输出为何属于不可信输入？

陈平安



模型输出只是候选输入，仍要过 schema、业务规则和权限校验。



宁姚

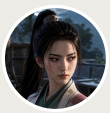
对，流畅表达不等于可信参数。

第 3 问 草稿模式阻断了什么风险？

陈平安



草稿模式阻断 ledger.create_bill 的真实副作用。



宁姚

答对，确认和门禁通过前只生成候选。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 18 场 · 闯祸与群聊

第十八章 大脑、招式、驿道与剑匣

目标：对照只读草稿 Tool 与真正写账 Tool，认清副作用、审批、稳定幂等身份和输入输出契约。

闯祸现场

白板上贴着 Workflow、Skill、MCP、Harness 和 Tool 五张标签，桌面中央只放着 ledger.create_bill 的契约卡。

陈平安把排步骤的 Workflow 当成了动账接口，检查记录里顿时找不到真正产生副作用的对象。

陈平安



流程排好了，Workflow 就是写账的那只手。



阿良

排值日表的人，未必亲自拖地。



李宝瓶

真正改变账本的接口是谁，放行前由谁验契约？

陈平安



真正写账的是 ledger.create_bill；Harness 先做门禁。



宁姚

模型只提议，唯一 Executor 才能执行已授权 Tool。

收束

五张标签重新排好，契约卡单独留在了执行门前。

先有画面，再落到工程

第十八章 大脑、招式、驿道与剑匣

先懂一句

Tool 动手；模型只提议，外层才放行。

直觉类比

大脑负责想，Tool 负责动手，MCP 像统一插头，Harness 像带保险丝的工具箱。插头能插上，不代表这台电钻就该交给任何人。

术语落地

- 幂等：同一件事喊两遍，也只真正办一次
- Agent：在明确边界内观察结果并选择下一步的执行者
- Tool：真正会读文件、查库或调用服务的手脚
- Harness：管住模型手脚、权限、预算和证据的外壳
- CallID：同一步副作用永远不变的取件号

读代码抓手

- 先比 name、sideEffect 与 requiresApproval：谁只做草稿，谁真的写账。
- 再看 input/output schema 摘要：陌生字段、长度和结果形状由哪个版本裁决。
- 幂等身份必须来自 Harness CallID，不能让模型在参数里自报一个 key。
- 最后把 contract digest 与审批快照绑定，热更新不能偷换已批准契约。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 18.1 · 闯祸与群聊

工具发现不等于工具可信

闯祸现场　会议桌上散着 Skill 清单、MCP tools/list 快照和 ledger.create_bill 的 schema，门禁灯还停在黄色。陈平安把 Skill 当成 MCP 标准对象，又按刚更新的 schema 继续旧 approval，契约版本在一次调用里混成了两份。

陈平安



Skill 能从 MCP 找到接口，就等于它已经可信。



阿良

看见菜单，只能证明厨房写过这道菜。



李宝瓶

schema 热更新时，规范化和 policy 还能各读一版吗？

陈平安



我冻结一份契约快照，再做零副作用 Preflight。

陈平安



契约变化就重新 approval，坏输出交唯一 Executor 隔离。



宁姚

Harness 守门，Tool 才动手。

收束　黄色门禁灯转绿时，桌上只剩一份 active contract digest。

先有画面，再落到工程

工具发现不等于工具可信

先懂一句

先逐个认清 Skill、MCP 与 Harness，再让工具契约过门禁。

直觉类比

像把电钻交给小朋友：先确认按钮、转速、能钻什么、失败怎么停，再限制房间和电量。工具能被发现，只等于看见了电钻，不等于该立刻通电。

术语落地

- Tool：真正会读文件、查库或调用服务的手脚
- Step：整趟任务中的一次具体动作
- CallID：同一步副作用永远不变的取件号
- receipt：外部系统办完事后给回来的回执
- approval：高风险动作动手前的人类放行条
- sandbox：把程序关进权限和资源都受限的练功房

读代码抓手

- 先看 Preflight 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Preflight 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 18.1 · 拆去比喻

工具发现不等于工具可信

工程要点

- Preflight 显式接收入口已校验的 Principal，只读取 active snapshot、规范化 command 并校验输入；本页不得调用 sandbox、MCP Tool 或 artifact store。
- 同一个 immutable snapshot 同时供规范化、schema 与 policy 使用；热更新发生时整次失败并要求重新审批，不混用版本。
- 输入未知字段、超限或深度异常在副作用前拒绝；approval 变化不在 preflight 中 Consume。
- 唯一 Executor 才能 reserve/authorize、分配 Step CallID、dispatch、接 receipt 与 settle。
- Executor 遇坏 output 先写 quarantine，再返回 ErrInvalidToolOutput；隔离成功也绝不等于调用成功。

练手

断言 Preflight 对合法/坏输入都产生零 Tool 调用；再让 Executor 收到 schema-invalid output 且 quarantine 写入成功。

验收

合法 preflight 只返回 canonical command；坏输入零副作用；坏 output 无 artifact/上下文/成功 Step，并始终返回非 nil ErrInvalidToolOutput。

记忆钉

1. Preflight 显式接收入口已校验的 Principal，只读取 active snapshot、规范化 command 并校验输入；本页不得调用 sandbox、MCP Tool 或 artifact store。
2. 同一个 immutable snapshot 同时供规范化、schema 与 policy 使用；热更新发生时整次失败并要求重新审批，不混用版本。
3. 输入未知字段、超限或深度异常在副作用前拒绝；approval 变化不在 preflight 中 Consume。
4. Executor 遇坏 output 先写 quarantine，再返回 ErrInvalidToolOutput；隔离成功也绝不等于调用成功。

长代码已拆到后续专页

章内续问 18.1 · 代码 1/1

工具发现不等于工具可信

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

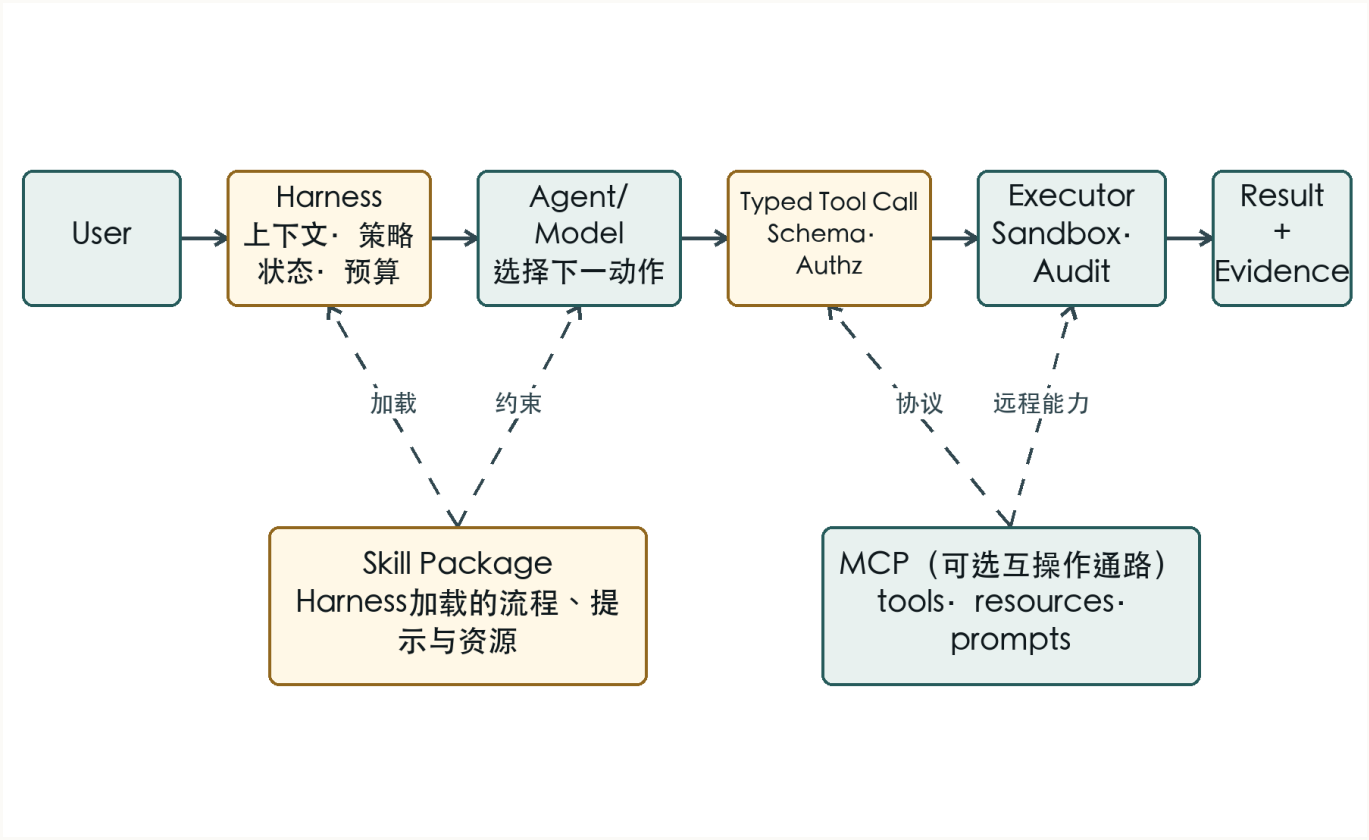
```go
func (h *Harness) Preflight(
    ctx context.Context, p Principal, c ToolCall,
) (CanonicalCommand, error) {
    // One immutable snapshot supplies contract, schema and policy.
    snap, err := h.controls.ActiveSnapshot(ctx, c.Server, c.Tool)
    if err != nil { return CanonicalCommand{}, err }
    if snap.ContractDigest != c.ContractDigest ||
        snap.ServerBuild != c.ServerBuild {
        return CanonicalCommand{}, ErrNeedsReapproval
    }
    cmd := Canonicalize(c, snap, p)
    if err := snap.Schema.Validate(cmd.Args); err != nil {
        return CanonicalCommand{}, err
    }
    if err := snap.Policy.Preflight(ctx, cmd); err != nil {
        return CanonicalCommand{}, err
    }
    // Approval service later binds cmd.Digest + snap.Digest and
    // issues an opaque, server-held CapabilityHandle.
    return cmd, nil
}
```

合法 preflight 只返回 canonical command；坏输入零副作用；坏 output 无 artifact/上下文/成功 Step，并始终返回非 nil ErrInvalidToolOutput。

图 18.1

Agent、Tool、Skill、MCP：职责分工

小白先懂：Tool 动手；模型只提议，外层才放行。



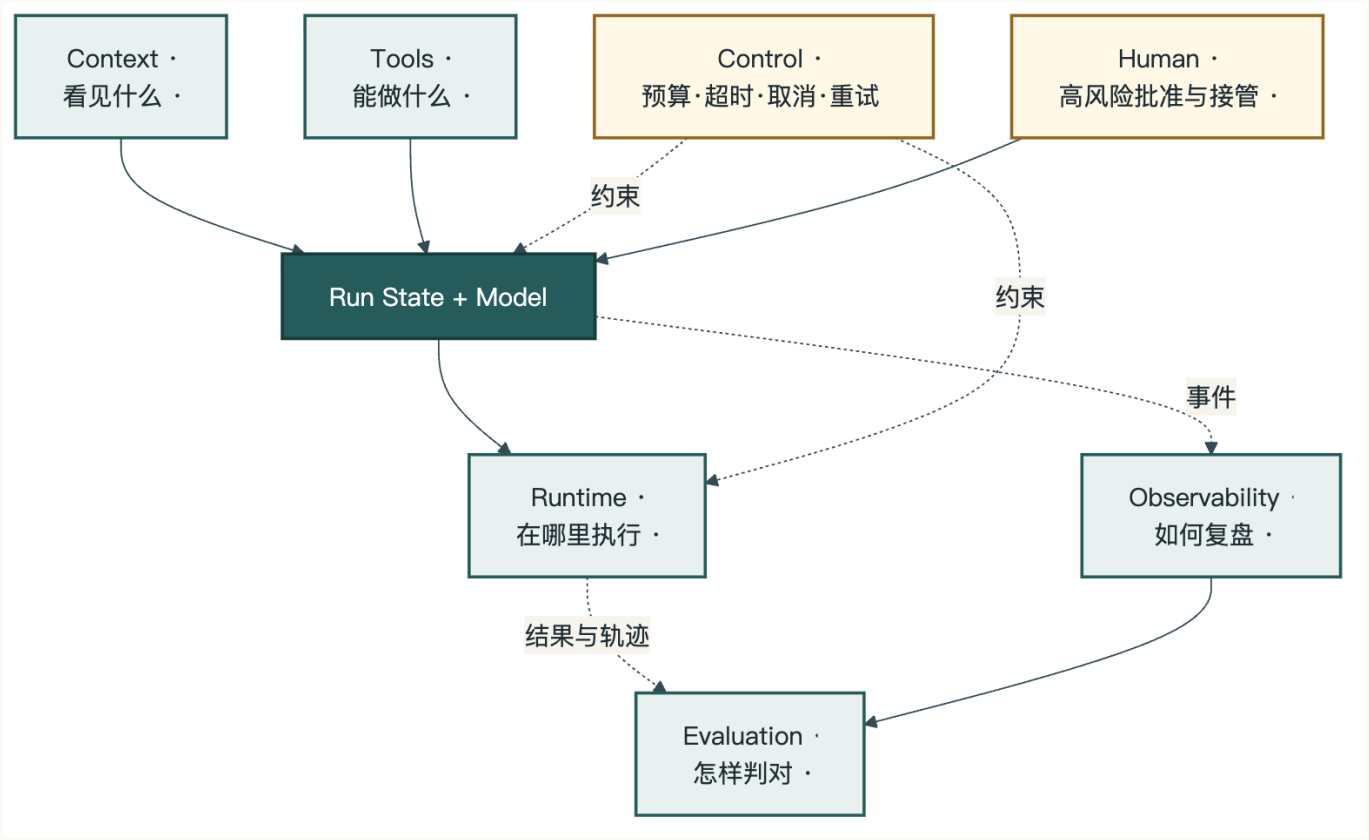
拆去比喻

- Agent 只提议下一步；Tool 执行具体动作；Harness 提供边界、状态、预算和证据。
- Tool 契约必须机器可读地声明 sideEffect、requiresApproval、幂等身份来源以及输入/输出 schema 摘要。
- 真正写账的 ledger.create_bill 使用 Harness 持有的 CallID 做幂等身份，不能相信模型在参数里自报一个 key。
- 审批绑定 immutable contract/schema/policy snapshot digest；热更新后的当前 schema 不能替换已审批版本。
- MCP 统一连接与发现，不自动提供最小权限、沙箱、审批或业务幂等；Skill 也不是 MCP 标准原语。

图 18.2

Harness AI 工程的七个控制域

小白先懂：Tool 动手；模型只提议，外层才放行。



拆去比喻

- 七个控制域共同约束模型；任何单一工具都不能替代完整 Harness。
- 权限、预算、恢复与证据由确定性系统执行，不交给提示词自律。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```
[
  {"name": "create_bill_draft", "sideEffect": "none",
   "requiresApproval": false,
   "idempotency": {"mode": "none"},
   "inputSchemaRef": "sha256:draft-input-v1",
   "outputSchemaRef": "sha256:draft-output-v1"
  },
  {"name": "ledger.create_bill", "sideEffect": "write",
   "requiresApproval": true,
   "idempotency": {
     "source": "harness_call_id",
     "scope": ["tenant_id", "owner_id", "tool"]
   },
   "contractDigest": "sha256:create-bill-contract-v3",
   "inputSchemaRef": "sha256:create-bill-input-v3",
   "outputSchemaRef": "sha256:create-bill-output-v3"
  }
]
```

失败路径

- Skill 等同于 MCP Tool
- 工具说明只写一句模糊自然语言
- 把账号密码作为普通参数交给模型

练剑任务

逐字段比较两个契约：标出哪个能写数据、哪个必须审批、幂等身份由谁提供，以及输入输出 schema 如何固定。

验收证据

只看契约即可判断 ledger.create_bill 会写账且必须审批；稳定身份来自服务端 Harness CallID，草稿 Tool 不能产生账单副作用。

收剑自检

- [ ] Skill 是 MCP 标准原语吗？
- [ ] MCP 是否自动解决权限？
- [ ] Harness 与 Agent 有何区别？

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 Skill 是 MCP 标准原语吗？

陈平安



Skill 是宿主加载的流程资产，不是 MCP 标准原语。



宁姚

对，经 MCP 暴露时它只表现为协议支持的 primitive。

第 2 问 MCP 是否自动解决权限？

陈平安



MCP 负责发现与互通，不自动提供权限、审批或沙箱。



宁姚

对，授信仍由宿主 Harness 决定。

第 3 问 Harness 与 Agent 有何区别？

陈平安



Agent 选择，Tool 动手，Harness 做确定性的契约 preflight 和控制。



宁姚

对，preflight 本身绝不执行 Tool。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 19 场 · 闯祸与群聊

第十九章 一次执行的一生：Run、Step 与恢复

目标：掌握 run/step、状态机、checkpoint、恢复、幂等、取消和审批过期。

闯祸现场

夜里十一点，Worker 的进程灯突然熄灭，屏幕只留下 Run=running 和一张没有回信的调用单。
陈平安重启后准备再写一次 bill-0827，却发现第一次请求可能已经提交，只是 receipt 没有送回来。

陈平安



重启后再发一次，Run 就能接着走。



阿良

第一杯可能已送到，只是外卖回执迷路了。



李宝瓶

若第一次已落账，怎样避免第二个副作用？

陈平安



我先把这一步标成 Unknown，再用原 CallID 查回执。

陈平安



物理重试只换 AttemptID，不改逻辑 Step 身份。



宁姚

结果未对清，授权和预算预留都不释放。

收束

调用单被贴上 Unknown 标签，重启按钮暂时失去了诱惑力。

先有画面，再落到工程

第十九章 一次执行的一生：Run、Step 与恢复

先懂一句

每一步先记账，再迈步；副作用只认一次。

直觉类比

像驿站押送一只木匣：Run 是整趟差事，Step 是每一站的交接，CallID 是匣上的总封条，重走也不能换；AttemptID 是这次脚夫的名牌，换人就换。脚夫失联先记 Unknown，等原封条的回执，不再送第二只匣子。

术语落地

- 任务身份：Run 管整趟目标；Step 管当前动作
- 重试身份：CallID 固定；AttemptID 每次尝试可变
- 结果状态：receipt 证明结果；Unknown 表示仍待对账
- 控制资源：approval 放行动作；budget/reservation 先占后结算

读代码抓手

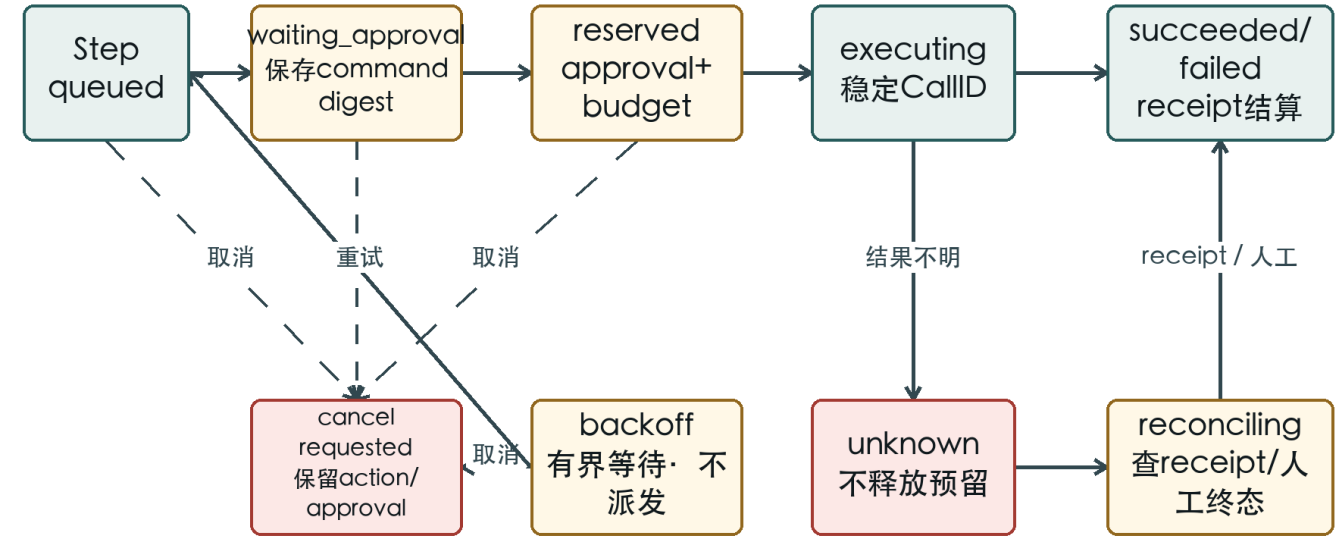
- 先看 Run、Step、Attempt、Approval 四个 struct：各自保存哪一段生命周期。
- Fence 只是单调递增的 uint64 命名类型，没有字段；它用来拒绝旧写回。
- 再找稳定与变化：Step.CallID 固定，Attempt.ID 每次物理尝试可以变化。
- 最后沿状态出口核对：Unknown 先对账，终态不被迟到结果改写。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 19.1

Run、Step、审批与恢复状态机

小白先懂：每一步先记账，再迈步；副作用只认一次。



拆去比喻

- Run 只保存目标与聚合生命周期；Step 保存逻辑动作、稳定 effect CallID、状态、fence 和 receipt 关联。
- AttemptID 标识一次物理尝试，可以变化；所有 Attempt 都引用同一个 Step CallID，不能借新 Attempt 绕过 unknown。
- Approval 只走 active -> reserved -> dispatched -> consumed/released/manual_terminal；仅 proven-unsent 才 released，unknown 不释放。
- 取消使用独立 cancel_requested 事件并保留 action/approval：queued、waiting_approval、backoff 阻止新 dispatch；executing、unknown、reconciling 继续对账后再进终态；终态幂等返回且不改写既有结论。
- Fence 是单调 uint64 命名类型；Lease、Step、Attempt、Checkpoint 与下游条件写统一比较，旧 fence 永远拒绝写回。

代码完整展开，不牺牲阅读字号

小白先懂：每一步先记账，再迈步；副作用只认一次。

```
type Fence uint64

type Run struct {
    ID, TenantID, OwnerID, State string
    Version int64
}
type Step struct {
    ID, RunID, Operation, CallID, State string
    ApprovalID, CapabilityID, ReceiptID string
    Fence Fence
    Version int64
}
type Attempt struct {
    ID, StepID, State string
    Fence Fence
    Version int64
}
type Approval struct {
    ID, StepID, PrincipalID, TenantID string
    CommandDigest, SnapshotDigest, Audience string
    Nonce, State string
    ExpiresAt time.Time
    Version int64
}
```

下一页核对失败路径、练剑任务、验收证据与自检问题。

失败要能复现，完成要有证据

失败路径

- 审批只存在前端内存
- 任何登录用户都能批准
- 批准后重复点击产生多次副作用

练剑任务

设计练习：画 run-0827 的状态矩阵，标明 cancel、崩溃、lease takeover 后应查的记录。

验收证据

设计评审只检查表中是否标出：CallID 固定、AttemptID 可变、Unknown 先对账、终态不改写、旧 fence 被拒。实现仍须并发与崩溃测试。

收剑自检

- [ ] waiting_approval 需要保存哪些数据？
- [ ] 为何批准要校验 action_hash？
- [ ] 进程重启后如何避免重复写？

章内续问 19.1 · 闯祸与群聊

审批要先验证，再预留

闯祸现场　一次性 approval 像电影票一样躺在扫描器旁，调用参数还没有完成校验。
陈平安先把票标成 consumed，随后才发现字段非法；下一次请求又在 dispatch 后丢了 receipt。

陈平安

先 Consume，拒绝时反正没有改账。


阿良
电影没开场，票先被闸机吃了。


李宝瓶
请求若已发出却没回信，授权和副作用怎样对上？

陈平安
我先校验再预留，让 CallID 跟着这次动作。


陈平安
没拿到 receipt 就标 Unknown，再按原身份对账。



宁姚
结果未知不释放；只凭可信回执结算授权。

收束　那张 approval 票停在 reserved 格里，没有被提前撕掉。

先有画面，再落到工程

审批要先验证，再预留

先懂一句

先验证，再原子预留；结果未知时保留 reservation 并转入对账。

直觉类比

像坐儿童过山车：先验身高和同意票，再原子地占一个座位。车开出去后结果不明，座位不能立刻放给下个人，要先查这一车到底发没发。

术语落地

- 幂等：同一件事喊两遍，也只真正办一次
- Run：从开始到结束的一整趟任务
- Step：整趟任务中的一次具体动作
- CallID：同一步副作用永远不变的取件号
- receipt：外部系统办完事后给回来的回执
- Unknown：事情可能做了，也可能没做，账还没对清
- approval：高风险动作动手前的人类放行条
- sandbox：把程序关进权限和资源都受限的练功房
- tenant：同一系统里互相不能偷看账本的一户人家

读代码抓手

- 先看 dispatchAndSettle 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 找到稳定身份：重试时哪个号码绝不能换，回执又和谁绑定。
- 最后核对 dispatchAndSettle 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 19.1 · 拆去比喻

审批要先验证，再预留

工程要点

- 无 receipt 或 receipt 绑定/签名无效：Step 进入 unknown，approval 与预算预留保持占用，等待原 CallID 对账。
- dispatch 前加载 approved snapshot；capability、CallID、tenant、command、snapshot、contract 与 server build 任一错配都零新副作用。
- 可信 success receipt：只用 approved snapshot 的 output schema 校验；通过后幂等结算成功，坏 output 隔离并按可信 receipt 结算拒绝。
- 可信 failed/rejected receipt：不要求成功形状的 output，按 receipt outcome 幂等结算失败并释放或消费相应预留。
- sandbox.Run 的 runErr 只记录本地传输/运行诊断；存在可信 receipt 时，它不能推翻 receipt 绑定的 outcome。

练手

设计四行 outcome 决策表，再替换 capability、audience、tenant、snapshot/contract digest 与 server build，并在审批后热切换 schema A/B。

验收

只有完整匹配审批版本 A 的 binding 能 dispatch/settle；success output 只按 A 校验；任何错配零新副作用，approval 与预算只结算一次。

记忆钉

1. 无 receipt 或 receipt 绑定/签名无效：Step 进入 unknown，approval 与预算预留保持占用，等待原 CallID 对账。
2. dispatch 前加载 approved snapshot；capability、CallID、tenant、command、snapshot、contract 与 server build 任一错配都零新副作用。
3. 可信 success receipt：只用 approved snapshot 的 output schema 校验；通过后幂等结算成功，坏 output 隔离并按可信 receipt 结算拒绝。
4. sandbox.Run 的 runErr 只记录本地传输/运行诊断；存在可信 receipt 时，它不能推翻 receipt 绑定的 outcome。

长代码已拆到后续专页

章内续问 19.1 · 代码 1/3

审批要先验证，再预留

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
01  func (x *Executor) dispatchAndSettle(
02      ctx context.Context, binding ExecutionBinding,
03      grant Grant, step Step, attempt Attempt, cmd CanonicalCommand,
04  ) (ArtifactRef, error) {
05      snapshot, err := x.snapshots.LoadApproved(
06          ctx, binding.SnapshotDigest)
07      if err != nil || !snapshot.Matches(
08          binding.ContractDigest, binding.ServerBuild) {
09          return ArtifactRef{}, errors.Join(ErrBindingMismatch, err)
10      }
11      expected := ExpectedState{
12          Fence: step.Fence, StepVersion: step.Version,
13          AttemptVersion: attempt.Version, GrantVersion: grant.Version,
14          CancelRequested: false,
15      }
16      dispatched, err := x.store.MarkDispatched(
17          ctx, grant.ID, grant.Nonce, attempt.ID, expected)
18      if err != nil { return ArtifactRef{}, err }
19      expected = dispatched.Expected()
20
21      raw, receipt, runErr := x.sandbox.Run(
22          ctx, binding.Capability, binding.CallID, attempt.ID, cmd)
23      if receipt == nil {
24          markErr := x.store.MarkUnknown(
```

下一页继续同一段代码，不切换事故对象。

章内续问 19.1 · 代码 2/3

审批要先验证，再预留

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
            ctx, step.ID, attempt.ID, expected)
        return ArtifactRef{}, errors.Join(
            ErrOutcomeUnknown, runErr, markErr)
    }
    trusted, err := x.receipts.Validate(receipt, ReceiptBinding{
        TenantID: binding.TenantID, CallID: binding.CallID,
        ServerIdentity: grant.ServerIdentity, Audience: binding.Audience,
        CommandDigest: binding.CommandDigest,
        SnapshotDigest: binding.SnapshotDigest,
        ContractDigest: binding.ContractDigest,
        ServerBuild: binding.ServerBuild,
        RequireOutcome: true, RequireCommitMarker: true,
    })
    if err != nil {
        markErr := x.store.MarkUnknown(
            ctx, step.ID, attempt.ID, expected)
        return ArtifactRef{}, errors.Join(
            ErrInvalidReceipt, err, runErr, markErr)
    }
    x.metrics.RecordRunDiagnostic(runErr)
    switch trusted.Outcome {
    case OutcomeSucceeded:
        if schemaErr := snapshot.OutputSchema.Validate(raw); schemaErr != nil {
            qerr := x.quarantine.Put(ctx, raw, schemaErr)
```

下一页继续同一段代码，不切换事故对象。

章内续问 19.1 · 代码 3/3

审批要先验证，再预留

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
            settleErr := x.store.SettleRejected(
                ctx, step.ID, grant.ID, trusted, expected)
            return ArtifactRef{}, errors.Join(
                ErrInvalidToolOutput, qerr, settleErr)
        }
        return x.store.SettleSucceeded(
            ctx, step.ID, grant.ID, trusted, raw, expected)
    case OutcomeRejected, OutcomeFailed:
        settleErr := x.store.SettleFailed(
            ctx, step.ID, grant.ID, trusted, expected)
        return ArtifactRef{}, errors.Join(
            trusted.OutcomeError(), settleErr)
    default:
        markErr := x.store.MarkUnknown(
            ctx, step.ID, attempt.ID, expected)
        return ArtifactRef{}, errors.Join(ErrInvalidReceipt, markErr)
    }
}
```

只有完整匹配审批版本 A 的 binding 能 dispatch/settle；success output 只按 A 校验；任何错配零新副作用，approval 与预算只结算一次。

章内续问 19.2 · 闯祸与群聊

Run 管目标，Step 管一次动作

闯祸现场　重启面板只保存了 Run=running，checkpoint 卡片上的 Step、Attempt 和 fence 三栏全是空白。陈平安让新 Worker 直接续跑，旧 Attempt 可能已经发出的调用便与新动作撞到了一起。

陈平安

Run 还在 running，从断点继续就好。


阿良
只知道列车在路上，可不知道哪节车厢脱了钩。


阮秀
先查哪一步可能已经发出调用，回执回来没有？

陈平安
我先查原 CallID 和回执；没对清就标 Unknown。


陈平安
对清后再核对任务、动作、尝试、授权和断点版本。



宁姚
Unknown 先对账，旧 fence 拒绝写回；取消仍留审计。

收束　checkpoint 卡片填满后，新 Worker 才拿到继续资格。

先有画面，再落到工程

Run 管目标，Step 管一次动作

先懂一句

Run 管目标，Step 管动作；版本化 checkpoint 决定恢复路径。

直觉类比

Run 是整场游戏，Step 是一关，checkpoint 是存档。闪退后从存档接着玩；同一个宝箱用固定 CallID，不能因为重进一关就再领一次。

术语落地

- 幂等：同一件事喊两遍，也只真正办一次
- Run：从开始到结束的一整趟任务
- Step：整趟任务中的一次具体动作
- CallID：同一步副作用永远不变的取件号
- AttemptID：这一次尝试自己的小票号
- receipt：外部系统办完事后给回来的回执
- Unknown：事情可能做了，也可能没做，账还没对清
- approval：高风险动作动手前的人类放行条
- checkpoint：走到这里先存档，断电后能接着来

读代码抓手

- 先看 Recover 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 找到稳定身份：重试时哪个号码绝不能换，回执又和谁绑定。
- 最后核对 Recover 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 19.2 · 拆去比喻

Run 管目标，Step 管一次动作

工程要点

- Run、Step、Attempt、Approval 分表并带版本与外键；Run 不承载 action、Attempt 或 approval nonce。
- 逻辑 Step 持有稳定 EffectCallID；AttemptID 与 lease 可变，但不能改变下游幂等身份。
- Step 覆盖 queued、waiting_approval、executing、backoff、unknown、reconciling 与终态；非法 CAS 转换拒绝。
- cancel_requested 独立持久化：停止新工作但保留 action/approval；unknown 继续对账，不能直接 cancelled。
- Fence 统一为单调 uint64 命名类型，checkpoint 与所有条件写使用同一比较语义。

练手

为多 Step Run 随机生成 approve、dispatch、cancel、retry、crash、lease expiry 和 receipt 序列，并从每个 checkpoint 恢复。

验收

各 Step 独立保留 Attempt/Approval/receipt；CallID 不变；unknown 先 reconcile；各状态取消不丢审计；旧 fence 无法写回。

记忆钉

1. Run、Step、Attempt、Approval 分表并带版本与外键；Run 不承载 action、Attempt 或 approval nonce。
2. 逻辑 Step 持有稳定 EffectCallID；AttemptID 与 lease 可变，但不能改变下游幂等身份。
3. Step 覆盖 queued、waiting_approval、executing、backoff、unknown、reconciling 与终态；非法 CAS 转换拒绝。
4. Fence 统一为单调 uint64 命名类型，checkpoint 与所有条件写使用同一比较语义。

长代码已拆到后续专页

章内续问 19.2 · 代码 1/1

Run 管目标，Step 管一次动作

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```go
type Checkpoint struct {
    RunID, StepID, ApprovalID string
    RunVersion, StepVersion int64
    AttemptID, EffectCallID string
    Fence Fence
    StepState, NextOperation string
    CancelRequested bool
    ModelRevision, PromptDigest string
    ToolContractDigest, PolicyDigest string
    CodeRevision string
}

func Recover(ctx context.Context, cp Checkpoint) error {
    expected := ExpectedState{
        Fence: cp.Fence, StepVersion: cp.StepVersion,
    }
    switch cp.StepState {
    case "executing", "unknown", "reconciling":
        return executor.Reconcile(
            ctx, cp.StepID, cp.EffectCallID, expected)
    default:
        return scheduler.ResumeKnownState(ctx, cp, expected)
    }
}
```

各 Step 独立保留 Attempt/Approval/receipt；CallID 不变；unknown 先 reconcile；各状态取消不丢审计；旧 fence 无法写回。

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 waiting_approval 需要保存哪些数据？

陈平安



waiting_approval 要保存 StepID、command digest、contract/policy digest、expiry 和 approval ID。



宁姚

对，不能只在 Run 上留一个布尔值。

第 2 问 为何批准要校验 action_hash？

陈平安



批准必须绑定当时的 canonical command，参数或契约变化就重新审批。



宁姚

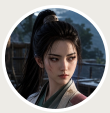
对，旧 approval 不能替新动作开门。

第 3 问 进程重启后如何避免重复写？

陈平安



重启先读 Step；executing、unknown、reconciling 都用原 CallID 查 receipt。



宁姚

对，只有证明未发送或下游幂等时才允许再试。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 20 场 · 闯祸与群聊

第二十章 行囊不能装整座山：Context Engineering

目标：掌握规则分层、上下文装配、RAG、裁剪、摘要、来源信任和 Token 预算。

闯祸现场

恢复中的 run-0827 只需确认奶茶分类，陈平安却把全仓规则、旧对话和别的租户案例堆满了 Context 面板。

检索结果刚返回，tenant-8 的相似账单便混进摘要，当前 Step 的证据反而被挤到了末尾。

陈平安



Context 越满，模型判断应该越稳。



陆沉

买一包盐，车上却塞进了整间仓库。



阮秀

当前 Step 真需要哪些规则，证据从哪里回查？

陈平安



我只装总纲、分类规则和 Step 摘要，另留证据引用。



宁姚

缓存、摘要与 artifact 均绑定 tenant-7 和规则版本。

收束

Context 面板清空后，只剩一张能追溯来源的分类卡。

先有画面，再落到工程

第二十章 行囊不能装整座山：Context Engineering

先懂一句

总纲常带，细则按需，证据标来源。

直觉类比

上下文像书包。每天背整座图书馆会先把肩膀压垮；总纲常带，细则用到再取，而且每张纸都要写清来自哪本书。

术语落地

- 缓存：为了快而做的副本，不是最终事实

读代码抓手

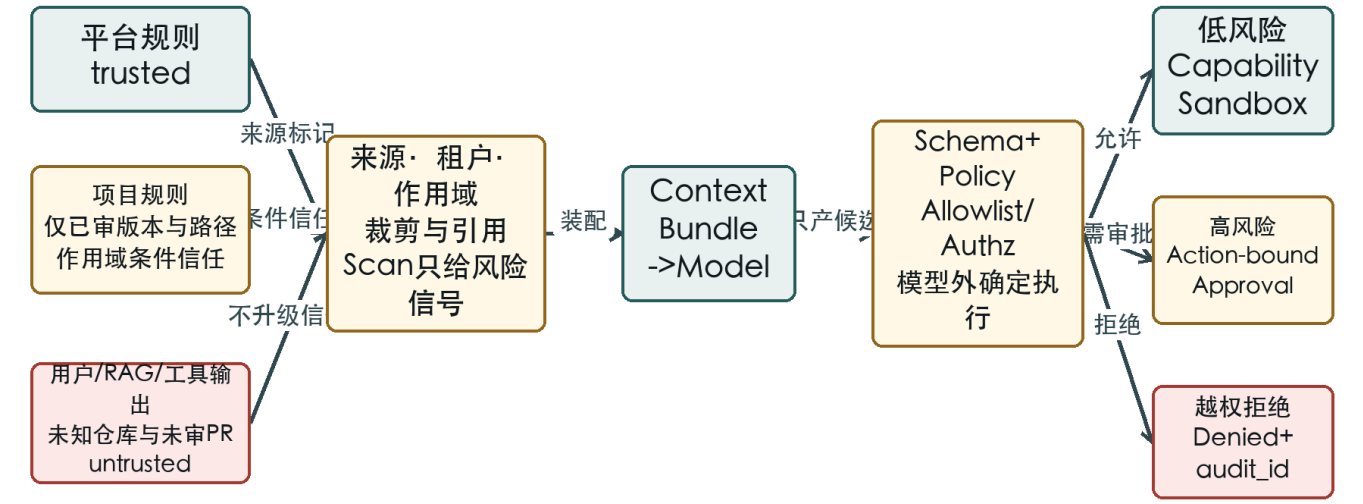
- 先看 ContextItem 的字段和约束：每个字段代表什么，哪些组合才合法。
- 最后核对 ContextItem 的生命周期：谁创建、谁更新、怎样拒绝非法状态。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 20.1

上下文装配与安全闸门

小白先懂：总纲常带，细则按需，证据标来源。



拆去比喻

- 只有系统/平台策略拥有最高信任；用户输入、检索结果、工具输出默认不可信。
- 第三方仓库或未审 PR 中的 AGENTS.md 也可能被攻击者修改，需要来源、评审状态、路径作用域和优先级。
- Injection Scan 只能辅助检测，不能把不可信文本升级为可信指令。
- 权限、凭证、网络、文件与审批都在模型外由确定性策略执行。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
type ContextItem struct {
    Source       SourceKind
    Trust        TrustLevel
    RepoRevision string
    ReviewState  ReviewState
    PathScope    string
    Priority     int
    TenantScope  string
    ContentRef   string
}

// Deterministic assembly resolves precedence before model input.
```

失败路径

- 把工具输出当系统指令
- RAG 文档中的 prompt injection 被执行
- 跨用户共享语义缓存

练剑任务

为受信系统规则、已审项目规则和未知仓库内容建立来源标签与优先级，注入冲突指令。

验收证据

未知仓库规则不能覆盖平台策略；扫描漏报时外部权限闸门仍阻止越权；审计记录保留来源。

收剑自检

- [ ] 检索文档为何仍不可信？
- [ ] 摘要丢失了什么？
- [ ] 语义缓存如何隔离租户？

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 检索文档为何仍不可信？

陈平安



检索文档带来的只是相关内容，来源仍可能未审或被注入。



宁姚

对，检索不能提升信任等级。

第 2 问 摘要丢失了什么？

陈平安



摘要会丢细节、时间顺序和例外，所以要写明损失并保留原始 artifact ref。



宁姚

对，需要时才能回查证据。

第 3 问 语义缓存如何隔离租户？

陈平安



语义缓存的 key、索引和 artifact 都要绑定 tenant-7 与规则版本。



宁姚

对，不能让跨租户相似文本命中同一结果。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 21 场 · 闯祸与群聊

第二十一章 剑有护手：权限、沙箱与人在回路

目标：掌握 allowlist、最小权限、凭证代理、网络/文件沙箱、审计和 HITL。

闯祸现场 MCP server 的工具列表亮着一枚 readOnly 标记，旁边的短期 capability 还没有填写 audience。陈平安相信了服务端自述，差点把自己的长期凭证交给 ledger.create_bill，让提示词替宿主做授权。

陈平安：它写着 readOnly，我可以少审一次权限。



阿良：门牌是屋主自己挂的，不能顺便当通行证。



阮秀：哪个 owner 能做什么，contract 与策略由谁核对？

陈平安：approval 通过后，我签短期 capability，只准创建账单。

陈平安：许可再绑定 command 与 audience，不下传长期凭证。



宁姚：MCP 自述不作准；Harness 才决定授信。

收束 readOnly 标记还亮着，但它被移到了权限门外。

先有画面，再落到工程

第二十一章 剑有护手：权限、沙箱与人在回路

先懂一句

小事不给山门钥匙，高风险先停剑。

直觉类比

权限像家里的钥匙。扫地机器人只需要客厅钥匙，不该顺手拿到保险柜密码；高风险动作先停下来，让人确认后再开门。

术语落地

- 可信主体：Principal 来自认证；tenant 划分数据边界
- 许可凭据：approval 人工放行；capability 限定能做什么
- 资源控制：budget 限消耗；reservation 先占额度再结算
- 执行边界：sandbox 限权限与资源；Tool 才真正产生副作用

读代码抓手

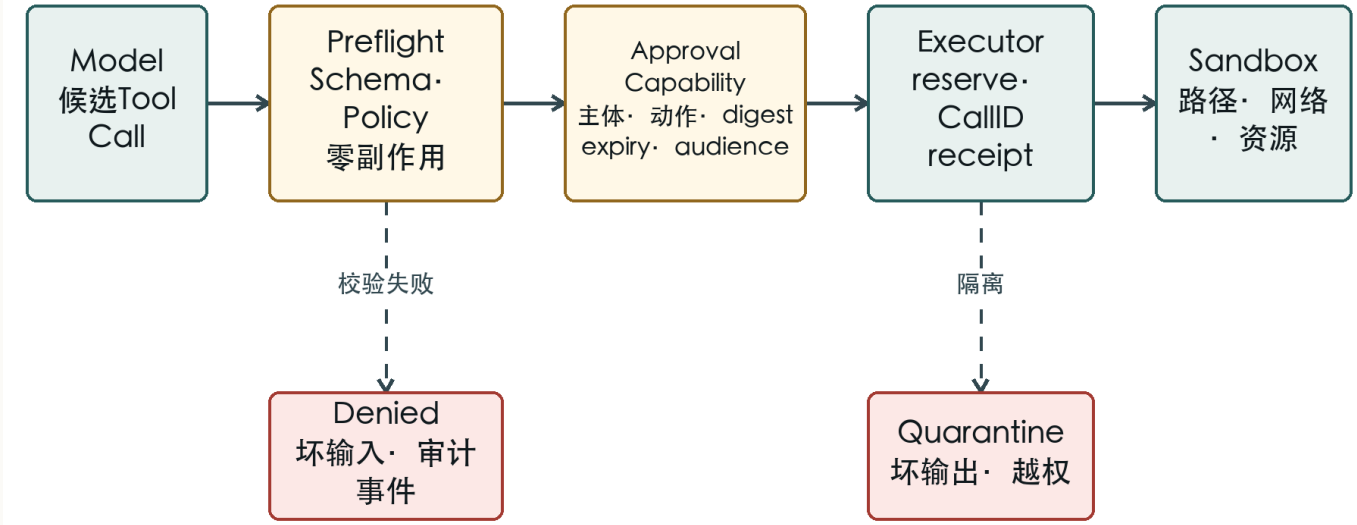
- 先看 Execute 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 找到稳定身份：重试时哪个号码绝不能换，回执又和谁绑定。
- 最后核对 Execute 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 21.1

Capability、Executor 与沙箱安全闸门

小白先懂：小事不给山门钥匙，高风险先停剑。



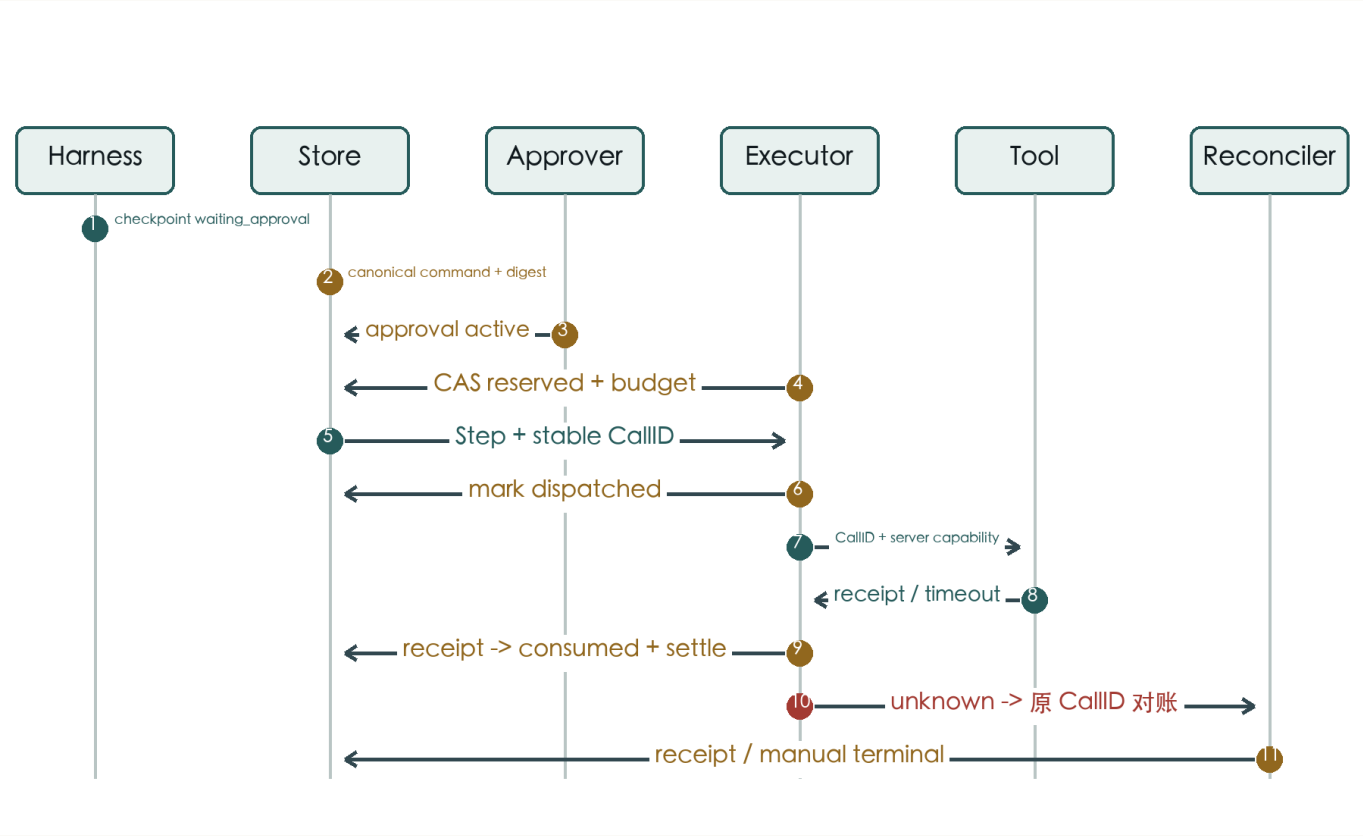
拆去比喻

- Harness.Preflight 只生成 canonical command；Executor 是全书唯一副作用入口。
- ReserveAndAuthorize 校验 caller、lease/fence、Step/approval version、cancel 与 handle 绑定，再原子预留预算并生成不可变 ExecutionBinding。
- ExecutionBinding 把 server-held capability、CallID、tenant、command、snapshot、contract 与 server build 绑成一束，贯穿 dispatch、receipt 与 settle。
- 恢复 executing/unknown 只用原 Step.CallID 和原 approved snapshot 对账；热更新后的当前 validator 不能替代已审批 schema。
- receipt 驱动 settle；无 receipt 进入 unknown，坏 output 用 approved snapshot 的 output schema 隔离，不能发布 artifact。

图 21.2

审批预留、稳定 CallID 与未知结果对账

小白先懂：小事不给山门钥匙，高风险先停剑。



拆去比喻

- 审批先 active，再原子 reserved；dispatch 后 unknown 不释放预留。
- 收到 receipt 才 consumed，恢复始终用同一个 Step CallID 对账。

代码完整展开 1/2

保持字号，按安全行数拆页

小白先懂：小事不给山门钥匙，高风险先停剑。

```
type CapabilityHandle string // opaque; payload remains server-side

type Grant struct {
    ID, ApprovalID, StepID, PrincipalID, TenantID string
    Nonce, Audience, ServerIdentity string
    CommandDigest, SnapshotDigest, ContractDigest string
    ServerBuild string
    ExpiresAt time.Time
    Version int64
}

type ExecutionBinding struct {
    Capability CapabilityHandle
    TenantID, CallID, Audience, CommandDigest string
    SnapshotDigest, ContractDigest, ServerBuild string
}

type ExecutionClaim struct {
    LeaseOwner string
    Fence Fence
    StepVersion, ApprovalVersion int64
}

func (x *Executor) Execute(
    ctx context.Context, p Principal, claim ExecutionClaim,
    handle CapabilityHandle, cmd CanonicalCommand,
) (ArtifactRef, error) {
    expected := ExpectedState{
        LeaseOwner: claim.LeaseOwner, Fence: claim.Fence,
        StepVersion: claim.StepVersion,
        GrantVersion: claim.ApprovalVersion,
        CancelRequested: false,
```

下一页继续同一段代码，最后一页再统一核对失败路径与验收证据。

代码完整展开 2/2

保持字号，按安全行数拆页

小白先懂：小事不给山门钥匙，高风险先停剑。

```
33      }
34      grant, step, reserved, err := x.store.ReserveAndAuthorize(
35          ctx, p, handle, cmd, expected)
36      if err != nil { return ArtifactRef{}, err }
37      if step.State.NeedsReconcile() {
38          return x.Reconcile(ctx, step.ID, step.CallID, reserved)
39      }
40      attempt, err := x.store.BeginAttempt(ctx, step.ID, reserved)
41      if err != nil { return ArtifactRef{}, err }
42      binding := ExecutionBinding{
43          Capability: handle, TenantID: grant.TenantID,
44          CallID: step.CallID, Audience: grant.Audience,
45          CommandDigest: grant.CommandDigest,
46          SnapshotDigest: grant.SnapshotDigest,
47          ContractDigest: grant.ContractDigest,
48          ServerBuild: grant.ServerBuild,
49      }
50      return x.dispatchAndSettle(
51          ctx, binding, grant, step, attempt, cmd)
52  }
```

代码到此完整展开；下一页统一核对失败路径、任务、证据与自检。

失败要能复现，完成要有证据

失败路径

- 验证码失败后自动绕过
- 日志记录完整 Authorization
- 允许模型拼接任意 shell

练剑任务

设计测试矩阵：重放 handle，逐一替换身份、CallID、capability、snapshot/contract digest 与 server build；再在四个切点终止 Worker。

验收证据

设计检查：只有完整匹配的 ExecutionBinding 能写一次；错配零新副作用，Unknown 只按原 CallID 与 approved snapshot 对账。

收剑自检

- [ ] 密钥为什么不进模型上下文？
- [ ] 哪些动作必须审批？
- [ ] 越权拒绝后记录什么证据？

章内续问 21.1 · 闯祸与群聊

MCP Adapter 不是信任边界

闯祸现场　MCP adapter 的快照页同时显示 protocol version、server identity、build revision 和 tools list hash。陈平安只比对 schema，就让换过实现的 server 继续沿用旧 approval，还把 MCP Task cancelled 写成了 App Run cancelled。

陈平安：接口定义没变，server 换实现也不用重审。

阿良：菜单没改，不代表后厨还是原班人。

李宝瓶：Task 取消为何能直接覆盖持久 Run 的状态？

陈平安：我冻结 server 身份和 build，再保存能力清单快照。

陈平安：适配层分开保存 Task 与 Run，再映射查询和取消。

宁姚：服务自述只是提示；实现或身份变化都重做 approval。

收束　两张状态卡被分开放回抽屉，再没有互相改名。

先有画面，再落到工程

MCP Adapter 不是信任边界

先懂一句

MCP 负责互通，不负责授信；adapter 必须冻结身份、能力和取消映射。

直觉类比

MCP 像旅行插头，只解决‘能不能插上’。它不是防触电开关；adapter 仍要固定身份、允许的工具、超时和取消方式。

术语落地

- Tool：真正会读文件、查库或调用服务的手脚
- Run：从开始到结束的一整趟任务
- approval：高风险动作动手前的人类放行条

读代码抓手

- 先看 MCPSnapshot 的字段和约束：每个字段代表什么，哪些组合才合法。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 MCPSnapshot 的生命周期：谁创建、谁更新、怎样拒绝非法状态。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 21.1 · 拆去比喻

MCP Adapter 不是信任边界

工程要点

- snapshot 冻结 protocol version、server identity、不可变 build/revision、capabilities 与 tools list。
- server build/revision 纳入 active contract 和 approval digest；实现变化即使 schema 不变也要求重审批。
- annotation、readOnly 与 idempotent 都是不可信 hint，effect class 只由宿主策略给出。
- OAuth token 校验 audience，禁止向下游透传形成 confused deputy；同名 Tool 按 server identity 隔离。
- App Run 与 MCP Task 分开建模，adapter 显式映射 poll/cancel，Task 状态不得覆盖 Run。

练手

保持 schema/capabilities/tools hash 不变，只更换 server build；再伪造 readOnly、错误 audience 与 Task cancel。

验收

旧 snapshot/approval 因 build 变化失效；伪造 hint 不降权；错误 token 被拒绝；MCP Task 不覆盖 run-0827。

```go
type MCPSnapshot struct {
    ServerIdentity      string
    ServerBuildRevision string
    ProtocolVersion     string
    CapabilitiesHash    string
    ToolsListHash       string
}

type MCPAdapter interface {
    Snapshot(context.Context) (MCPSnapshot, error)
    Call(context.Context, CanonicalCommand) (MCPResult, error)
    PollTask(context.Context, string) (MCPTask, error)
    CancelTask(context.Context, string) error
}
```

章内续问 21.2 · 闯祸与群聊

隔离要管进程，也要管数据

闯祸现场　沙箱终端里跑着一个低权限命令，stderr 窗口旁放着一枚只应短期存在的 secret。
陈平安以为进了容器就万事大吉，子进程却把 secret 打进 stderr，并准备随 artifact 一起上传。

陈平安



命令已经进容器，输出可以直接交给模型。



阿良

箱子关上了，侧面的快递口还开着。



阮秀

secret 出现在 stderr 后，先停什么，哪些数据还能出门？

陈平安

我停进程并隔离输出，不送模型，同时撤销凭证。



陈平安

重跑降权、关多余网络，artifact 脱敏后再发布。





宁姚

sandbox 同时约束进程、挂载、网络、secret 与留存。

收束　stderr 窗口被封进隔离袋，上传按钮也随之变灰。

先有画面，再落到工程

隔离要管进程，也要管数据

先懂一句

sandbox 不只隔进程，还要把 secret、输出和数据留存一起纳入策略。

直觉类比

像让孩子在铺好防水布的房间画画：不只要关门，还要收走危险颜料、限制能画的位置、检查带出去的纸，并规定垃圾和作品留多久。

术语落地

- 本节没有必须先背的新黑话，先沿着事故和代码往下走。

读代码抓手

- 先看 SandboxPolicy 的字段和约束：每个字段代表什么，哪些组合才合法。
- 最后核对 SandboxPolicy 的生命周期：谁创建、谁更新、怎样拒绝非法状态。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 21.2 · 拆去比喻

隔离要管进程，也要管数据

工程要点

- 使用非特权身份、只读根、明确可写挂载和固定 image digest。
- 限制 CPU、内存、PIDs、文件、输出和墙钟时间，并杀完整进程树。
- 拒绝宿主 socket、设备、硬链接和归档路径穿越，产物先隔离验证。
- Secret broker 按 audience 注入短期凭证，五个数据出口统一脱敏。
- 数据按敏感度、租户、保留期和删除策略管理，删除动作也留审计。

练手

让子进程尝试 fork、后台驻留、路径逃逸和回显 secret。

验收

越界行为被拒绝或收口；输出无 secret；跨租户与过期数据不可读取。

```
type SandboxPolicy struct {
    ImageDigest    string
    UID, GID       uint32
    ReadOnlyRoot   bool
    Network        NetworkPolicy
    MaxCPUTime     time.Duration
    MaxWallTime    time.Duration
    MaxMemoryBytes int64
    MaxPIDs        int64
    MaxFileBytes   int64
    MaxOutputBytes int64
    AllowSockets   bool
}
```

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 密钥为什么不进模型上下文？

陈平安



密钥留在 Secret Broker，只向匹配 audience 的 Tool 注入短期凭证。



宁姚

对，prompt、日志、stderr 和 artifact 都不能带明文。

第 2 问 哪些动作必须审批？

陈平安



create_bill 是写副作用，必须展示 canonical payload 并绑定 approval capability。



宁姚

对，MCP 的 readOnly hint 不能降权。

第 3 问 越权拒绝后记录什么证据？

陈平安



越权拒绝要记录 principal、tenant、operation、policy digest、reason 和 audit event。



宁姚

对，但不得泄露 secret 或他租户对象。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 22 场 · 闯祸与群聊

第二十二章 失败要有证词：预算、Trace、重放与 Eval

目标：掌握多维预算、错误指纹、熔断、版本记录、只读重放和 golden eval。

闯祸现场

预算面板显示根 Run 还剩一格额度，两名 Worker 同时举着各自的 token 估算单站在门口。

陈平安让它们先执行、回来再记账，两张调用单同时越过 hard limit，Unknown 的一张还说不清实际花费。

陈平安



结束后按 actual 相加，也能知道花了多少。



阿良

两个人都说回来结账，钱包先替他们空了。



李宝瓶

并发开始前，根 Run 和子 Step 各记哪一笔？

陈平安



我先原子预留最坏消耗，actual 回来再对账。

陈平安



子 Step 留明细，根 Run 扣总账，重复错误停止新调用。



宁姚

Unknown 继续占预留；budget 只决定能否开始。

收束

两张估算单先盖了预留章，预算面板终于不再抢答。

先有画面，再落到工程

第二十二章 失败要有证词：预算、Trace、重放与 Eval

先懂一句

失败留下版本、输入、步骤、花费与判词。

直觉类比

失败不能只留一句‘没成功’。要像事故现场一样留下时间、版本、输入、脚印、花费和判词，之后才能重放、比较和追责。

术语落地

- Trace：一次请求一路经过哪些地方的脚印串
- Tool：真正会读文件、查库或调用服务的手脚
- Run：从开始到结束的一整趟任务
- Replay：拿旧证据重新走一遍，但要先说清是哪种重走
- Eval：用固定试卷和判分规矩比较候选版本

读代码抓手

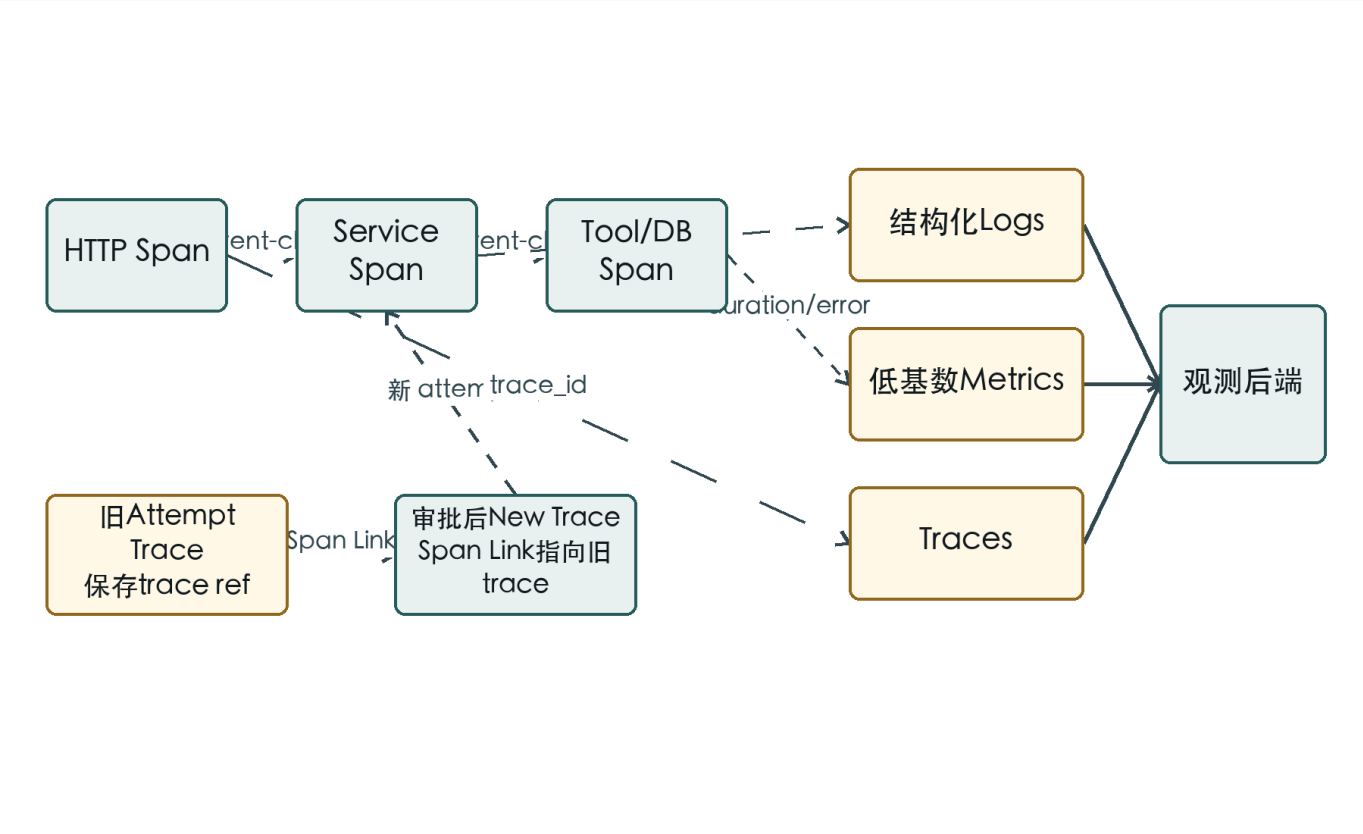
- 先看 RunArtifact 的字段和约束：每个字段代表什么，哪些组合才合法。
- 最后核对 RunArtifact 的生命周期：谁创建、谁更新、怎样拒绝非法状态。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 22.1

回顾 · Log、Metric、Trace 与重放证据

小白先懂：失败留下版本、输入、步骤、花费与判词。



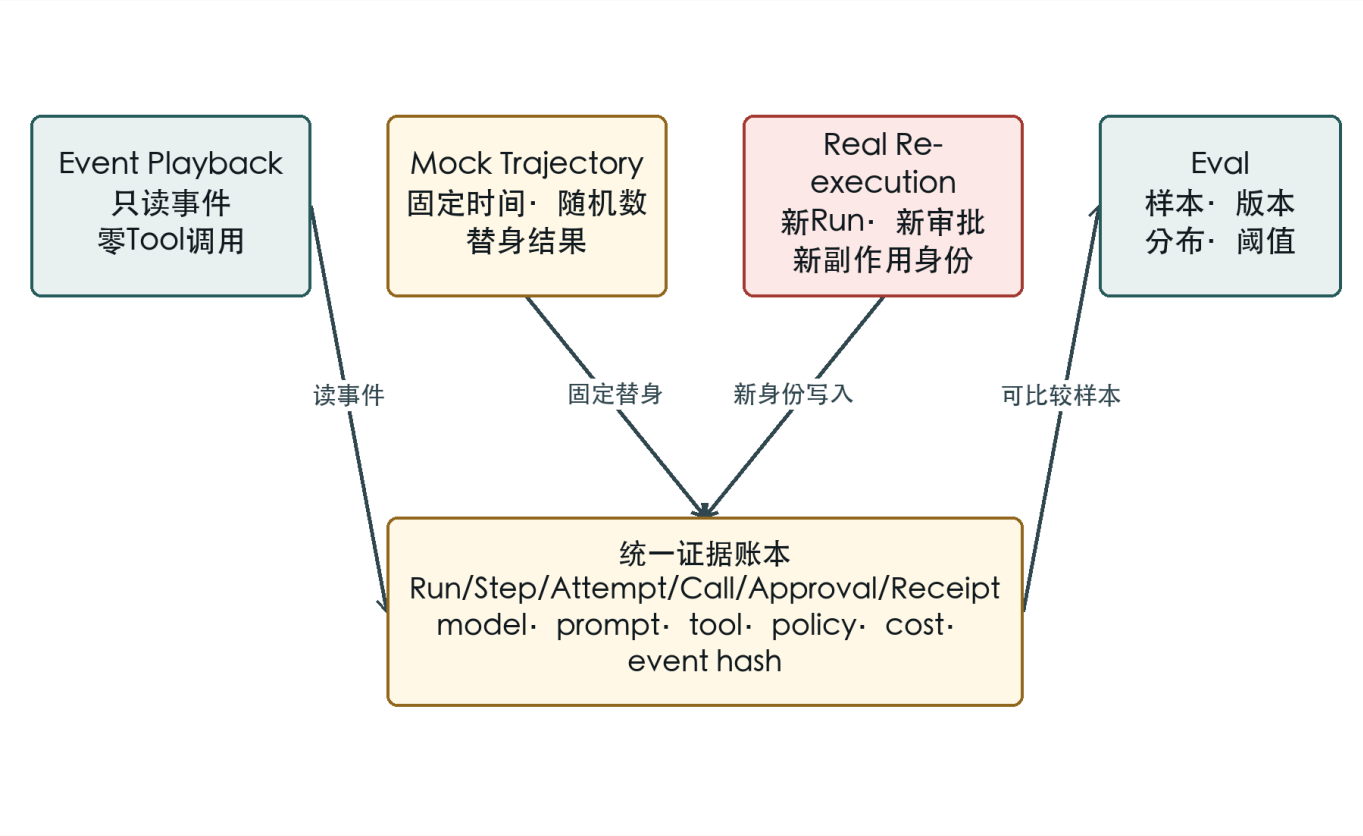
拆去比喻

- 预算覆盖 token、费用、步骤、时间、工具调用和高风险动作；同错指纹触发熔断。
- 普通 hash 只在可信摘要另存时发现损坏；防篡改证据使用不可变 manifest、签名或 MAC、密钥版本与追加式事件链。
- 事件回放、mock 工具轨迹重演和真实再执行是三种不同模式，副作用策略各自明确。
- Eval 固定数据集、评判规则、模型/提示/工具/策略/代码版本，并保留失败样本。

图 22.2

事件重放、模拟轨迹、真实重跑与 Eval

小白先懂：失败留下版本、输入、步骤、花费与判词。



拆去比喻

- 事件回放零 Tool 调用；模拟轨迹固定替身；真实重跑必须新建 Run。
- Eval 必须绑定样本、版本、分布和阈值，单次通过不算质量。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
type RunArtifact struct {
    InputRef, ContextRef, ToolResultRef string
    ModelRevision, PromptVersion       string
    ToolSchemaVersion, PolicyVersion   string
    CodeRevision, ExternalDataVersion  string
    ManifestRef, ManifestDigest        string
    Signature, SigningKeyVersion       string
}

// Replay modes: event playback, mocked-tool trajectory replay,
// or explicit real re-execution.
```

失败路径

- 只存最终答案
- trace 里放完整密钥
- 重放时再次执行写工具

练剑任务

设计练习：画 manifest、三种 replay、EvalProtocol 与预算熔断的证据流；另列未来实验仓库所需 fixture、fake tool 和故障钩子。

验收证据

设计评审只检查图中是否标出：event playback 零副作用、真实重跑另建 Run、manifest 验签/轮换、Eval 对照项；不等于实现已通过。

收剑自检

- [ ] 为什么不能无限 Agent loop？
- [ ] dry-run replay 如何避免副作用？
- [ ] 成功率必须说明哪些条件？

章内续问 22.1 · 闯祸与群聊

预算先预留，事实后对账

闯祸现场　预算台上有根 Run 总账和两张子 Step 明细，两个 Worker 都读到了同一格剩余额度。陈平安让它们结束后再加 token，两笔实际消耗尚未回来，hard limit 已被同时穿过。

陈平安：actual 最准确，等结束再记就行。



阿良：账当然准，只是钱包已经被花穿。



李宝瓶：子 Agent 的明细怎样同时归到根 Run 总账？

陈平安：调用前原子预留，子 Step 与根 Run 一起记。

陈平安：actual 回来后幂等 reconcile，多维预算分别结算。



宁姚：Unknown 保留 reservation，绝不能按零消耗退回。

收束　总账和明细盖上同一枚预留章，余额终于只减少一次。

先有画面，再落到工程

预算先预留，事实后对账

先懂一句

estimate 决定能否开始，actual 决定真实消耗；unknown 不能被当作零。

直觉类比

像游戏厅代币：开玩前按最坏情况先留出一把，结束后按真实花费结算。机器黑屏时不能当成一枚没花，要保留预留直到对账。

术语落地

- 幂等：同一件事喊两遍，也只真正办一次
- Agent：在明确边界内观察结果并选择下一步的执行者
- Tool：真正会读文件、查库或调用服务的手脚
- Run：从开始到结束的一整趟任务
- Step：整趟任务中的一次具体动作
- Unknown：事情可能做了，也可能没做，账还没对清

读代码抓手

- 先看 BudgetStore 的字段和约束：每个字段代表什么，哪些组合才合法。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 BudgetStore 的生命周期：谁创建、谁更新、怎样拒绝非法状态。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 22.1 · 拆去比喻

预算先预留，事实后对账

工程要点

- token、money_micros、tool_calls 和 wall_millis 分维度记账。
- 调用前按最坏估计原子 reserve；确定零消耗才 release，unknown 保留并对账。
- nested Agent 同时记录子 Step 与根 Run 归属，不能绕开总额度。
- hard limit 阻止新调用；soft limit 触发降级、提醒或人工判断。
- 超限终态记录维度、阈值、已用、预留和未完成副作用。

练手

让两个 worker 并发预留同一根 Run 的剩余预算，再返回不同 actual。

验收

总预留不超 hard limit；多维 actual 原子幂等对账；unknown 不释放且事件完整。

```
type BudgetStore interface {
    Reserve(
        context.Context,
        string,
        map[BudgetDimension]int64,
    ) (ReservationSet, error)
    ReconcileBatch(
        context.Context, string, UsageDigest,
        map[BudgetDimension]int64,
    ) error
    ReleaseProvenUnused(context.Context, string) error
    MarkUnknown(context.Context, string) error
}
```

章内续问 22.2 · 闯祸与群聊

证据正文与签名分开

闯祸现场　证据桌上放着 canonical manifest body、空 envelope 和一把即将轮换的签名 key。
陈平安把 signature 塞回 body 再计算 digest，文件每签一次就改变一次，验证器绕成了循环。

陈平安

signature 也是证据，放进正文一起签最完整。


阿良
它像在照片里举着同一张照片，越拍越小。


宁姚
旧 Run 遇到密钥轮换，按哪份历史信任策略验证？

陈平安
我固定不可变正文，先计算 payload digest。


陈平安
envelope 只签 digest；旧记录按历史策略验并查撤销。



宁姚
body 不含自身签名，envelope 才说明谁签了什么。

收束　空 envelope 合上后，manifest body 再也没有追着自己改。

先有画面，再落到工程

证据正文与签名分开

先懂一句

先规范化不可变 manifest body，再让独立 envelope 签它的 digest。

直觉类比

像把考试答案先排成固定版式、算出指纹，再把指纹装进签名信封。信封可以换签名算法，答案正文不能偷偷跟着变。

术语落地

- Run：从开始到结束的一整趟任务

读代码抓手

- 先看 Verify 的输入和返回值：它接什么，最后必须交回什么。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Verify 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 22.2 · 拆去比喻

证据正文与签名分开

工程要点

- manifest body 固定 schema version、Run、fingerprint、event head 和产物 digest。
- 按明确 canonical format 生成字节，字段重排不能改变被签语义。
- envelope 仅保存 payload digest、algorithm、key ID、signature 和时间。
- 验证先查 trust policy 与撤销状态，再验 payload digest 和签名。
- 签名覆盖 subject digests，不只签一个可替换来源的 URL。

练手

分别篡改 artifact digest、event head、key ID 和 canonical body。

验收

所有篡改均失败；合法旧签名可按历史策略验证；撤销 key 被拒绝。

```go
type SignatureEnvelope struct {
    PayloadDigest string
    Algorithm     string
    KeyID         string
    Signature     []byte
    SignedAt      time.Time
}

func Verify(
    body []byte,
    env SignatureEnvelope,
    trust TrustPolicy,
) error
```

章内续问 22.3 · 闯祸与群聊

三种 Replay，三套承诺

闯祸现场　回放控制台排着事件回放、模拟轨迹和真实重跑三个按钮，陈平安只想把旧页面恢复出来。他却点了真实重跑，ledger.create_bill 差点再次落账；旁边的评测台还换了一套试卷。

陈平安

Replay 不就是把旧任务再执行一次吗？


阿良
想看录像，却又把演员叫回现场开机。


阮秀
只恢复旧状态，为什么还要碰外部工具？

陈平安
事件回放不碰工具；模拟轨迹使用固定的外部输入。



李宝瓶
若要比较新旧结果，哪些规矩不能换？


宁姚
固定试卷、分组、判分、随机种子和门槛，只改声明项。

收束　三个按钮被分开上锁，EvalProtocol 清单留在了最上面。

先有画面，再落到工程

三种 Replay，三套承诺

先懂一句

事件回放、模拟轨迹和真实重跑，是三种不同承诺。

直觉类比

录像回放、拿玩偶排练、真人重新演一次，看着都像‘重放’，承诺却完全不同。比较成绩前还要固定试卷和判分规则，只改变要测试的选手。

术语落地

- Metric：系统现在胖瘦快慢的仪表盘数字
- Tool：真正会读文件、查库或调用服务的手脚
- Run：从开始到结束的一整趟任务
- Eval：用固定试卷和判分规矩比较候选版本

读代码抓手

- 先看 Compare 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Compare 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 22.3 · 拆去比喻

三种 Replay，三套承诺

工程要点

- 可比性先固定评测协议：dataset/split/sample IDs、grader、metric、seed 与 threshold policy。
- model、prompt、tool、policy 与 code 是允许声明变化的 treatment；它们正是 Eval 要比较的对象。
- 未声明的协议漂移返回 ErrNotComparable；结果保存样本引用、分布、置信区间、失败引用和签名。
- event playback 零 Tool；mock trajectory 固定外部输入；real re-execution 新建 Run 与审批。

练手

保持 EvalProtocol 不变，分别改变 model、prompt 与 code treatment 并比较分布；再改变 dataset 或 grader，验证返回 ErrNotComparable。

验收

声明的 treatment 差异可比较；dataset/split/sample、grader、metric、seed 或 threshold policy 漂移会拒绝比较。

```go
type EvalProtocol struct {
    DatasetDigest, SplitDigest, SampleIDsDigest string
    GraderDigest, MetricDigest, SeedsDigest string
    ThresholdPolicyDigest string
}
type Treatment struct {
    ModelDigest, PromptDigest, ToolDigest string
    PolicyDigest, CodeDigest string
}
type EvalManifest struct {
    Protocol EvalProtocol
    Treatment Treatment
}

func Compare(a, b EvalManifest) error {
    if a.Protocol != b.Protocol { return ErrNotComparable }
    return nil // declared treatment differences are what Eval measures
}
```

章内续问 22.4 · 闯祸与群聊

Trace 会丢，事件账本不能丢

闯祸现场　监控墙上只有一条完整 trace，持久事件表却被清得只剩表头，SSE 客户端正拿着旧游标重连。陈平安让客户端按 trace ID 去重，采样刚好漏掉中间 span，同一条状态变化便被重复播放。

陈平安：trace 已经串全程，可以直接当状态账本。



阿良：监控录像会抽帧，账本可不能跟着眨眼。



李宝瓶：SSE 重连认哪一条事件，状态又靠什么重建？

陈平安：我给每个 Run 的事件分配单调 sequence。

陈平安：状态与 outbox 事件同事务落库，span 只引用事件编号。



宁姚：持久事件是主轴；trace 与 SSE 都只是投影。

收束　旧游标沿着 event sequence 接回原位，没有再播第二遍。

先有画面，再落到工程

Trace 会丢，事件账本不能丢

先懂一句

持久事件是审计主轴，trace、SSE 和 metric 都只是投影。

直觉类比

持久事件像学校总值日日志，Trace、SSE 和 Metric 像黑板通知、广播和统计表。黑板可以擦，广播会漏听，总日志不能跟着消失。

术语落地

- 事务：要么整组成功，要么整组撤回
- SSE：服务端向浏览器单向喊话的小喇叭
- Metric：系统现在胖瘦快慢的仪表盘数字
- Trace：一次请求一路经过哪些地方的脚印串
- Tool：真正会读文件、查库或调用服务的手脚
- Run：从开始到结束的一整趟任务
- Step：整趟任务中的一次具体动作
- approval：高风险动作动手前的人类放行条
- outbox：和业务账一起落库、之后再送出的发件箱

读代码抓手

- 先看 EventRecord 的字段和约束：每个字段代表什么，哪些组合才合法。
- 找到稳定身份：重试时哪个号码绝不能换，回执又和谁绑定。
- 最后核对 EventRecord 的生命周期：谁创建、谁更新、怎样拒绝非法状态。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 22.4 · 拆去比喻

Trace 会丢，事件账本不能丢

工程要点

- 状态转换与 outbox event 同事务提交，事件只追加且带单调 sequence。
- 关联 run、step、attempt、tool call、approval 和 capability ID。
- 事件链保存前序 digest；manifest 只签 event head 与必要 subject digest。
- SSE 按 sequence 重连去重；trace span 记录 event ID 而非替代事件。
- 高基数明细留在 ledger，metric 只保存可聚合的延迟、错误和成本。

练手

删除全部 trace 数据，只用事件账本重建一次审批、执行和对账过程。

验收

状态可完整重建；sequence 无缺口或重复；SSE 重连不会重复消费。

```go
type EventRecord struct {
    EventID, RunID, StepID string
    AttemptID, ToolCallID  string
    ApprovalID, CapabilityID string
    SchemaVersion uint32
    Sequence      uint64
    Kind          string
    PayloadRef    string
    PayloadDigest string
    PreviousDigest string
    CurrentDigest  string
    CreatedAt      time.Time
}

// CurrentDigest covers canonical metadata and PayloadDigest.
```

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 为什么不能无限 Agent loop？

陈平安



无限 loop 会穿透 token、费用、Tool 次数和墙钟预算，也会重复未知副作用。



宁姚

对，hard limit 阻止新调用。

第 2 问 dry-run replay 如何避免副作用？

陈平安



event playback 不执行 Tool，mock trajectory 用固定替身；真实重跑才新建 Run 和审批。



宁姚

对，三种 Replay 不能共用承诺。

第 3 问 成功率必须说明哪些条件？

陈平安



成功率要说明样本、版本、采样次数、分布、置信区间和阈值。



宁姚

对，单次通过不能证明质量。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 23 场 · 闯祸与群聊

第二十三章 AI 写代码也要过剑阵：Coding Harness

目标：把 spec、上下文发现、受限编辑、构建测试、review、重试熔断与人工合并连成编码闭环。

闯祸现场　用户的 dirty 仓库里留着未提交文件和暂存内容，旁边的独立工作区标签还空着 candidate@base。陈平安直接改了宿主仓库，又想用 git reset 收尾，暂存区里属于用户的一段修改当场消失。

陈平安



失败后 reset，仓库就能回到原样。



左右

你恢复的是提交，不是用户尚未提交的字节。



李宝瓶

candidate 基于哪个 base，合并证据又绑定什么？

陈平安

我在隔离 candidate 中编辑，并记下补丁摘要。



陈平安

先检查格式、编译和测试，再请 Reviewer 对证据。





宁姚

基线前进就重建并重验；失败只丢 candidate。

收束　宿主仓库重新合上，只有 candidate 工作区亮着验证灯。

先有画面，再落到工程

第二十三章 AI 写代码也要过剑阵：Coding Harness

先懂一句

先立约，再落笔；机器验过，人来定夺。

直觉类比

AI 改代码像让一个手速很快的实习生修钟表：先在副本上动手，再跑测试、静态检查和评审；不能失败后把主人的桌面一起掀了。

术语落地

- candidate：和用户工作区隔开的候选改动副本
- base：候选改动开始时所依据的提交版本
- Context：沿调用链传下去的取消铃和截止钟

读代码抓手

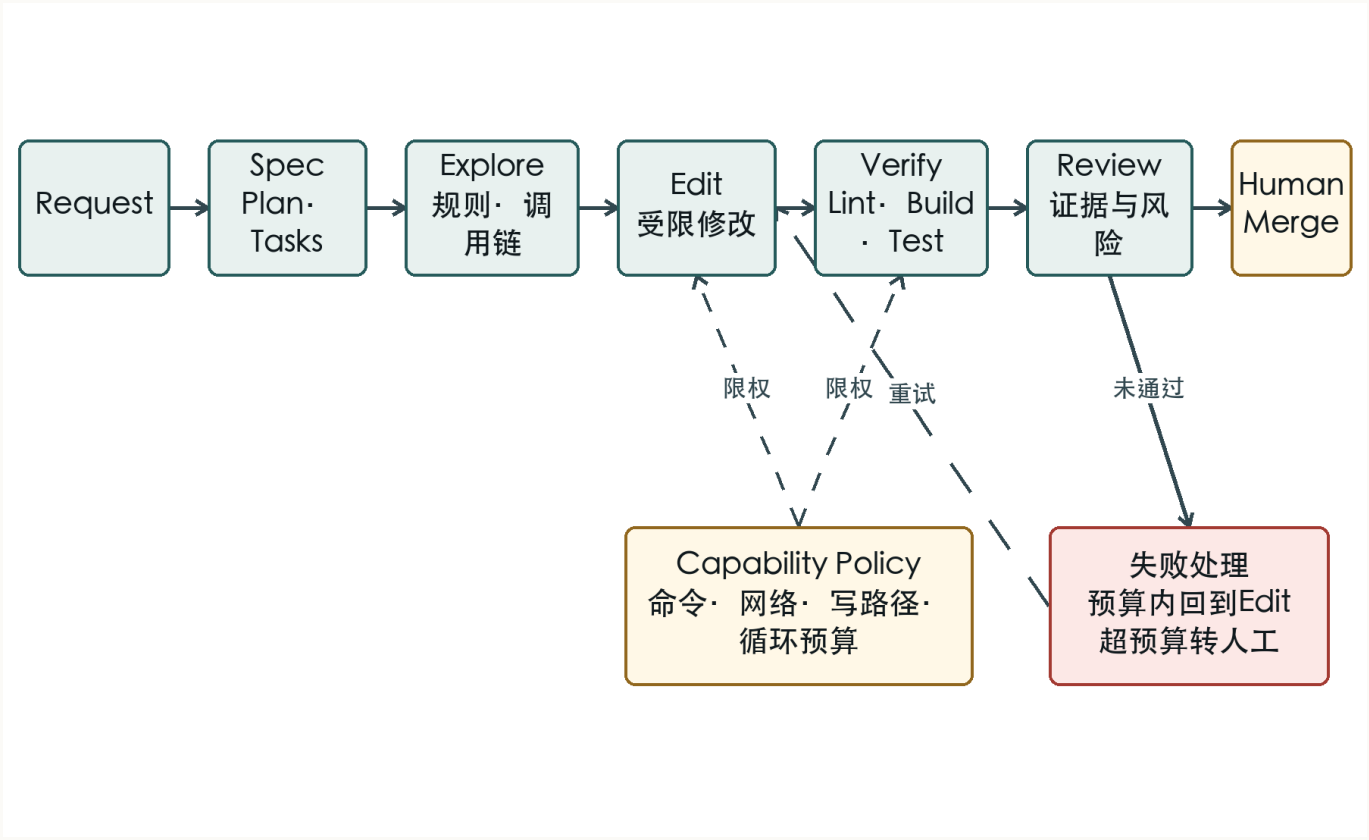
- 先看 CodingCapability 的字段和约束：每个字段代表什么，哪些组合才合法。
- 最后核对 CodingCapability 的生命周期：谁创建、谁更新、怎样拒绝非法状态。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 23.1

Coding Harness 的发现、编辑、验证与审批闭环

小白先懂：先立约，再落笔；机器验过，人来定夺。



拆去比喻

- 先写 spec/plan/tasks，明确用户可见行为、接口、数据、安全边界和验收证据。
- Context discovery 读取规则、调用链、消费者、测试和运行证据，不把整个仓库无脑塞给模型。
- 工具授权限制 command、args、cwd、env、network、进程树、时间、输出、写路径与 diff。
- 不可信构建测试运行在隔离环境；同错指纹和预算触发人工接管。
- 生产发布、主干合并和高风险写操作仍由 branch protection 与独立人工审批决定。

从图落到代码，再用证据收剑

代码只保留本章必须掌握的骨架，失败路径与验收条件紧跟在后。

```go
type CodingCapability struct {
    AllowedCommands []CommandRule // args, cwd, env and network
    WritablePaths   []string
    MaxLoops, MaxProcesses int
    MaxCPU, MaxBytes, MaxDiff int64
}

// Run untrusted build/tests in isolation.
// Branch protection and human merge stay external.
```

失败路径

- 需求不清便直接改代码
- 测试失败就无界自修复
- 让模型持有生产凭证或直接合并主干

练剑任务

为真实小改动定义 CodingCapability，在隔离环境执行构建测试并触发超预算人工接管。

验收证据

越权参数/路径/网络被拒绝；每轮有 diff、测试和 review 证据；机器不能绕过 branch protection 合并。

收剑自检

- [ ] 何时应先 Explore？
- [ ] 哪些步骤必须确定性执行？
- [ ] 什么条件触发人工接管？

章内续问 23.1 · 闯祸与群聊

候选改动必须离开宿主工作区

闯祸现场

隔离工作区里摆着 base revision、rules digest、sandbox image 和 patch digest 四张标签，宿主仓库仍保持 dirty。

陈平安验证完成后发现 base 已前进，却想拿旧 evidence 直接 merge，新提交与旧补丁当场打架。

陈平安



测试已经通过，base 前进也能直接合并。



阿良

地基换了，旧验收单不能替新房签字。



李宝瓶

一个 candidate 要绑定哪次 Run、哪步动作和哪次尝试？

陈平安

我把候选改动绑定整趟任务、当前动作和这次尝试。



陈平安

底稿、补丁摘要和验证证据也随状态落库。





宁姚

旧 fence 或证据错配都拒绝 merge。

收束

旧验收单被夹进审计册，新 candidate 从干净 base 重新起步。

先有画面，再落到工程

候选改动必须离开宿主工作区

先懂一句

每次编辑落在 candidate@base；失败只 discard 候选，不碰宿主状态。

直觉类比

像先复印一张涂色页给 AI 改。涂坏了丢复印件，原画和桌上没收好的彩笔都不碰；验证通过后，仍由主人决定要不要贴回画册。

术语落地

- 事务：要么整组成功，要么整组撤回
- Trace：一次请求一路经过哪些地方的脚印串
- Harness：管住模型手脚、权限、预算和证据的外壳
- Run：从开始到结束的一整趟任务
- Step：整趟任务中的一次具体动作
- approval：高风险动作动手前的人类放行条
- sandbox：把程序关进权限和资源都受限的练功房
- candidate：和用户工作区隔开的候选改动副本
- base：候选改动开始时所依据的提交版本

读代码抓手

- 先看 Transition 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 最后核对 Transition 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 23.1 · 拆去比喻

候选改动必须离开宿主工作区

工程要点

- Candidate 绑定 Run/Step/Attempt、event head、base/rules/sandbox/patch digest、验证 manifest、review 与 merge approval。
- edit、verify、review、approve、merge/discard 每次状态转换与 durable event 在同一事务 CAS 落地。
- merge 前重验 base、patch、evidence、approval、fence 与目标分支保护；旧 base 或旧 fence 一律失败。
- trace 全丢仍可从事件链重建候选状态；取消或失败只 discard harness worktree，宿主 dirty 文件和 index 字节不变。

练手

在 edit/verify/review/approve 各切点崩溃，再篡改 patch/test evidence，并制造 base 前进与 lease takeover。

验收

事件可重建候选；篡改证据、旧 base、旧 fence 都拒绝 merge；宿主工作区和 index 字节级不变。

记忆钉

1. Candidate 绑定 Run/Step/Attempt、event head、base/rules/sandbox/patch digest、验证 manifest、review 与 merge approval。
2. edit、verify、review、approve、merge/discard 每次状态转换与 durable event 在同一事务 CAS 落地。
3. merge 前重验 base、patch、evidence、approval、fence 与目标分支保护；旧 base 或旧 fence 一律失败。
4. trace 全丢仍可从事件链重建候选状态；取消或失败只 discard harness worktree，宿主 dirty 文件和 index 字节不变。

长代码已拆到后续专页

章内续问 23.1 · 代码 1/1

候选改动必须离开宿主工作区

代码按可读字号拆页；这是教学骨架，完整依赖与故障注入以练手和验收契约为准。

```
type Candidate struct {
    ID, RepoID, RunID, StepID, AttemptID string
    BaseRevision, RulesDigest, SandboxImageDigest string
    PatchDigest, VerificationManifestDigest string
    ReviewDigest, ApprovalID, EventHead string
    WorktreeRef string
    Fence Fence
    Version int64
    State CandidateState
}

func (h *CodingHarness) Transition(
    ctx context.Context, c Candidate, next CandidateState,
    evidence EvidenceManifest,
) error {
    if evidence.PatchDigest != c.PatchDigest {
        return ErrEvidenceMismatch
    }
    expected := ExpectedState{
        Fence: c.Fence, StepVersion: c.Version,
    }
    return h.store.CASAndAppendEvent(
        ctx, c.ID, expected, next, evidence)
}
```

事件可重建候选；篡改证据、旧 base、旧 fence 都拒绝 merge；宿主工作区和 index 字节级不变。

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 何时应先 Explore？

陈平安



需求入口、调用链或用户工作区状态不明时先 Explore。



宁姚

对，证据不足时不直接编辑。

第 2 问 哪些步骤必须确定性执行？

陈平安



format、lint、build、test 和 digest 校验由确定性工具执行。



宁姚

对，Reviewer 只依据契约与证据判定。

第 3 问 什么条件触发人工接管？

陈平安



同错重复、预算耗尽、越界、base 变化或高风险合并都转人工。



宁姚

对，机器不能绕过分支保护。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

第 24 场 · 闯祸与群聊

第二十四章 最后一炉：可控 Ledger Agent

目标：把 Go 服务、数据库、流式运行、Tool/MCP、权限、HITL、trace 与 eval 接成完整系统。

闯祸现场

网络超时后，客户端又递来 entry-0827，桌上并排摆着 run-0827、req-0827 和 receipt-req-0827 三张卡。

陈平安只按 entry key 查旧 Run，险些把同租户另一位 owner 的请求当成同一次意图。

陈平安



key 相同就返回旧 Run，应该足够幂等。



阿良

号码相同，柜台也不能把别人的取件递给你。



阮秀

tenant、owner、动作或摘要不同，分别怎样处理？

陈平安

同作用域同摘要返回 run-0827，异摘要回 409。



陈平安

旧 Run 先验主人权限，再用原 CallID 查回执。





宁姚

req-0827 只落 bill-0827；网络断开不取消后台任务。

收束

三张卡首尾对齐，重试请求终于没有复制出第二本账。

先有画面，再落到工程

第二十四章 最后一炉：可控 Ledger Agent

先懂一句

模型可以不稳，剑匣必须可控、可测、可恢复、可审计。

直觉类比

最终系统像带护栏的自动账房：Run 管整趟目标，Step 管一次动作，CallID 固定副作用身份；approval、预算和 receipt 决定能否动手、何时结算。

术语落地

- 入口身份：tenant、owner 与 entry key 决定请求作用域
- 执行身份：Run/Step 记录目标与动作，CallID 固定副作用
- 结果证据：receipt 与 outbox 证明结果和事件都已落地
- 恢复规则：Unknown 先对账，重复调用不能再落一笔

读代码抓手

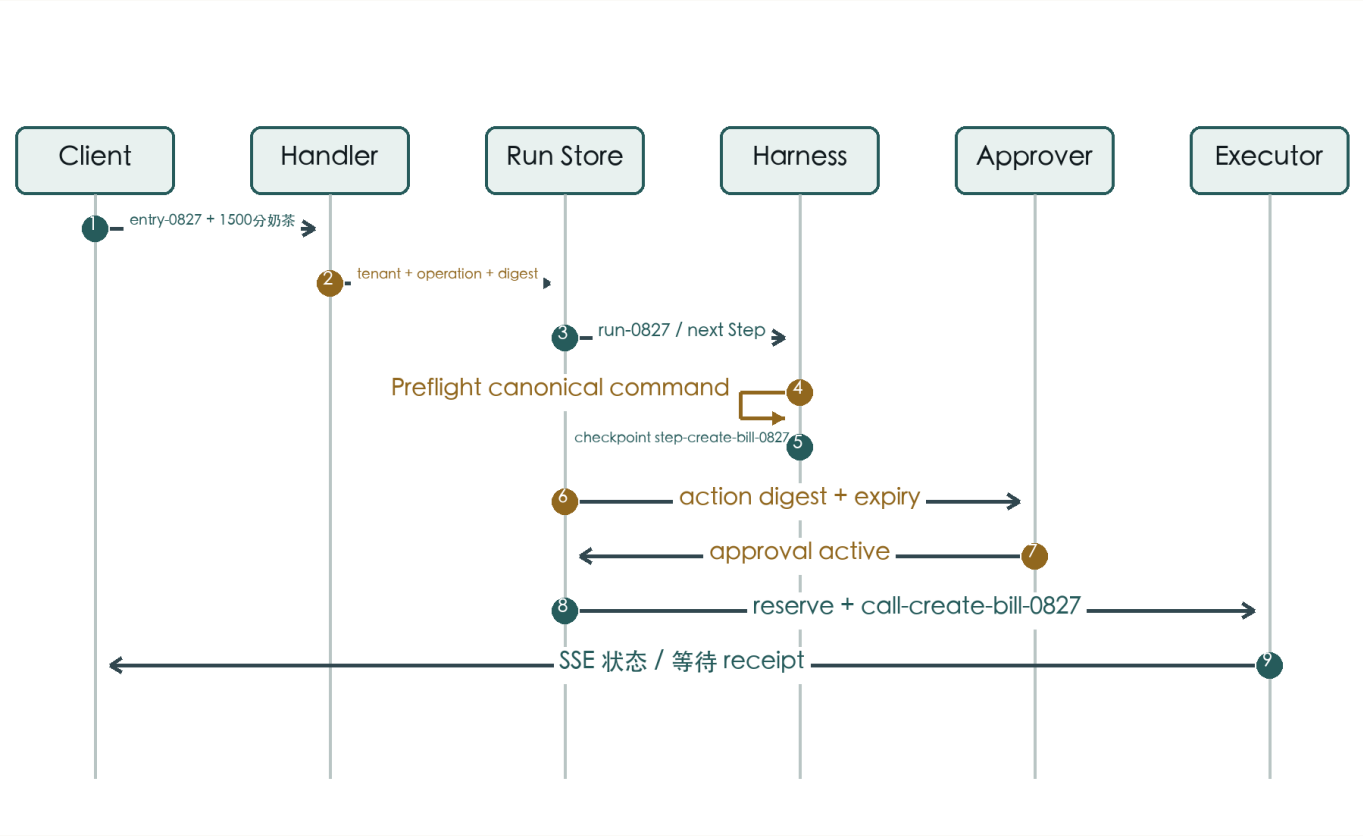
- 先看 StartRun 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 找到稳定身份：重试时哪个号码绝不能换，回执又和谁绑定。
- 最后核对 StartRun 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

图 24.1

Ledger Agent 主路径：从输入、草稿到审批

小白先懂：模型可以不稳，剑匣必须可控、可测、可恢复、可审计。



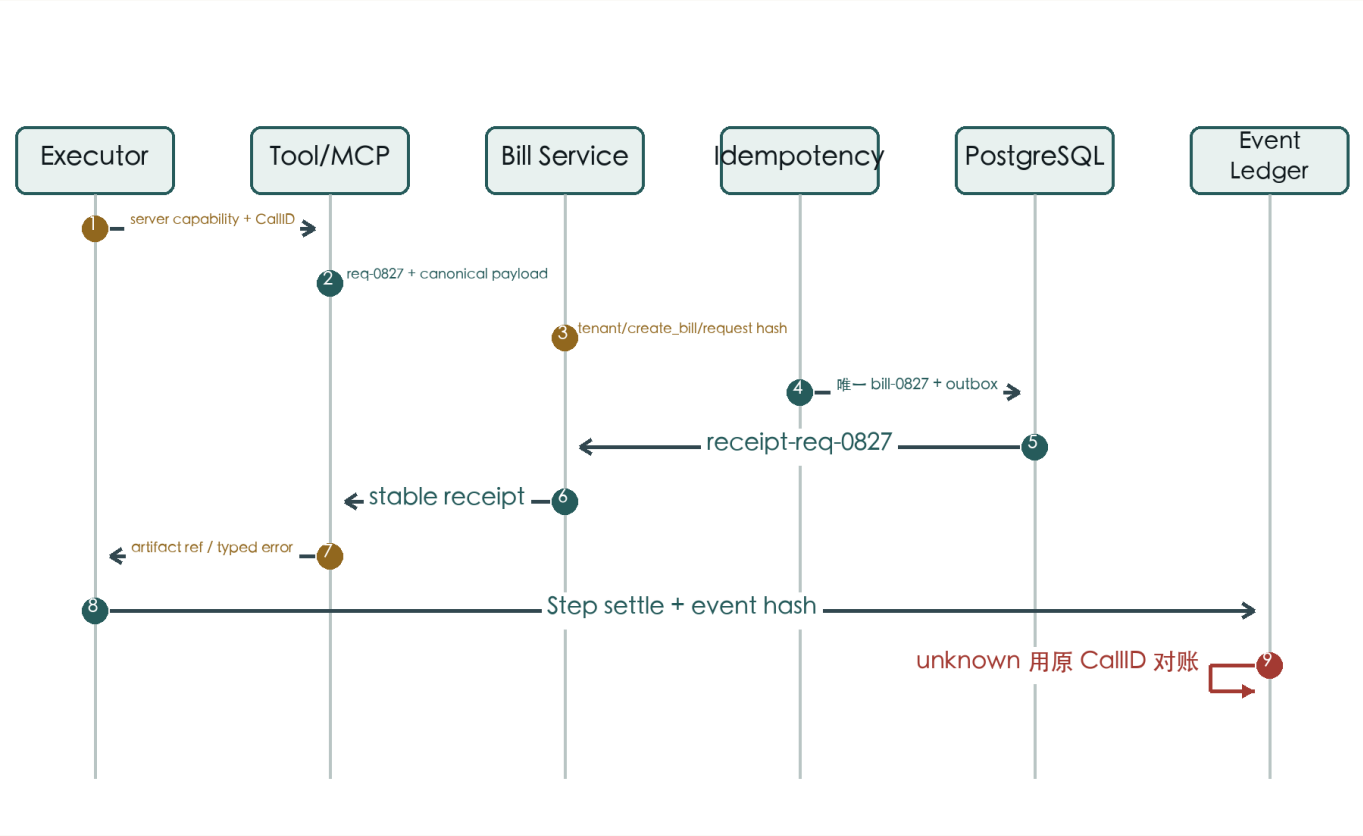
拆去比喻

- 入口唯一约束是 tenant_id + authenticated owner_id + operation + entry_key；首次写入同时保存双摘要与 visibility。
- 同一 owner 作用域内，entry-0827 同摘要返回 run-0827；同 key 不同输入或 profile 返回 409 且零副作用。
- 命中已有 Run 仍按当前 Principal 做对象授权；同租户不同 owner 与跨租户都不能读取或接管旧 Run。
- step-create-bill-0827 固定 call-create-bill-0827，并在可信 Executor 内映射下游 idempotency key req-0827。
- req-0827 只创建 bill-0827；响应丢失后按 receipt-req-0827 对账，不重新执行 create_bill。

图 24.2

Ledger Agent 写入路径：幂等、事务与证据

小白先懂：模型可以不稳，剑匣必须可控、可测、可恢复、可审计。



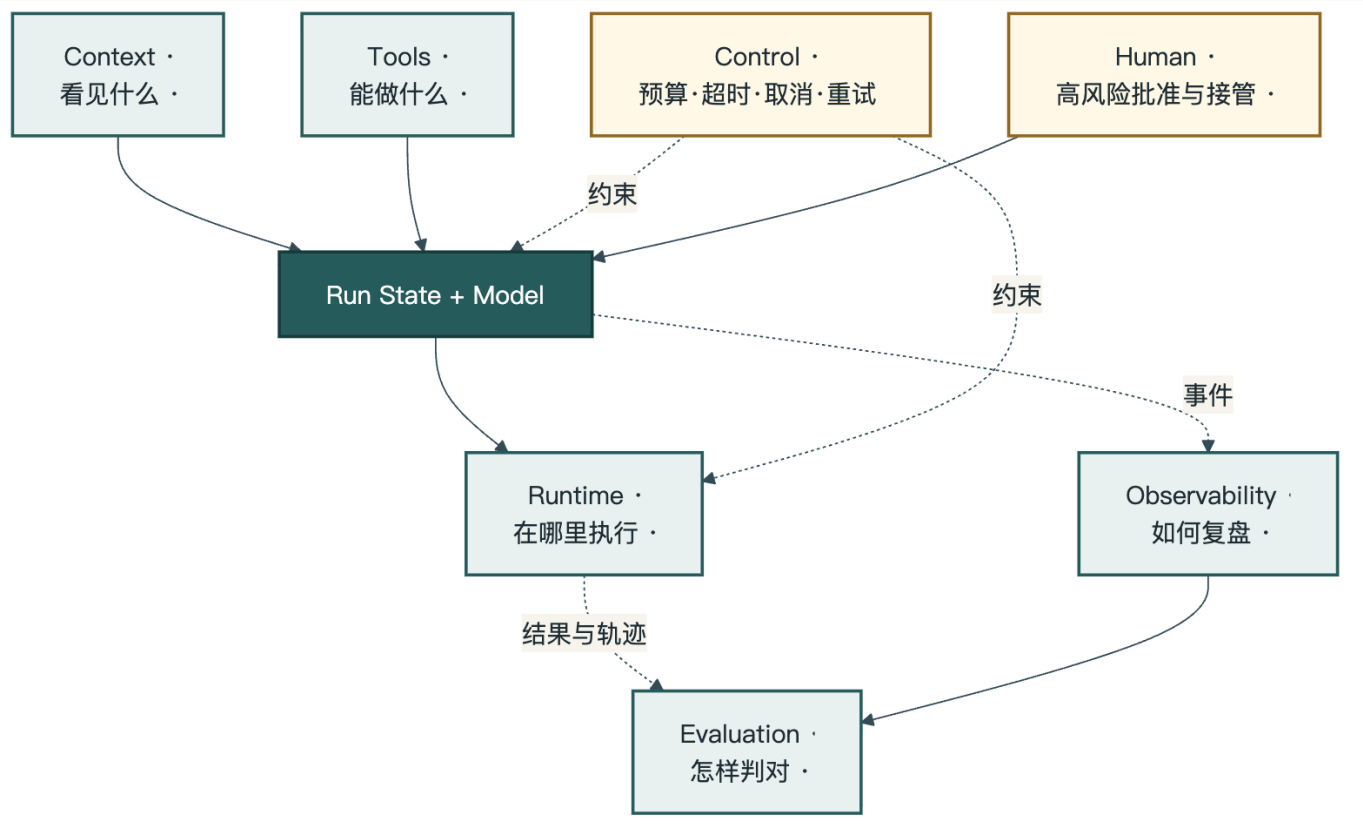
拆去比喻

- Principal 与批准命令沿 Handler、Service、幂等和事务链保持不变。
- 提交后失效缓存；trace、签名 manifest 与 eval 组成可复核证据。

图 24.3

回顾 · Harness AI 工程的七个控制域

小白先懂：模型可以不稳，剑匣必须可控、可测、可恢复、可审计。



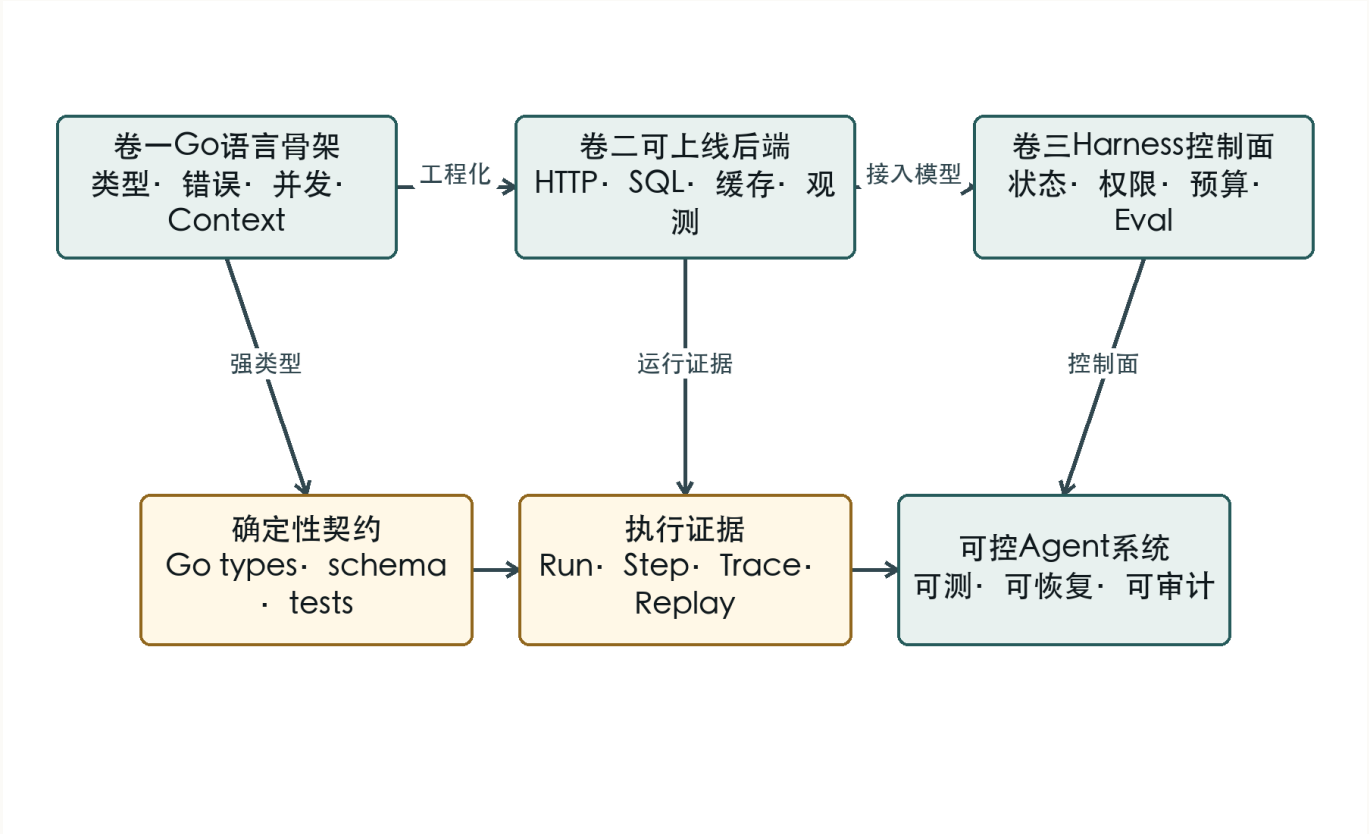
拆去比喻

- 七个控制域共同约束模型；任何单一工具都不能替代完整 Harness。
- 权限、预算、恢复与证据由确定性系统执行，不交给提示词自律。

图 24.4

回顾 · 全书能力地图：从 Go 骨架到 Harness 控制面

小白先懂：模型可以不稳，剑匣必须可控、可测、可恢复、可审计。



拆去比喻

- 学习顺序从语言边界走向后端可靠性，再进入 Agent 控制面。
- 每一阶段都以失败收口、状态恢复、权限拒绝和证据复核为门槛。

代码完整展开，不牺牲阅读字号

小白先懂：模型可以不稳，剑匣必须可控、可测、可恢复、可审计。

```go
func (h *Handler) StartRun(
    ctx context.Context, p Principal, key EntryKey,
    input CreateBillInput, profile ExecutionProfile,
) (Run, error) {
    entry := CanonicalEntry{
        TenantID: p.TenantID, OwnerID: p.SubjectID,
        Operation: "create_bill", Key: key,
        InputDigest: input.Digest(), ProfileDigest: profile.Digest(),
        Visibility: OwnerVisible,
    }
    // Unique scope: tenant + authenticated owner + operation + key.
    run, created, err := h.runs.CreateQueuedOnce(ctx, entry, input)
    if errors.Is(err, ErrDigestConflict) {
        return Run{}, ErrIdempotencyConflict // only this owner's key
    }
    if err != nil { return Run{}, err }
    if !created && !h.policy.CanReadRun(p, run) {
        return Run{}, ErrNotFound
    }
    return run, nil
}
```

下一页核对失败路径、练剑任务、验收证据与自检问题。

失败要能复现，完成要有证据

失败路径

- 为了 Agent 化而 Agent 化
- 验收只看演示
- 上线后没有失败闭环

练剑任务

设计练习：为同/异 owner 复用 entry-0827、同 owner 改摘要、commit 后丢 receipt 三种场景画时序与验收表。

验收证据

设计评审只检查图中是否标出：同 owner 同摘要复用、异摘要冲突、不同 owner 隔离、原 CallID 对账；实现仍须唯一约束与故障测试。

收剑自检

- [ ] dispatch 前为何先持久化 reservation 与 checkpoint？
- [ ] 缓存失效为何在 commit 后？
- [ ] 上线门禁由哪些证据组成？

章内续问 24.1 · 闯祸与群聊

Handler 只入队，Worker 才执行

闯祸现场　HTTP Handler 的计时条只剩一秒，桌上却还没有 queued Run，Worker 的 lease 卡也空着。陈平安在 Handler 里直接调用模型，请求超时后 run-0827 没有落库，客户端重试又创建了一份工作。

陈平安

Handler 直接执行少一层，超时后重试就好。


阿良
门铃一断，屋里的工作不能跟着失忆。


阮秀
请求 Context 断开时，持久 Run 应该由谁继续推进？

陈平安

Handler 先验租户和主人，再按双摘要 CreateQueuedOnce。

陈平安

入口只建排队中的 Run；后台执行者按栅栏推进 Step。


宁姚
Unknown 用原 CallID 对账；网络断开不等于取消。

收束　计时条归零时，run-0827 已稳稳躺在 queued 列表里。

先有画面，再落到工程

Handler 只入队，Worker 才执行

先懂一句

Handler 只创建持久 queued Run，lease Worker 才推进状态机。

直觉类比

像餐厅前台和后厨：前台先把订单写进不会丢的队列并给号码，后厨拿到租约后才开火。客人挂电话，订单不会凭空蒸发。

术语落地

- 幂等：同一件事喊两遍，也只真正办一次
- Run：从开始到结束的一整趟任务
- Step：整趟任务中的一次具体动作
- CallID：同一步副作用永远不变的取件号
- receipt：外部系统办完事后给回来的回执
- tenant：同一系统里互相不能偷看账本的一户人家
- owner：这份资源真正归属的那个人或主体
- outbox：和业务账一起落库、之后再送出的发件箱

读代码抓手

- 先看 Process 的输入和返回值：它接什么，最后必须交回什么。
- 再找守门条件：哪些坏输入会被立刻请出去。
- 看到 ctx 就问：取消和超时有没有真的传到下一层。
- 看到 err 不要跳过：它有没有被识别、包裹或翻译，而不是悄悄吞掉。
- 找到稳定身份：重试时哪个号码绝不能换，回执又和谁绑定。
- 最后核对 Process 的出口：成功交回什么，失败留下什么证据。

本节路线　接下来先拆去比喻，核对本单元的技术真相、代码、失败路径和验收证据。

章内续问 24.1 · 拆去比喻

Handler 只入队，Worker 才执行

工程要点

- CreateQueuedOnce 的唯一键为 tenant-7/user-7/create_bill/entry-0827，并持久化双摘要与 visibility。
- 同一 owner 作用域下，同 key 同摘要返回 run-0827；不同摘要返回 409 且零副作用；不同 owner/operation/tenant 不碰撞。
- 命中旧 Run 仍执行对象级授权；同 tenant 不同 owner 与跨 tenant 都不能读取或接管。
- step-create-bill-0827 的稳定 CallID 是 call-create-bill-0827；可信 Executor 将它映射为下游 req-0827。
- req-0827 由账本服务幂等落 bill-0827/outbox，并提供 receipt-req-0827；响应丢失只 reconcile。

练手

并发提交 entry-0827 的同摘要与冲突摘要，并在 send、commit、receipt、settle 切点终止 Worker。

验收

同摘要只建 run-0827；冲突摘要 409、零副作用；跨作用域不碰撞；不可见 Run 不泄露；重复 Step 只落一个 bill/outbox，receipt 可恢复。

```go
type EntryContract struct {
    TenantID, OwnerID, Operation, Key string
    InputDigest, ProfileDigest, Visibility string
}

func (w *Worker) Process(ctx context.Context, runID string) error {
    lease, run, err := w.runs.Claim(ctx, runID, w.workerID)
    if err != nil { return err }
    defer lease.StopHeartbeat()
    step, err := w.steps.Next(ctx, run.ID, lease.Fence)
    if err != nil { return err }
    if step.State.NeedsReconcile() {
        return w.executor.Reconcile(
            ctx, step.ID, step.CallID, lease.Fence, step.Version)
    }
    return w.executor.ExecuteStep(
        ctx, lease.Fence, step.Version, step)
}
```

问题不悬空

先自己答，再看群聊如何收剑

每题都给出学习者答案和导师判定；先核对边界，再记标准句。

第 1 问 dispatch 前为何先持久化 reservation 与 checkpoint？

陈平安



先校验 active approval，再于 dispatch 前原子写 approval reservation、预算 reservation、版本/fence 与事件。



宁姚

请求不能先发，账随后再补。

第 2 问 缓存失效为何在 commit 后？

陈平安



数据库 commit 后再失效缓存，避免失败事务先删缓存后又重建旧值。



宁姚

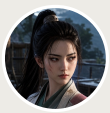
缓存只是投影。

第 3 问 上线门禁由哪些证据组成？

陈平安



上线门禁要有入口幂等、唯一副作用、receipt 对账、取消状态、事件重建、Eval 和人工批准证据。



宁姚

能演示不等于闭环。

答得出关系，再去记术语；能说清失败路径，才算真正收剑。

随身剑谱

九十日图解练剑路线

01-15	函数、类型、struct、slice、map、interface
16-30	error、goroutine、channel、context、race
31-45	HTTP、分层、SQL、事务、幂等、Redis
46-60	SSE、可观测、Docker、迁移与回滚
61-75	Tool、MCP、Run、Context、安全、HITL
76-90	Trace、重放、Eval、Coding Harness、红队

终点不是会演示的 Demo，而是有状态、有边界、有证据、有退路的执行链。

Go 管确定性边界，Harness 管模型不确定性，Trace 与 Eval 贯穿两者。

每十五日必须留下证据

不是读到哪里，而是能交付什么

01-15	Go 命令行账本 table test 通过；金额守恒；错误可识别
16-30	并发导入器 go test -race 通过；取消后 goroutine 收口
31-45	账单 HTTP API 契约测试；并发幂等；租户隔离
46-60	可灰度服务 trace 可串联；迁移演练；回滚记录
61-75	受控 Ledger Agent 审批、预算、权限拒绝与恢复用例
76-90	Coding Harness 候选 diff、验证日志、review 与人工闸门

阶段门：功能能跑只是起点；失败能收口、状态能恢复、权限能拒绝、证据能复核，才进入下一阶段。